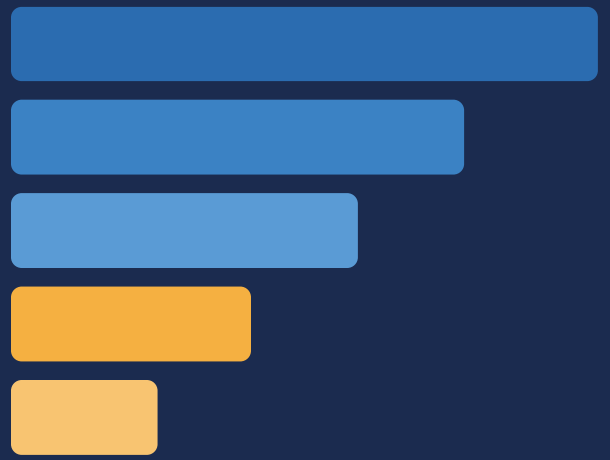
BigQuery Masterclass

Google Cloud’s serverless data warehouse, dissected layer by layer, with the full GoogleSQL language taught through worked examples and annotated outputs.

Architecture to BigQuery ML
Every query shown with its output
One sample dataset throughout



BEFORE WE BEGIN

How this tutorial works

Who it is for

Analysts, data engineers, data scientists and machine learning engineers who want to understand BigQuery deeply: not only what to type, but why the engine behaves as it does and what every query costs.

How it is organised

Twenty six parts move from architecture, to the full GoogleSQL language, to table design, performance, cost, security, machine learning and a capstone project. Each part opens with a dark divider slide listing its topics.

How to read code slides

Dark panel: the query

White table: the output

Amber band: what the output means

All outputs in Parts 6 to 16 are computed from the small sample dataset introduced on the next slides, so you can check every number by hand. Machine learning outputs are marked as illustrative.

LEARNING PATH

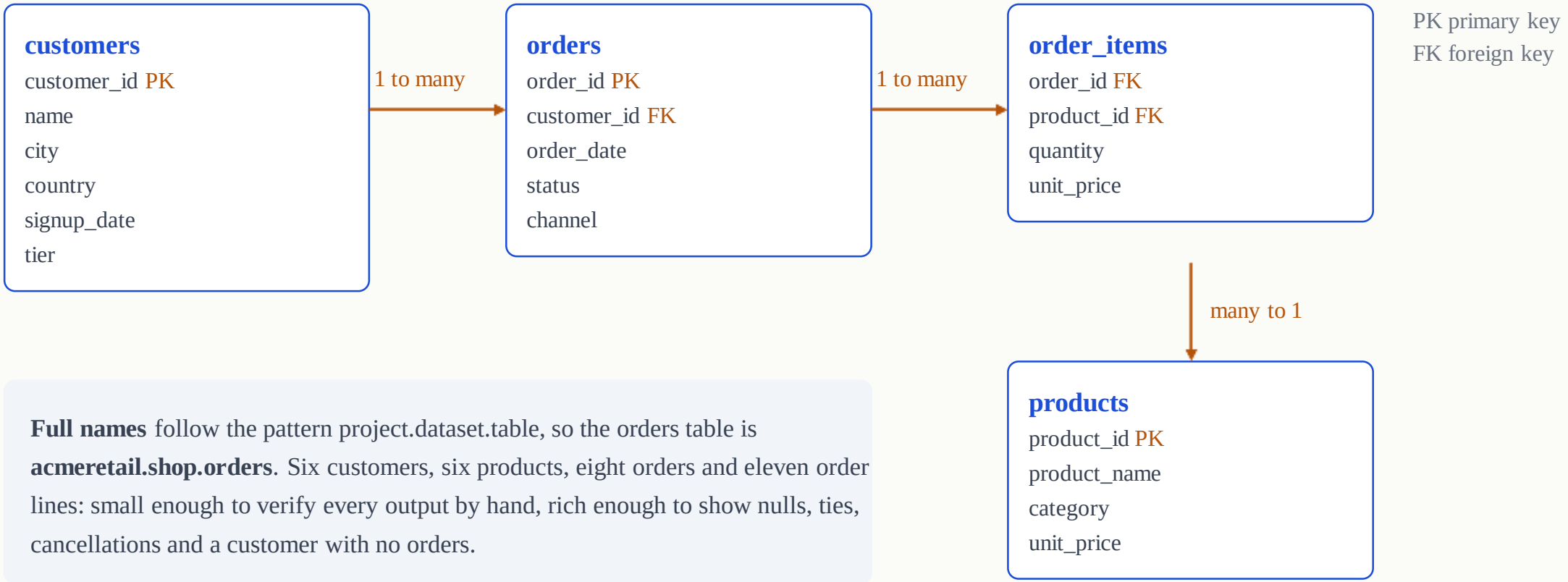
Twenty six parts, one continuous journey

00 Opening and dataset	**01** Foundations	**02** Resource model	**03** Working with BigQuery	**04** Data types	**05** Data in and out	**06** SQL foundations
07 Aggregation	**08** Joins	**09** Subqueries and CTEs	**10** Window functions	**11** Strings and regex	**12** Dates and times	**13** Nested data
14 JSON, GIS, pipes	**15** DDL	**16** DML and MERGE	**17** Partitioning and clustering	**18** Performance	**19** Pricing and cost	**20** Scripting and UDFs
21 Security	**22** BigQuery ML and AI	**23** Ecosystem	**24** Capstone project	**25** Reference and close		

Navy platform **Blue** the SQL language **Brown** engineering and cost **Green** governance, ML and practice

THE SAMPLE DATASET

acmeretail.shop: four related tables



Full names follow the pattern project.dataset.table, so the orders table is **acmeretail.shop.orders**. Six customers, six products, eight orders and eleven order lines: small enough to verify every output by hand, rich enough to show nulls, ties, cancellations and a customer with no orders.

THE SAMPLE DATASET

customers and products

CUSTOMERS

id	name	city	country	signup_date	tier
1	Amara Okafor	Manchester	UK	14 Mar 2025	Gold
2	Ben Clarke	London	UK	2 Jun 2025	Silver
3	Chloe Martin	Paris	France	21 Jul 2025	Gold
4	David Osei	Accra	Ghana	30 Sep 2025	Bronze
5	Elena Rossi	Milan	Italy	11 Nov 2025	Silver
6	Farid Haddad	Manchester	UK	20 Jan 2026	Bronze

PRODUCTS

id	product_name	category	unit_price
101	Laptop Pro 14	Electronics	1200.00
102	Wireless Mouse	Electronics	25.00
103	Desk Chair	Furniture	180.00
104	Standing Desk	Furniture	450.00
105	Coffee Beans 1kg	Grocery	18.50
106	Headphones	Electronics	300.00

Notes. signup_date is a DATE column; it is printed in long form here only for readability. Prices are in pounds sterling stored as NUMERIC. Customer 6, Farid, has never ordered: watch for him in the LEFT JOIN and anti join examples.

THE SAMPLE DATASET

orders and order_items

ORDERS

order_id	customer_id	order_date	status	channel
5001	1	5 Jan 2026	delivered	web
5002	2	9 Jan 2026	delivered	app
5003	1	14 Feb 2026	delivered	app
5004	3	20 Feb 2026	cancelled	web
5005	4	2 Mar 2026	delivered	web
5006	5	15 Mar 2026	shipped	app
5007	3	28 Mar 2026	delivered	web
5008	2	3 Apr 2026	shipped	web

ORDER_ITEMS

order_id	product_id	quantity	unit_price
5001	101	1	1200.00
5001	102	2	25.00
5002	105	4	18.50
5003	106	1	300.00
5004	104	1	450.00
5005	103	2	180.00
5005	105	2	18.50
5006	101	1	1200.00
5007	104	1	450.00
5007	102	1	25.00
5008	106	2	300.00

Line revenue = quantity × unit_price. Grand total of all lines: £4,746.00. Excluding the cancelled order 5004: £4,296.00. Keep these two numbers in mind.

01

Foundations

What BigQuery is, the problem it was built to solve, and the machinery underneath: Colossus storage, the Dremel engine, the Jupiter network and the slot.

Warehouses versus databases · Columnar storage · History · Serverless model · Architecture · Execution tree · Shuffle · Slots · Query lifecycle · Jobs · Result caching

PART 1 · FOUNDATIONS

What BigQuery is

BigQuery is Google Cloud's fully managed, serverless, columnar data warehouse that runs SQL over gigabytes to petabytes by spreading each query across thousands of machines, while storage and compute scale independently.

Serverless

No clusters, nodes, indexes or vacuuming to manage. You submit SQL; Google provisions the compute.

Columnar

Each column is stored and read separately, so a query pays only for the columns it touches.

Massively parallel

A query is cut into stages and thousands of work units that run at the same time.

Standard SQL

GoogleSQL follows the ANSI standard and adds arrays, structs, JSON, geography and ML.

Transactional databases versus analytical warehouses

Dimension	OLTP database (Cloud SQL, Postgres)	OLAP warehouse (BigQuery)
Typical question	Fetch or update order 5001	Revenue by month for five years
Rows touched	One to a few, by key	Millions to billions, by scan
Storage layout	Row oriented	Column oriented
Writes	Many small, concurrent	Large batches or streams
Speed goal	Milliseconds per transaction	Seconds over huge data
Schema style	Highly normalised	Denormalised, nested allowed
Indexes	Central to performance	Mostly unnecessary; partitions and clusters instead
Scaling	Bigger machine or replicas	Automatic, thousands of workers

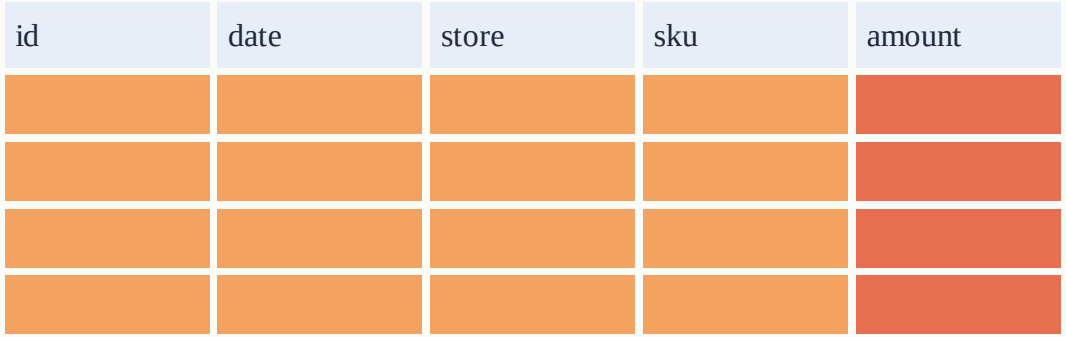
Rule of thumb. Keep your application running on an OLTP database and copy its data into BigQuery for reporting, analytics and machine learning. BigQuery supports DML, but it is not designed to serve thousands of single row updates per second.

PART 1 · FOUNDATIONS

Why columnar storage changes everything

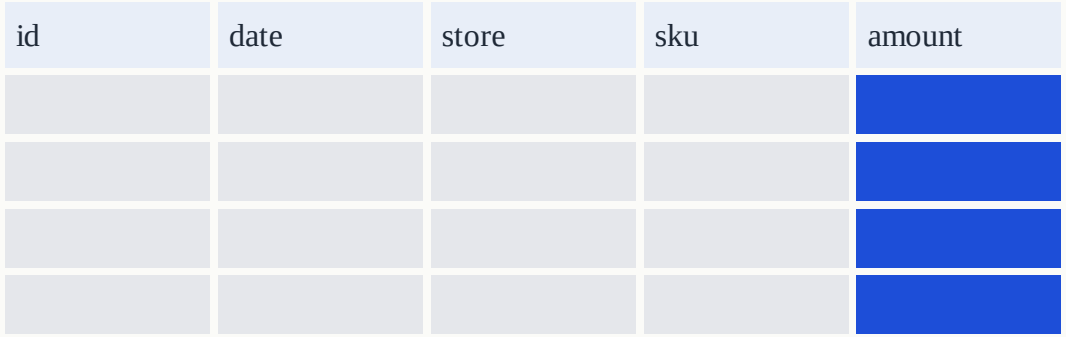
Query: **SELECT SUM(amount) FROM sales** on a table with five columns. Shaded cells are what the engine must read.

Row store: reads every column



Rows are stored contiguously, so fetching one value drags the whole row off disk. **Bytes read: 100% of the table.**

Column store: reads one column



Each column lives in its own files, so the engine opens only amount. **Bytes read: roughly one fifth.**

Two bonuses. Values in one column share a type and often repeat, so they compress far better. And under on demand pricing you are billed for bytes read, so selecting fewer columns directly lowers the bill.

PART 1 · FOUNDATIONS

From a research paper to an AI data platform

2010
Google publishes the Dremel paper and announces BigQuery.

2016
Standard SQL, today called GoogleSQL, becomes the recommended dialect.

2018
BigQuery ML brings model training into SQL.

2020
BigQuery Omni extends querying to data held in other clouds.

2023
Editions with autoscaling slots replace flat rate pricing.

2025
Pipe query syntax reaches general availability in February.

2026
Conversational analytics GA in June; new legacy SQL use limited from 1 June.

The direction of travel is clear: from fast SQL over big data, to a unified platform for analytics, governance and AI.

PART 1 · FOUNDATIONS

What serverless really means: the split of responsibility

You own

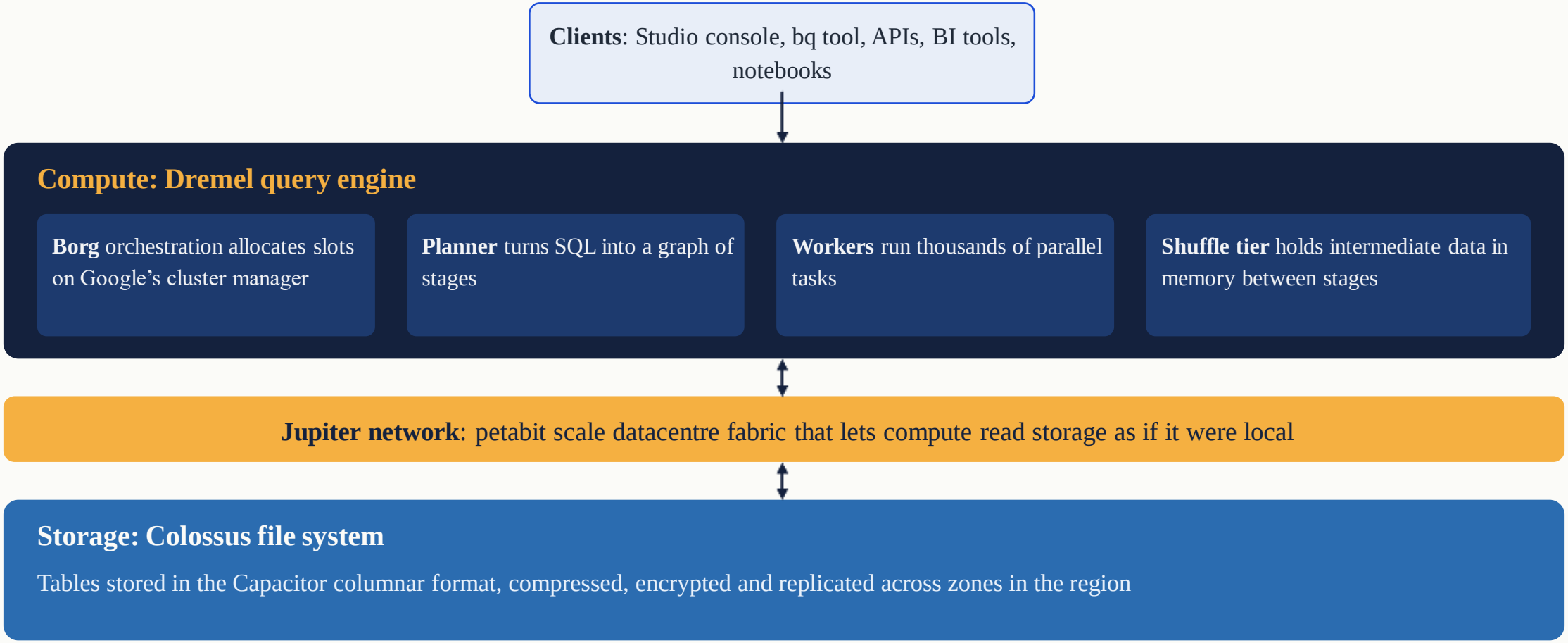
- The SQL you write and its efficiency
- Dataset and table design: schema, partitioning, clustering
- Who can see what: IAM and policies
- Choosing a pricing model and setting cost guards
- Data quality and pipeline logic

Google owns

- Servers, disks, operating systems, patching
- Replication, durability and availability
- Query planning, scheduling and scaling to thousands of workers
- Storage format, compression and file compaction
- Encryption at rest and in transit by default

PART 1 · FOUNDATIONS

The architecture in one picture



PART 1 · FOUNDATIONS

The storage layer: Colossus and Capacitor

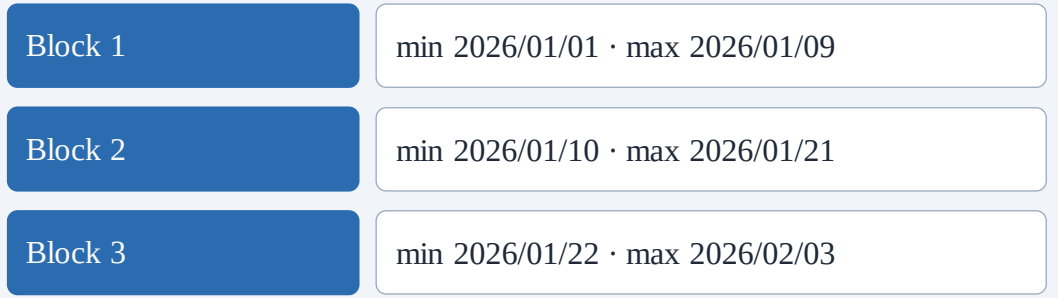
Colossus

Google’s distributed file system and the successor to GFS. It spreads data across many disks, encodes it for durability and replicates it across zones, so losing a disk or a machine loses nothing.

Capacitor

BigQuery’s internal columnar file format. Each column is encoded on its own, with dictionary and run length encoding, compression and per block statistics such as minimum and maximum values.

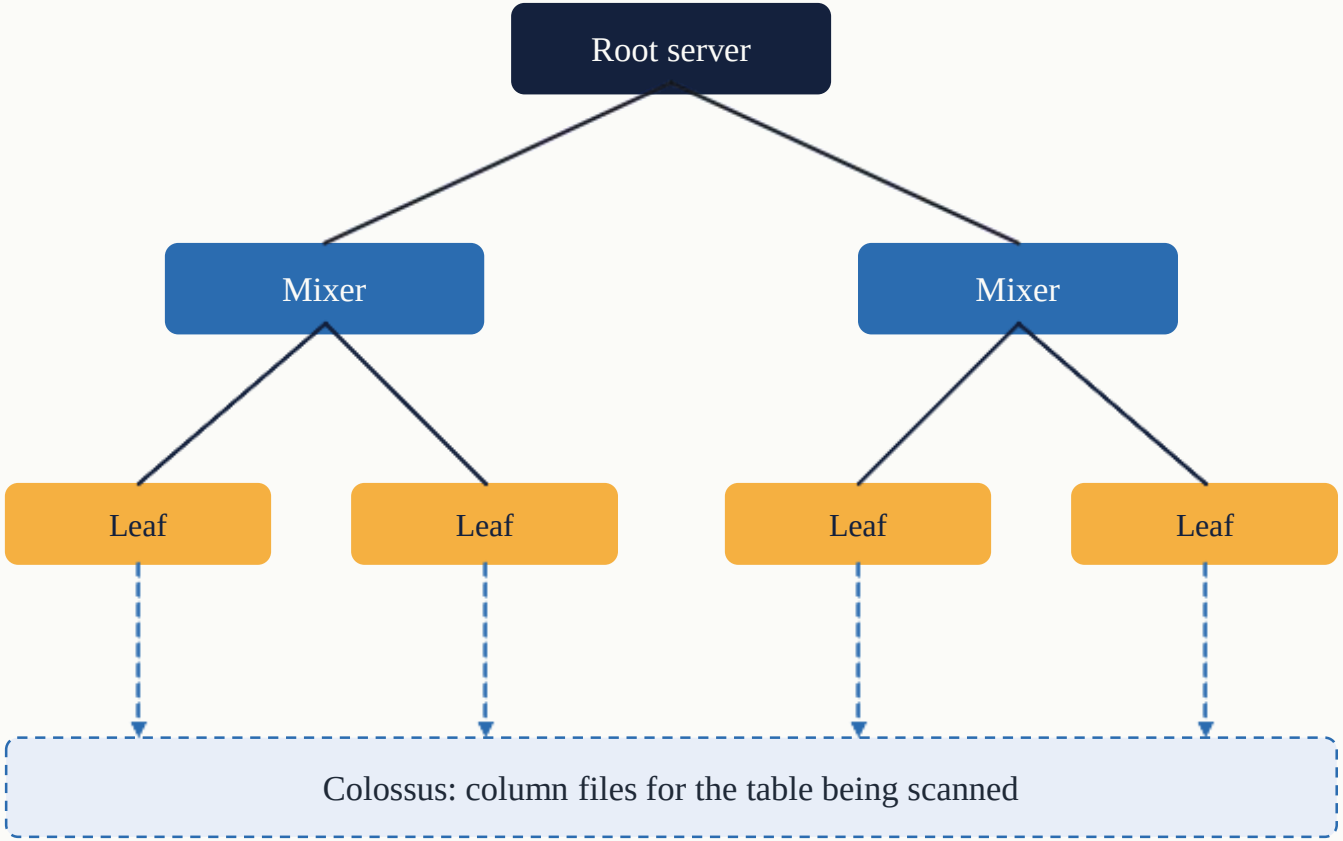
Inside one Capacitor column file



A filter such as order_date in February lets the engine skip blocks 1 and 2 entirely by consulting these statistics. This is the seed of partition and cluster pruning in Part 17.

PART 1 · FOUNDATIONS

The Dremel execution tree



1. Leaves read column blocks in parallel and apply filters and partial aggregation.

2. Mixers combine partial results from many leaves.

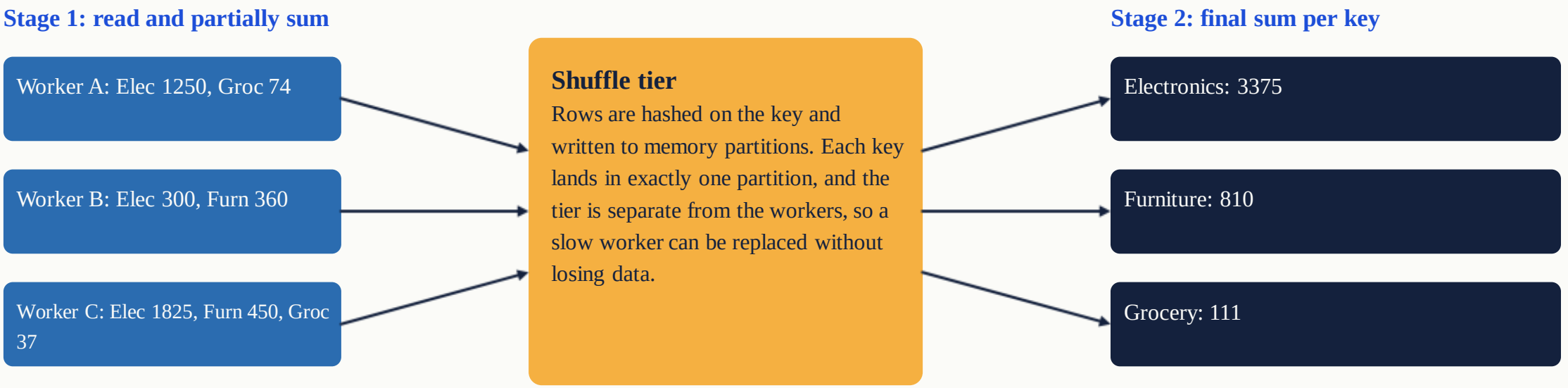
3. The root produces the final answer and returns it to the client.

> Today’s engine generalises this fixed tree into a flexible graph of stages linked by an in memory shuffle, but the core idea holds: push work down to many leaves, then merge upward.

PART 1 · FOUNDATIONS

Distributed shuffle: how stages hand data to each other

Example: **SELECT category, SUM(revenue) … GROUP BY category**. Rows for one category may be on any worker, so they must be regrouped by key.



Why it matters. Shuffle is often the most expensive part of a query. Execution details report bytes shuffled per stage; large GROUP BY keys, JOINs and DISTINCTs drive it. Reducing rows before a join or aggregation reduces shuffle.

PART 1 · FOUNDATIONS

Separating storage from compute: five consequences

1. **Independent scaling.** Store a petabyte and query it occasionally, or store a gigabyte and query it constantly. You pay for each side separately.

2. **No loading into a cluster.** Data is already reachable by every worker over the Jupiter network, so a query can start on thousands of workers instantly.

3. **Workload isolation.** Many teams can read the same table with separate slot reservations without competing for disks.

4. **Zero copy sharing.** Data can be shared across projects and organisations without duplicating bytes.

5. **Cheap snapshots and time travel.** Storage keeps historical versions without involving compute.

PART 1 · FOUNDATIONS

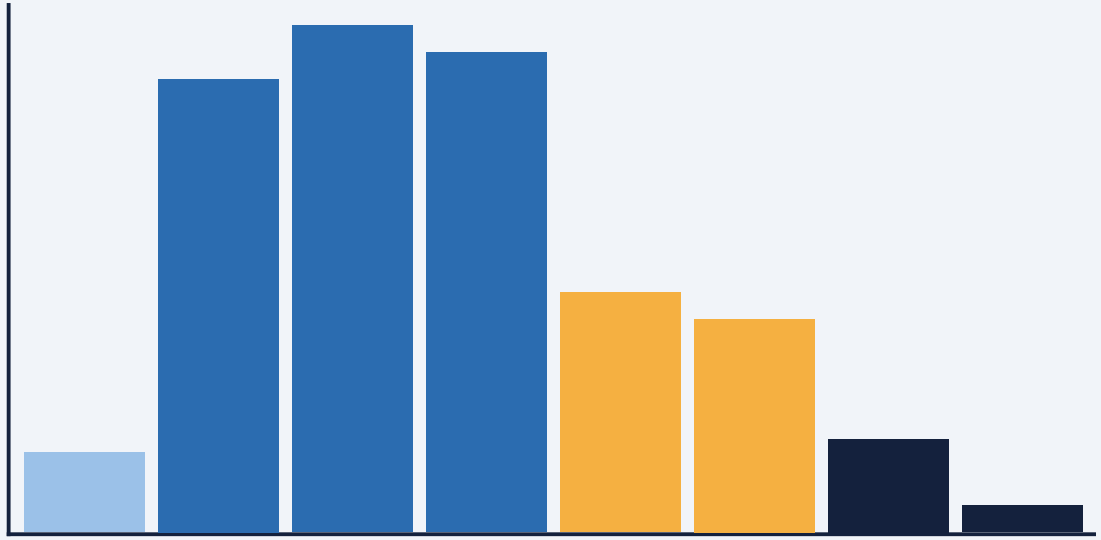
The slot: BigQuery’s unit of compute

A **slot** is a virtual worker: a bundle of CPU, memory and network that executes one unit of work at a time. A query stage is split into many units, and the scheduler assigns them to whatever slots are free.

On demand projects draw from a large shared pool. **Editions** customers buy slot capacity through reservations, with a baseline and autoscaling headroom.

Slot milliseconds in execution details measure total work: 2 seconds on 500 slots is 1,000,000 slot ms.

Slots in use during one query (illustrative)



Scan stage Aggregate stage Output

Time runs left to right. Wide scans use many slots briefly; later stages need fewer.

PART 1 · FOUNDATIONS

The life of a query in seven steps



Steps 1 to 3

A failing permission or syntax check stops the job before any bytes are billed. A cache hit returns instantly at no cost.

Steps 4 to 6

The optimiser prunes partitions, pushes filters down, chooses join strategies and can rebalance work while the query runs.

Step 7

Results go to a hidden temporary table kept about 24 hours, or to a destination table you name.

PART 1 · FOUNDATIONS

Everything is a job

Query
Runs SQL, including DDL, DML and scripts. Billed by bytes or slot time.

Load
Imports files from Cloud Storage or local upload. Free on the shared pool.

Extract
Exports a table to Cloud Storage as CSV, JSON, Avro or Parquet.

Copy
Copies tables within or across datasets in the same location.

Every job has

- a job ID, unique within its project
- a location where it runs, matching its data
- a state: PENDING, RUNNING, DONE
- statistics: bytes processed, slot ms, timeline
- an error result if it failed

```
# Python: the three most recent jobs
for job in client.list_jobs(max_results=3):
    print(job.job_id, job.job_type, job.state)

output
bquxjob_4f1a query DONE
bquxjob_3e9c load DONE
bquxjob_2b7d query DONE
```

PART 1 · FOUNDATIONS

Query result caching: free answers, with conditions

The cache is used when

- the query text is identical, character for character
- the referenced tables have not changed
- the cached result is less than about 24 hours old
- caching is enabled for the job (the default)

Cost of a cache hit: zero bytes billed.

The cache is skipped when

- the query uses CURRENT_TIMESTAMP, RAND or other nondeterministic functions
- results are written to a named destination table
- a table has a streaming buffer or new DML
- the query reads external tables or uses row level security

Practical tip. Dashboards that rerun the same SQL benefit greatly from caching. When benchmarking speed, disable cached results in Query settings, otherwise the second run looks impossibly fast.

PART 1 · RECAP

Five ideas to carry forward

- **Columns, not rows.** Bytes read depend on the columns you select.
- **Storage and compute are separate.** They scale and bill independently.
- **Slots do the work.** Queries become stages, stages become thousands of parallel units.
- **Shuffle connects stages.** Joins and groupings move data; less data moved means faster queries.
- **Everything is a job.** Jobs have IDs, locations, statistics and can be audited.

02

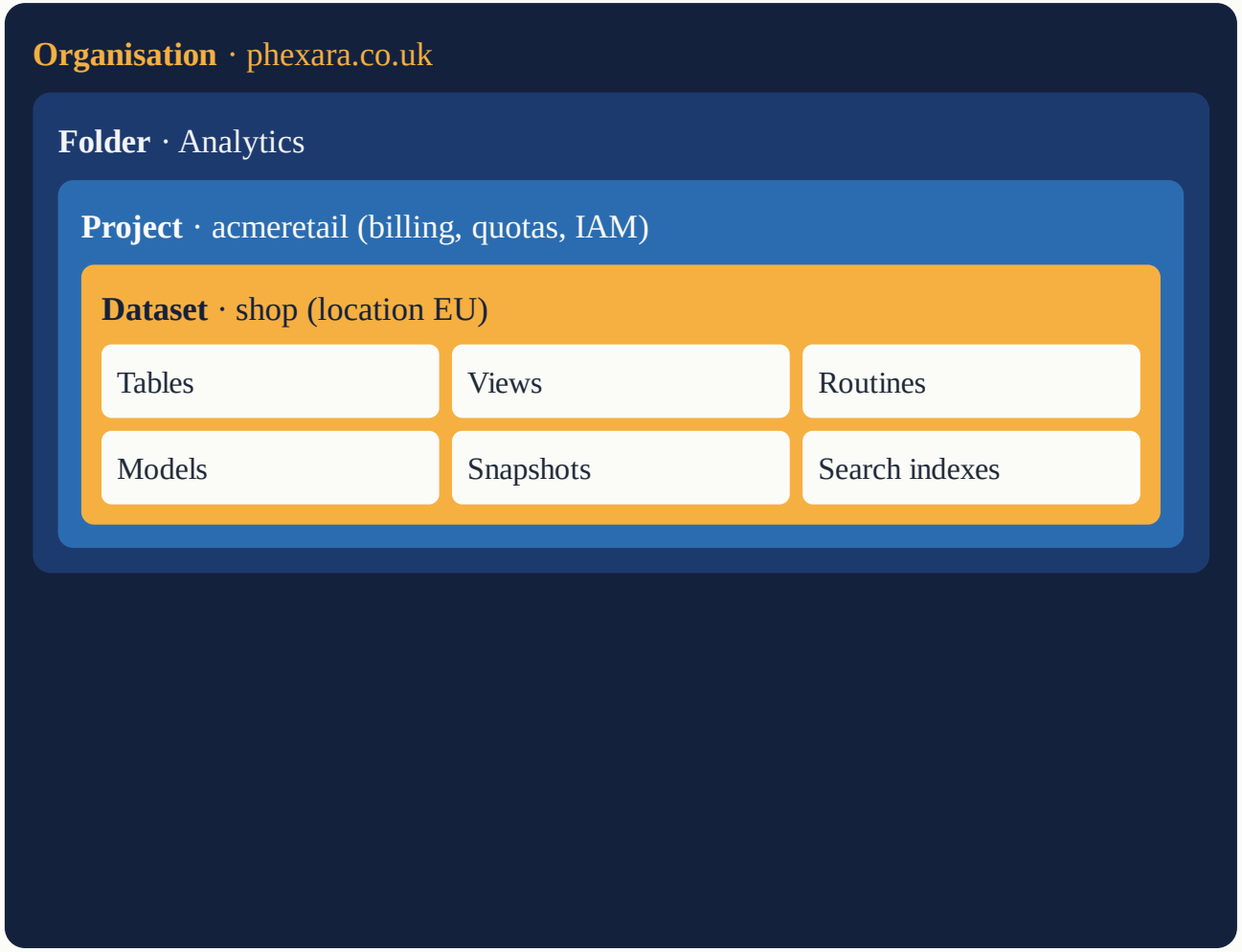
The Resource Model

Where things live in BigQuery: the containers that hold your data, how they are named, where they are physically located, and how to inspect them with SQL.

Hierarchy · Projects · Datasets · Table types · Fully qualified names · Regions and multiregions · Naming rules · INFORMATION_SCHEMA

PART 2 · RESOURCE MODEL

The resource hierarchy as nested containers



Organisation: the root, tied to your Google Workspace or Cloud Identity domain.

Folders: optional grouping by department or environment.

Project: the unit of billing, quotas and API enablement.

Dataset: the unit of location and access control for data.

Objects: tables, views, functions, procedures and models.

IAM permissions granted higher up are inherited by everything inside.

PART 2 · RESOURCE MODEL

Projects: who pays for storage, who pays for compute

Storage project
acmeretail holds dataset shop. Storage charges for its tables land on this project’s bill.



Compute project
analyticsteam runs the query job. Query charges land on this project, the one that runs the job.

Project ID
Globally unique, permanent, lowercase. Appears in every fully qualified name.

Quotas
Concurrent queries, bytes per day and API limits are applied per project and per user.

Sandbox
A project without billing can still query within the free tier, with table expiry limits.

PART 2 · RESOURCE MODEL

Datasets: location, defaults and access in one place

DDL

```
CREATE SCHEMA IF NOT EXISTS acmeretail.shop
OPTIONS (
  location = 'EU',
  description = 'Retail sample data',
  default_table_expiration_days = 365,
  labels = [('team', 'analytics')]
);
```

OUTPUT

This statement created a new dataset named shop.

SCHEMA and **DATASET** mean the same thing in GoogleSQL DDL.

Location is fixed at creation and cannot be changed later; copy the data to move it.

Default expiration is applied to new tables, ideal for scratch datasets.

Labels flow into billing exports, so you can attribute cost by team.

Access granted on a dataset applies to every table within it.

PART 2 · RESOURCE MODEL

Six kinds of table you will meet

Standard table
Data stored in BigQuery managed storage. The default and the fastest to query.

External table
Metadata in BigQuery, data left in Cloud Storage, Drive or another system.

View
A saved query that runs every time it is referenced. Stores no data.

Materialized view
A precomputed, automatically refreshed result that the optimiser can reuse.

Snapshot
A read only, point in time copy that stores only the bytes that later change.

Clone
A writable, lightweight copy that shares storage with its base table.

Each is created with DDL in Part 15. For now, recognise the icons in the Explorer panel and know that querying a view runs its SQL against the underlying tables.

PART 2 · RESOURCE MODEL

Structure of a fully qualified table name

acmeretail.shop.orders

project: optional if it is your current project

dataset: required

table: required

```
# three equivalent references
FROM acmeretail.shop.orders
FROM `acmeretail.shop.orders`
FROM shop.orders # inside project acmeretail
```

When are backticks required? When any part contains characters outside letters, digits and underscores, such as a project ID with a dash, or when a name collides with a reserved keyword. Wrapping the whole path in one pair of backticks is always safe.

PART 2 · RESOURCE MODEL

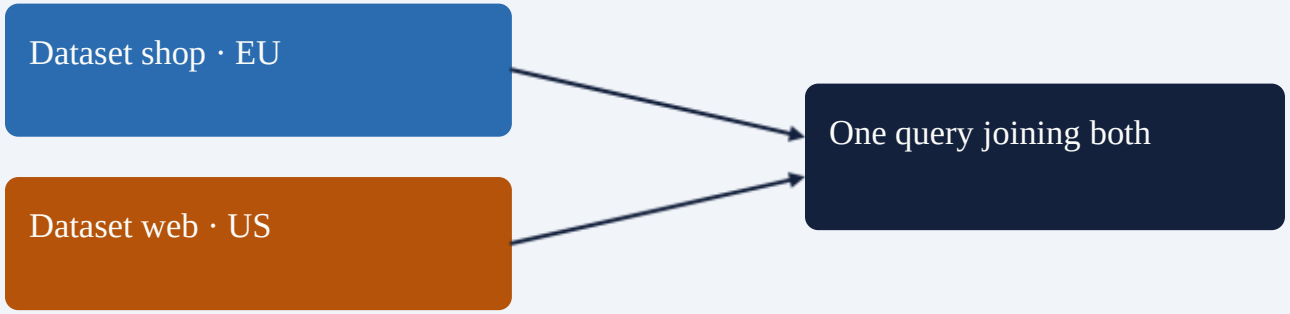
Locations: regions, multiregions and the colocation rule

Region

One geographic place, such as the London region or the Iowa region. Data stays in that region, which helps with residency rules such as UK GDPR.

Multiregion

A large area: **US** or **EU**. Google chooses regions within it for storage and processing, giving very high capacity.



Error: Not found: dataset was not found in location EU. A query runs in a single location; copy or replicate one dataset first.

PART 2 · RESOURCE MODEL

Naming rules and conventions

Object	Rules	Good convention
Project ID	6 to 30 characters: lowercase letters, digits and dashes, starting with a letter; globally unique; permanent	company plus purpose, such as acmeretail
Dataset	Letters, digits, underscores; up to 1,024 characters; case sensitive by default	layer and domain: raw_shop, mart_sales
Table	Up to 1,024 bytes; Unicode letters, marks, digits, connectors, spaces allowed	snake_case nouns: orders, order_items
Column	Up to 300 characters; must not start with reserved prefixes such as _TABLE_ or _PARTITION	snake_case, units in name: amount_gbp
Routine	Letters, digits, underscores; up to 256 characters	verb first: clean_email, calc_margin

Advice. Stick to lowercase snake_case everywhere. It needs no backticks, survives every client tool, and reads cleanly in SQL. Use dataset names to signal pipeline layers such as raw, staging and mart.

PART 2 · RESOURCE MODEL

INFORMATION_SCHEMA: metadata you can query with SQL

QUERY

```
SELECT
  table_name,
  table_type
FROM shop.INFORMATION_SCHEMA.TABLES
ORDER BY table_name;
```

```
SELECT column_name, data_type
FROM shop.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'orders';
```

OUTPUT 1

table_name	table_type
customers	BASE TABLE
order_items	BASE TABLE
orders	BASE TABLE
products	BASE TABLE

OUTPUT 2

column_name	data_type
order_id	INT64
customer_id	INT64
order_date	DATE
status	STRING
channel	STRING

Reading the output. BASE TABLE marks a standard table; views show VIEW. Other useful views include PARTITIONS, TABLE_STORAGE, JOBS and OBJECT_PRIVILEGES, which we use in later parts.

PART 2 · RECAP

The resource model on one card

Level	Controls	Remember
Organisation	Policies for everything below	Org policies can restrict locations
Project	Billing, quotas, APIs	The job’s project pays for compute
Dataset	Location, defaults, access	Location is permanent
Table or view	Schema, partitioning, policies	Six kinds; views store no data
Column and row	Policy tags, row access policies	Covered in Part 21

Names follow **project.dataset.table**

One query runs in **one location**

Metadata lives in **INFORMATION_SCHEMA**

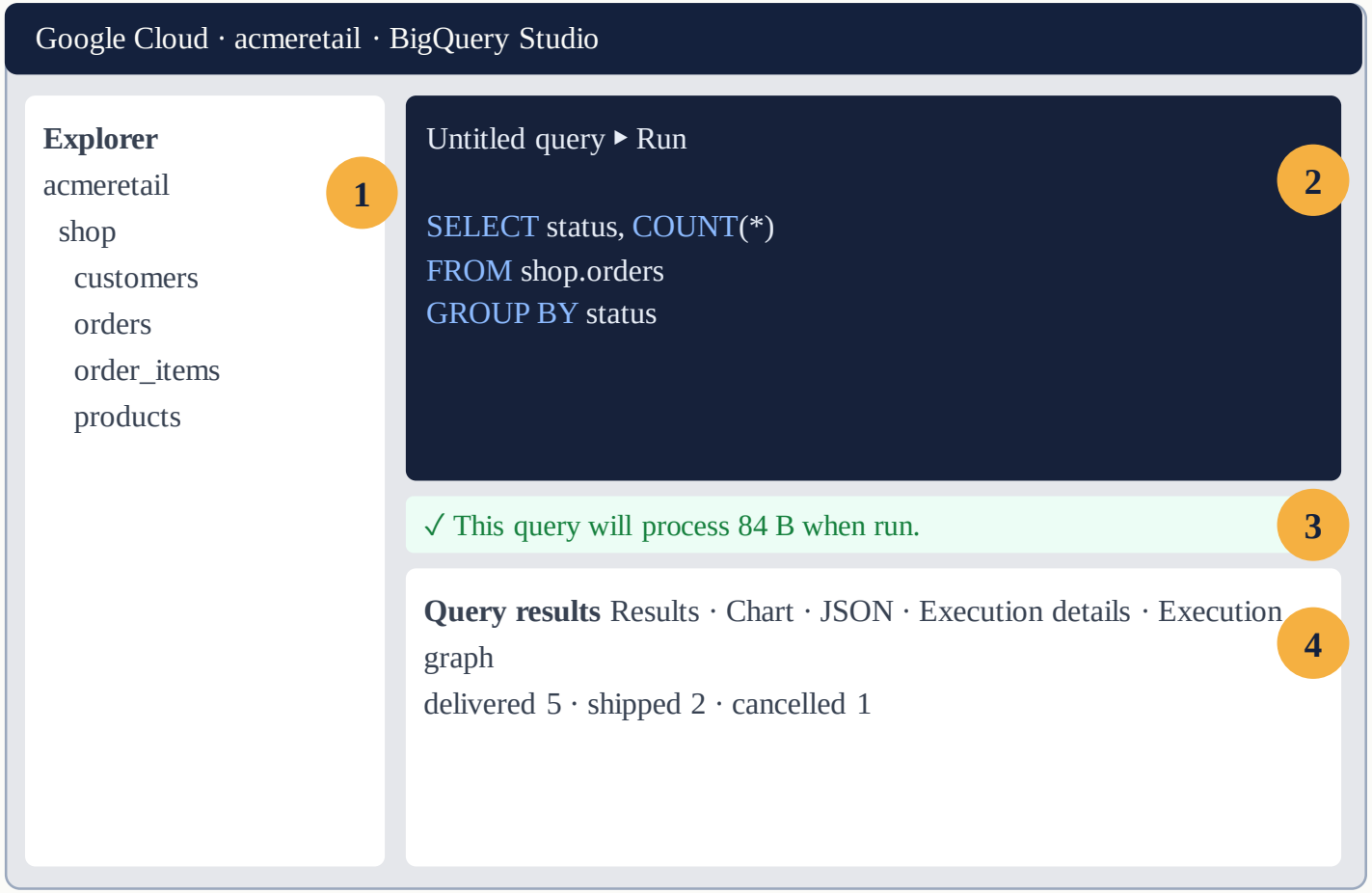
03

Working with BigQuery

The interfaces you will use every day: BigQuery Studio in the console, the bq command line tool, the client libraries, notebooks with BigQuery DataFrames, and Gemini assistance.

Studio layout · Query editor · Results pane · bq tool · Python client · BigQuery DataFrames · Gemini and conversational analytics · Drivers and BI tools

A map of BigQuery Studio



1 Explorer. Browse projects, datasets and tables; star favourites; open schema, details and preview tabs.

2 Editor. Multiple tabs, autocomplete, formatting, saved queries, scheduling.

3 Validator. Checks syntax live and estimates bytes before you spend anything.

4 Results. Grid, charts, JSON and the execution plan of the finished job.

PART 3 · WORKING WITH BIGQUERY

The query validator is your first cost control

✓ This query will process 1.2 GB when run.	Valid query; the estimate is an upper bound of bytes scanned under on demand pricing.
Unrecognized name: custmer_id at [3:7]	A typo caught before running. [3:7] means line 3, character 7.
This query will process 0 B when run.	Common for metadata queries, cached results or a query over a fully pruned partition.

Dry run

The validator performs a dry run: the query is planned but never executed and never billed.

Why an upper bound

Clustering can skip blocks at run time, so actual bytes can be lower than the estimate.

Habit to build

Glance at the estimate before every run on large tables; it costs nothing to look.

PART 3 · WORKING WITH BIGQUERY

Five tabs in the results pane

Results	Chart	JSON	Execution details	Execution graph
Paged grid of rows. Export to CSV, Sheets or Drive, or save as a table.	Quick bar, line or scatter chart of the result for a first look.	Rows as JSON objects. Best view for nested STRUCT and ARRAY output.	Elapsed time, slot time, bytes processed and billed, per stage timings.	Visual plan of stages and shuffle links, with hot spots highlighted.

Job information sits beside these tabs: job ID, user, location, start and end time, bytes billed and the destination table. Personal and project query history keeps this for every job, which is invaluable for debugging and cost review.

The bq command line tool

```
# list datasets in the project
$ bq ls acmeretail:
datasetId
shop
sandbox

# show a table's schema and stats
$ bq show acmeretail:shop.orders
Schema: order_id INTEGER, customer_id INTEGER, ...
Total Rows 8 Total Bytes 316

# preview rows without a query charge
$ bq head acmeretail:shop.products

# make a dataset, then remove it
$ bq mk acmeretail:scratch
$ bq rm acmeretail:scratch
```

Notation. The command line separates project and dataset with a colon: acmeretail:shop.orders.

bq head and the console Preview tab read rows directly from storage, so they cost nothing, unlike SELECT * with LIMIT.

bq query runs SQL from scripts. Set use_legacy_sql to false in your .bigqueryrc file so every query uses GoogleSQL.

bq load and **bq extract** move files in and out (Part 5).

Output abbreviated for space. Flags are passed with a double dash prefix.

Querying from Python with the client library

PYTHON

```
from google.cloud import bigquery

client = bigquery.Client(project="acmeretail")

sql = """
SELECT channel, COUNT(*) AS orders
FROM shop.orders
WHERE status != @excluded
GROUP BY channel
"""
cfg = bigquery.QueryJobConfig(query_parameters=[
  bigquery.ScalarQueryParameter(
    "excluded", "STRING", "cancelled")])

df = client.query(sql, job_config=cfg).to_dataframe()
print(df)
```

OUTPUT

```
   channel  orders
0  web      4
1  app      3
```

Reading the output. Eight orders minus the one cancelled web order leaves 4 web and 3 app. The row order is not guaranteed without ORDER BY.

@excluded is a named query parameter. Parameters keep values out of the SQL text, which blocks SQL injection and lets BigQuery reuse plans. Libraries also exist for Java, Go, Node.js, C#, PHP and Ruby.

PART 3 · WORKING WITH BIGQUERY

Notebooks and BigQuery DataFrames

PYTHON IN A STUDIO NOTEBOOK

```
import bigframes.pandas as bpd

orders = bpd.read_gbq("acmeretail.shop.orders")
counts = orders.groupby("status").size()
counts.to_pandas()
```

OUTPUT

status	size
cancelled	1
delivered	5
shipped	2

What happens. The pandas style API is translated into SQL and executed inside BigQuery. Only the tiny final result travels to the notebook.

Why it matters. Plain pandas would download all rows into notebook memory. With billions of rows, that fails; with BigQuery DataFrames it scales.

Also available. bigframes.ml offers a scikit learn style interface to BigQuery ML models.

> Studio notebooks run on managed Colab Enterprise runtimes, sit beside your SQL, and can be scheduled.

PART 3 · WORKING WITH BIGQUERY

Gemini in BigQuery and conversational analytics

Write SQL from a comment

Describe what you want in plain English inside the editor and Gemini drafts the query against your real schema.

Explain and fix

Select a query to get a plain language explanation, or ask for a fix when it fails.

Data insights

Generated starter questions and queries for a table or dataset, based on its metadata and profile.

Conversational analytics

Chat with data agents over curated tables. Generally available since June 2026.

Professional stance. Treat generated SQL like a colleague’s pull request: read it, check the joins and filters, confirm the byte estimate and validate a few numbers by hand. The rest of this tutorial gives you the knowledge to do exactly that.

PART 3 · WORKING WITH BIGQUERY

Every other door into BigQuery

JDBC and ODBC drivers
For Java apps, SQL clients and legacy tools

BI tools
Looker, Looker Studio, Tableau, Power BI

BigQuery
REST and Storage APIs

Connected Sheets
Pivot billions of rows from Google Sheets

Storage Read API
Fast parallel reads for Spark and ML

Transformation tools
Dataform, dbt, Airflow and Composer

PART 3 · RECAP

Choosing the right interface

If you want to	Use	Because
Explore and prototype SQL	BigQuery Studio	Validator, autocomplete, plans
Automate admin tasks	bq tool	Scriptable, works in CI
Build an application	Client libraries	Parameters, retries, typed results
Do data science at scale	BigQuery DataFrames	pandas syntax, BigQuery engine
Ask a question in English	Gemini, data agents	Fast drafts you then verify
Feed dashboards	BI connectors	Live queries, caching, BI Engine

Whatever the door, the same engine, the same SQL and the same billing apply. From Part 4 onward we concentrate on that SQL.

04

Data Types

Every value in BigQuery has a type, and the type decides storage size, precision, what functions apply and how comparisons behave. Choosing well prevents silent rounding errors and wasted bytes.

Type families · Integers and decimals · Floating point traps · STRING and BYTES · BOOL and NULL logic · Temporal types · STRUCT and ARRAY · JSON, GEOGRAPHY, RANGE, INTERVAL · CAST and SAFE_CAST · Coercion

The GoogleSQL type families

Numeric
INT64
NUMERIC
BIGNUMERIC
FLOAT64

Text and binary
STRING (UTF 8)
BYTES

Logical
BOOL
with NULL as the third state

Temporal
DATE
TIME
DATETIME
TIMESTAMP

Duration
INTERVAL
RANGE

Complex
STRUCT
ARRAY

Semistructured
JSON

Spatial
GEOGRAPHY

Aliases. INT, INTEGER, BIGINT, SMALLINT and TINYINT all map to INT64. DECIMAL is NUMERIC, BIGDECIMAL is BIGNUMERIC. The console and older APIs may show INTEGER, FLOAT and RECORD, which are the legacy names for INT64, FLOAT64 and STRUCT.

PART 4 · DATA TYPES

Numeric types compared

Type	Bytes	Precision and range	Use it for
INT64	8	Whole numbers, about 9.2 × 10^18 either side of zero	IDs, counts, quantities
NUMERIC	16	Exact decimal: 38 digits, 9 after the point	Money, prices, rates
BIGNUMERIC	32	Exact decimal: about 76 digits, 38 after the point	Scientific or crypto values needing extreme precision
FLOAT64	8	IEEE 754 double, about 15 significant digits, plus NaN and infinity	Measurements, ML features, statistics

Parameterised forms

NUMERIC(10, 2) enforces at most 10 digits with 2 decimals, and STRING(50) caps length. Values that do not fit are rejected on write.

Our dataset

IDs and quantities are INT64. unit_price is NUMERIC, so 18.50 × 4 is exactly 74.00, never 73.99999.

PART 4 · DATA TYPES

The floating point trap, demonstrated

QUERY

```
SELECT
  0.1 + 0.2 AS float_sum,
  0.1 + 0.2 = 0.3 AS float_equal,
  NUMERIC '0.1' + NUMERIC '0.2' AS exact_sum,
  NUMERIC '0.1' + NUMERIC '0.2'
    = NUMERIC '0.3' AS exact_equal;
```

OUTPUT

column	value
float_sum	0.30000000000000004
float_equal	false
exact_sum	0.3
exact_equal	true

Reading the output. Decimal literals without a type prefix are FLOAT64. In binary, 0.1 and 0.2 cannot be stored exactly, so their sum is a hair above 0.3 and the equality test fails. The NUMERIC versions are exact decimals, so the sum is precisely 0.3. Rule: store money as NUMERIC, and never compare floats with equals; test that the absolute difference is below a tolerance instead.

PART 4 · DATA TYPES

STRING versus BYTES: characters are not bytes

QUERY

```
SELECT
  LENGTH('Café') AS characters,
  BYTE_LENGTH('Café') AS utf8_bytes,
  CAST('Café' AS BYTES) AS raw_bytes,
  TO_HEX(CAST('Café' AS BYTES)) AS hex;
```

OUTPUT

column	value
characters	4
utf8_bytes	5
raw_bytes	Q2Fmw6k=
hex	436166c3a9

Reading the output. STRING holds Unicode text in UTF 8. The letter é needs two bytes (c3 a9), so four characters occupy five bytes. BYTES values display as base64 in the console. Use BYTES for hashes, encrypted values and binary payloads; use STRING for anything a human reads. Storage is billed on bytes, so the encoding matters at scale.

PART 4 · DATA TYPES

BOOL and three valued logic

a AND b

a \ b	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

a OR b

a \ b	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

NULL means unknown

NULL = NULL is not TRUE; it is NULL. NOT NULL is also NULL. Comparisons with NULL never match.

WHERE keeps only TRUE

Rows whose condition is FALSE or NULL are both dropped. This is why IS NULL exists.

Null safe test

a IS NOT DISTINCT FROM b treats two NULLs as equal and never returns NULL.

PART 4 · DATA TYPES

Four temporal types, four different meanings

Type	Represents	Time zone	Typical column
DATE	A calendar day	None	order_date, birth_date
TIME	A clock time within a day	None	store_opening_time
DATETIME	A calendar day plus clock time, as seen on a wall clock	None, so ambiguous across zones	local_appointment_time
TIMESTAMP	An exact instant in history, microsecond precision	Stored as UTC, displayed in any zone	event_ts, created_at

The same **TIMESTAMP** is 09:00 in London and 10:00 in Paris: one instant, two wall clock readings.

Rule. Record events as TIMESTAMP. Convert to DATETIME or DATE in a named zone only at reporting time.

All four support years 1 to 9999. Part 12 covers the functions that construct, extract, truncate and convert them.

PART 4 · DATA TYPES

STRUCT and ARRAY: types that hold other values

STRUCT: a record inside a cell

city	country	postcode
Paris	France	75004

Named fields of possibly different types. Access with a dot: **address.city**. Shown as RECORD in older tools.

ARRAY: an ordered list inside a cell

[0]	[1]	[2]
gift	express	fragile

Zero or more elements of one type. Shown as REPEATED mode. An ARRAY cannot directly contain another ARRAY.

Combine them as **ARRAY<STRUCT<…>>** and a single order row can carry all its line items. This is the heart of BigQuery's denormalised design, explored fully in Part 13.

PART 4 · DATA TYPES

JSON, GEOGRAPHY, RANGE and INTERVAL at a glance

JSON

```
JSON '{"gift": true, "tags": ["red"]}'
```

Schema free documents, stored in a parsed, columnar friendly form. Part 14.

GEOGRAPHY

```
ST_GEOGPOINT(2.3522, 48.8566)
```

Points, lines and polygons on the WGS84 Earth surface. Longitude comes first.

RANGE

```
RANGE(DATE(2026, 1, 1), DATE(2026, 4, 1))
```

A contiguous span with an inclusive start and exclusive end: Q1 2026 here.

INTERVAL

```
INTERVAL 3 DAY · INTERVAL 90 MINUTE
```

A duration in years, months, days and time. Add it to dates and timestamps.

PART 4 · DATA TYPES

CAST versus SAFE_CAST

QUERY

```
SELECT
  CAST('42' AS INT64) AS a,
  SAFE_CAST('4x2' AS INT64) AS b,
  CAST(3.7 AS INT64) AS c,
  CAST(2.5 AS INT64) AS d,
  CAST(TRUE AS STRING) AS e;
```

And the unsafe version: CAST('4x2' AS INT64) stops the whole query with the error Bad int64 value: 4x2

OUTPUT

a	b	c	d	e
42	NULL	4	3	true

Reading the output. a converts cleanly. b cannot be parsed, so SAFE_CAST returns NULL instead of failing. c and d show that FLOAT64 to INT64 rounds to the nearest integer, with halves rounded away from zero, so 2.5 becomes 3. e is the lowercase text true.

Use SAFE_CAST when cleaning messy input, then count the NULLs it produced to measure data quality.

PART 4 · COERCION AND RECAP

Implicit coercion and supertypes



When two numeric types meet, BigQuery widens to the right along this chain. INT64 plus NUMERIC is NUMERIC; anything plus FLOAT64 is FLOAT64, which can lose exactness.

Expression	Result type	Why
quantity * unit_price	NUMERIC	INT64 widened to NUMERIC
unit_price * 1.2	FLOAT64	1.2 is a FLOAT64 literal
unit_price * NUMERIC '1.2'	NUMERIC	Typed literal keeps exactness
10 / 4	FLOAT64	The / operator always returns FLOAT64: 2.5
DIV(10, 4)	INT64	Integer division: 2

05

Getting Data In and Out

A warehouse is only as good as the data that reaches it. This part compares every ingestion path, from free batch loads to real time streams to querying data where it already lives, and shows how to export results.

Ingestion map · File formats · LOAD DATA · Schemas · Write dispositions · Storage Write API · Data Transfer Service · External tables and BigLake · EXPORT DATA · Choosing a path

PART 5 · DATA IN AND OUT

Five ways data reaches BigQuery

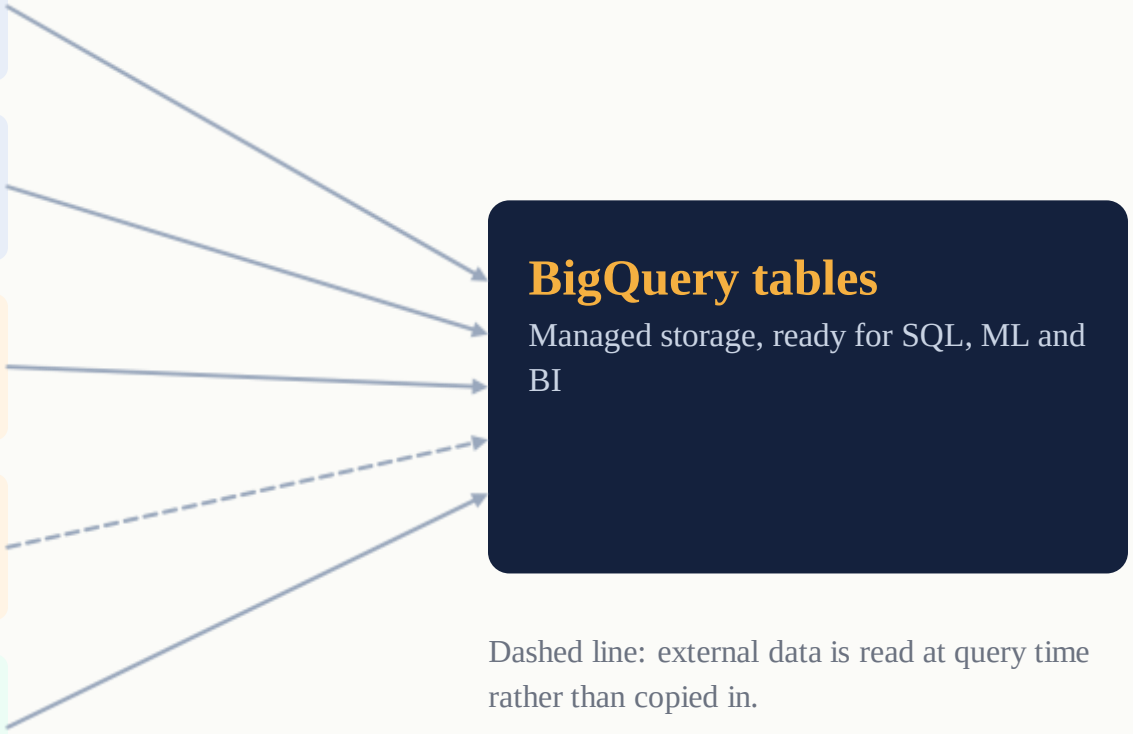
1 Batch load jobs
Files from Cloud Storage or upload. Free on the shared pool. Minutes of latency.

2 Streaming
Storage Write API rows available to query within seconds. Paid per GB.

3 Data Transfer Service
Scheduled, managed connectors from SaaS apps, databases and other clouds.

4 External and BigLake tables
Query files in place without loading. Data stays in Cloud Storage or elsewhere.

5 SQL: INSERT, CTAS, MERGE
Create new tables from queries over existing BigQuery data.



Dashed line: external data is read at query time rather than copied in.

Choosing a file format for batch loads

Format	Schema inside	Nested data	Load speed	Notes
Avro	Yes	Yes	Fastest	Splittable even when compressed
Parquet	Yes	Yes	Fast	Columnar, great from Spark
ORC	Yes	Yes	Fast	Common in Hadoop estates
JSON (newline delimited)	No	Yes	Slower	One object per line
CSV	No	No	Slower	Quoting and encoding pitfalls

Prefer
Avro or Parquet: typed, compressed, self describing, nested friendly.

Beware
Gzipped CSV and JSON cannot be split, so one huge file loads on one worker.

Cost
Load jobs on the shared slot pool are free; you pay only for stored bytes.

PART 5 · DATA IN AND OUT

Loading files with the LOAD DATA statement

SQL

```
LOAD DATA INTO shop.orders_staging (
  order_id INT64,
  customer_id INT64,
  order_date DATE,
  status STRING,
  channel STRING
)
FROM FILES (
  format = 'CSV',
  skip_leading_rows = 1,
  uris = ['gs://acme_landing/orders/*.csv']
);
```

JOB RESULT

Load job completed.
Rows loaded: 8 · Files: 2
Bytes billed: 0 B

> **Reading the output.** The wildcard picked up two CSV files containing eight data rows after each header line was skipped. Loading bills no query bytes.

LOAD DATA INTO appends; **LOAD DATA OVERWRITE** replaces the table. Omit the column list to use the existing schema or schema detection. Options also cover field delimiter, quote character, null markers and maximum bad records.

PART 5 · DATA IN AND OUT

Schemas: modes, detection and evolution

NULLABLE
The default mode. The column may hold NULL.

REQUIRED
Declared as NOT NULL in DDL. Loads with a missing value fail.

REPEATED
An ARRAY column. Never NULL; an empty array instead.

Schema detection

For CSV and JSON, BigQuery samples rows and guesses types. Convenient for exploration, risky for production: a postcode such as 07001 can become an integer and lose its leading zero.

Safe evolution

Allowed without rewriting data: add a NULLABLE column, relax REQUIRED to NULLABLE, widen a parameterised type. Renaming or changing a type needs ALTER TABLE or a rewrite.

Professional practice. Declare schemas explicitly in code and keep them in version control. Treat detection as a scaffolding tool, never as the contract between producers and consumers.

PART 5 · DATA IN AND OUT

Write dispositions: what happens to existing rows

WRITE_APPEND



8 existing + 2 new = 10 rows. Default for load jobs and INSERT.

WRITE_TRUNCATE



Old rows replaced: 2 rows remain. Used by LOAD DATA OVERWRITE and CREATE OR REPLACE.

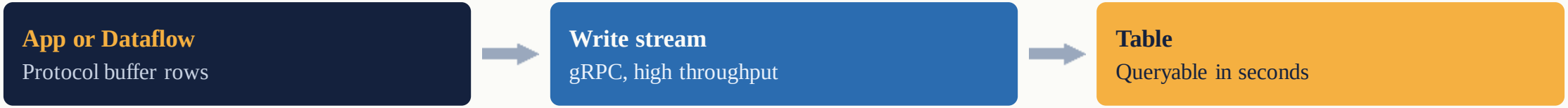
WRITE_EMPTY



Writes only if the table is empty; otherwise the job fails. A safety net.

Atomicity. A load or query job writes all of its rows or none of them. A truncating load swaps the new data in at the end, so readers never see a half empty table.

Streaming with the Storage Write API



Stream type	Delivery guarantee	Best for
Default	At least once	Simple continuous ingestion where rare duplicates are acceptable
Committed	Exactly once, with offsets	Event pipelines that must not duplicate
Pending	Exactly once, all or nothing	Batch style writes committed atomically
Buffered	Row level flush control	Advanced connectors needing manual flushes

Notes. The older insertAll streaming API still works but costs more per GB. Streaming ingestion is billed, unlike batch loads. Recently streamed rows may not yet be eligible for some DML and copy operations.

PART 5 · DATA IN AND OUT

BigQuery Data Transfer Service

Managed, scheduled connectors that land data in BigQuery with retries, backfills and monitoring, and no code to maintain.

Google sources
Google Ads, Campaign Manager, Google Play, YouTube channel reports

Cloud storage
Cloud Storage, Amazon S3, Azure Blob Storage

Databases
PostgreSQL, MySQL, SQL Server, Oracle, Teradata, Redshift, Snowflake

SaaS apps
Salesforce, Shopify, Mailchimp, Klaviyo, Facebook Ads and more

Recent additions

2026 release notes added transfers from PostgreSQL, Shopify, Mailchimp and Snowflake, plus incremental transfers for SQL Server and Klaviyo.

Scheduled queries

The same service also runs your saved SQL on a schedule, covered in Part 23.

External tables, BigLake and Apache Iceberg

```
# plain external table
CREATE EXTERNAL TABLE shop.ext_returns
OPTIONS (
  format = 'PARQUET',
  uris = ['gs://acme_landing/returns/*.parquet']
);
```

```
# BigLake table: same files, governed access
CREATE EXTERNAL TABLE shop.lake_returns
WITH CONNECTION `acmeretail.eu.lake_conn`
OPTIONS (
  format = 'PARQUET',
  uris = ['gs://acme_landing/returns/*.parquet']
);
```

External table. Users need access to the bucket itself. No caching; bytes estimates unavailable; slower than native.

BigLake table. A connection's service account reads the files, so you grant access to the table, not the bucket. Supports row and column security and metadata caching.

Iceberg. BigLake tables for Apache Iceberg let BigQuery and engines such as Spark share one open table format with full DML.

> Use external data for exploration and lakehouse sharing; load into native tables for the hottest, most queried data.

Exporting results with EXPORT DATA

SQL

```
EXPORT DATA OPTIONS (
  uri = 'gs://acme_exports/delivered/part_*.parquet',
  format = 'PARQUET',
  overwrite = TRUE
) AS
SELECT order_id, customer_id, order_date
FROM shop.orders
WHERE status = 'delivered';
```

FILES WRITTEN

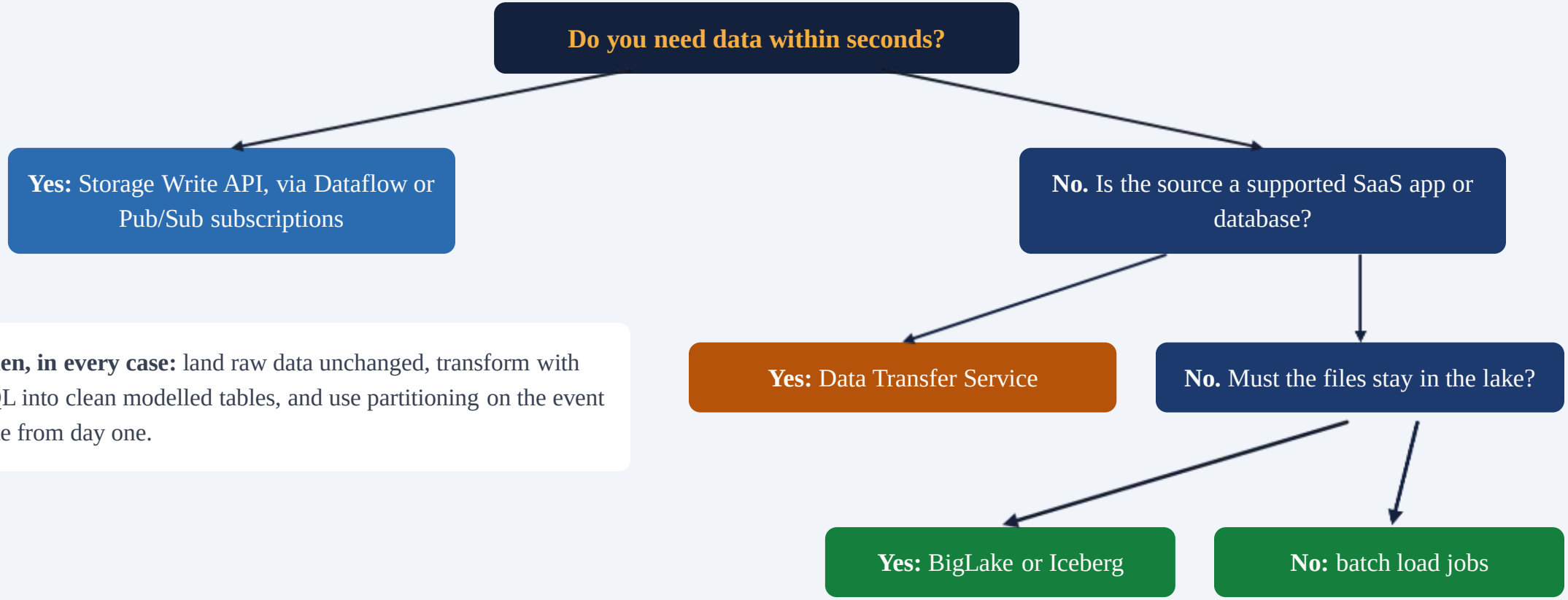
```
gs://acme_exports/delivered/
  part_000000000000.parquet (5 rows)
```

Reading the output. The single asterisk is required; BigQuery replaces it with a twelve digit sequence number and splits large results into many files of up to about 1 GB each. Five delivered orders fit in one file.

You pay for the query that selects the rows. Formats: CSV, JSON, Avro, Parquet. Exports can also target Bigtable, Spanner and Pub/Sub for reverse ETL.

PART 5 · RECAP

A decision tree for ingestion



Then, in every case: land raw data unchanged, transform with SQL into clean modelled tables, and use partitioning on the event date from day one.

06

SQL Foundations

The core of GoogleSQL: how a query is built, the order in which its clauses really run, and the everyday tools for choosing columns, filtering rows, handling NULL, sorting and writing conditional logic.

GoogleSQL versus legacy · Query anatomy · Logical order · SELECT, EXCEPT, REPLACE · Aliases · WHERE and operators · IN, BETWEEN, LIKE · NULL functions · ORDER BY · LIMIT · DISTINCT · CASE and IF · Parameters · Style · SAFE functions · Practice

PART 6 · SQL FOUNDATIONS

GoogleSQL is the only dialect worth learning

Feature	GoogleSQL (standard)	Legacy SQL
Table reference	`acmeretail.shop.orders`	[acmeretail:shop.orders]
Standards	ANSI SQL 2011 compliant	Proprietary
DML and DDL	Full support	Not supported
Nested data	STRUCT, ARRAY, UNNEST	FLATTEN, implicit
Modern features	Pipe syntax, ML, JSON, scripting	None added
Status in 2026	Default everywhere	New use limited from 1 June 2026

What this means for you. Every example in this tutorial is GoogleSQL. If you inherit legacy queries, BigQuery will keep existing workloads running, but plan to migrate them. A query can opt in explicitly with the line #standardSQL at the top.

Structure of a complete query

```
WITH sales AS (…)
SELECT category, SUM(revenue) AS rev
FROM sales
JOIN shop.products USING (product_id)
WHERE status != 'cancelled'
GROUP BY category
HAVING rev > 100
QUALIFY RANK() OVER (ORDER BY rev DESC) <= 3
ORDER BY rev DESC
LIMIT 10;
```

WITH names a reusable subquery

SELECT chooses and computes output columns

FROM names the starting table

JOIN combines another table on a key

WHERE filters individual rows

GROUP BY collapses rows into groups

HAVING filters groups after aggregation

QUALIFY filters on window function results

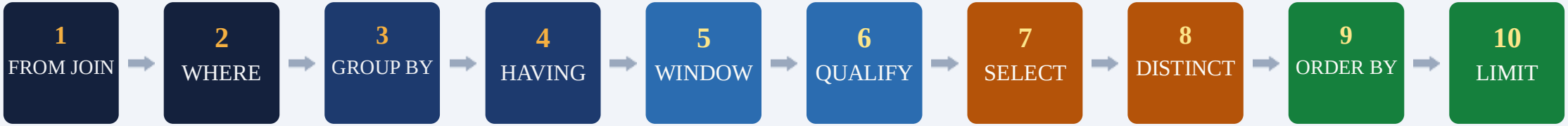
ORDER BY sorts the final rows

LIMIT caps how many rows return

> Only SELECT is mandatory. SELECT 1 + 1 is a valid query.

PART 6 · SQL FOUNDATIONS

Written order is not execution order



Consequence 1

WHERE runs before SELECT, so a column alias defined in SELECT cannot be used in WHERE. Repeat the expression or use a CTE.

Consequence 2

Aggregates do not exist yet in WHERE, so filtering on SUM() needs HAVING.

Consequence 3

Window functions are computed after grouping, so they can rank aggregated values, and QUALIFY filters them.

BigQuery convenience

GROUP BY, HAVING and ORDER BY may refer to SELECT aliases, as the anatomy slide did with rev.

PART 6 · SQL FOUNDATIONS

SELECT: choosing columns

QUERY

```
SELECT
  customer_id,
  name,
  city
FROM shop.customers
ORDER BY customer_id;
```

Bytes read: only the three named columns, never country, signup_date or tier.

OUTPUT · 6 ROWS

customer_id	name	city
1	Amara Okafor	Manchester
2	Ben Clarke	London
3	Chloe Martin	Paris
4	David Osei	Accra
5	Elena Rossi	Milan
6	Farid Haddad	Manchester

Reading the output. One output row per input row, with columns in the order listed. ORDER BY makes the order deterministic; without it BigQuery may return rows in any order, and that order can change between runs.

PART 6 · SQL FOUNDATIONS

SELECT * EXCEPT and REPLACE: BigQuery’s shortcuts

QUERY

```
SELECT
  * EXCEPT (signup_date, tier)
    REPLACE (UPPER(city) AS city)
FROM shop.customers
WHERE country = 'UK'
ORDER BY customer_id;
```

OUTPUT · 3 ROWS

customer_id	name	city	country
1	Amara Okafor	MANCHESTER	UK
2	Ben Clarke	LONDON	UK
6	Farid Haddad	MANCHESTER	UK

Reading the output. EXCEPT removed two columns from the star; REPLACE swapped city for its uppercase version while keeping its name and position. Only three customers live in the UK. Note that EXCEPT still reads fewer bytes: excluded columns are never scanned.

Avoid bare SELECT * on wide production tables: you pay for every column.

Use the free Preview tab or bq head to look at sample rows instead.

PART 6 · SQL FOUNDATIONS

Computed columns and aliases

QUERY

```
SELECT
  oi.order_id,
  oi.product_id,
  oi.quantity,
  oi.unit_price,
  oi.quantity * oi.unit_price AS line_total
FROM shop.order_items AS oi
WHERE oi.order_id IN (5001, 5005)
ORDER BY oi.order_id, oi.product_id;
```

OUTPUT · 4 ROWS

order_id	product_id	quantity	unit_price	line_total
5001	101	1	1200	1200
5001	102	2	25	50
5005	103	2	180	360
5005	105	2	18.5	37

Reading the output. line_total is calculated per row. The console prints NUMERIC values without trailing zeros, so 25.00 shows as 25. AS is optional but makes intent obvious; the table alias oi shortens every column reference.

PART 6 · SQL FOUNDATIONS

WHERE: filtering rows with conditions

QUERY

```
SELECT order_id, customer_id, status
FROM shop.orders
WHERE status = 'delivered'
  AND channel = 'web'
ORDER BY order_id;
```

OUTPUT · 3 ROWS

order_id	customer_id	status
5001	1	delivered
5005	4	delivered
5007	3	delivered



Reading the output. Both conditions must be TRUE. Of five delivered orders, 5002 and 5003 came through the app, leaving three web orders. String comparison is case sensitive: 'Delivered' would match nothing.

Operators you will use in every WHERE clause

Operator	Meaning and example
= != <>	Equal, not equal: status != 'cancelled'
< <= > >=	Ordering: unit_price >= 100
AND OR NOT	Combine conditions; AND binds tighter than OR
IN (…)	Membership: country IN ('UK', 'France')
BETWEEN a AND b	Inclusive range: quantity BETWEEN 1 AND 2
LIKE	Pattern: name LIKE 'A%'
IS [NOT] NULL	Null test: tier IS NOT NULL
IS [NOT] DISTINCT FROM	Null safe equality
EXISTS (…)	Subquery returns at least one row

Precedence trap. status = 'shipped' OR status = 'delivered' AND channel = 'web' is read as shipped OR (delivered AND web). Always add parentheses when mixing AND with OR.

PART 6 · SQL FOUNDATIONS

IN, BETWEEN and LIKE working together

QUERY

```
SELECT product_id, product_name, unit_price
FROM shop.products
WHERE category IN ('Electronics', 'Furniture')
  AND unit_price BETWEEN 100 AND 500
  AND product_name LIKE '%Desk%'
ORDER BY unit_price;
```

OUTPUT · 2 ROWS

product_id	product_name	unit_price
103	Desk Chair	180
104	Standing Desk	450

IN keeps 5 of 6 products; Coffee Beans is Grocery.

BETWEEN drops the 1200 laptop and the 25 mouse; both ends are inclusive.

LIKE '%Desk%' drops Headphones. % matches any run of characters, _ exactly one.

Reading the output. Desk Chair matches because Desk appears anywhere in the name. LIKE is case sensitive; use LOWER(product_name) LIKE '%desk%' for a case insensitive match.

Handling NULL: IS NULL, IFNULL, COALESCE, NULLIF

QUERY

```
WITH promo AS (
  SELECT 'A' AS code, 10 AS discount UNION ALL
  SELECT 'B', NULL UNION ALL
  SELECT 'C', 0
)
SELECT
  code, discount,
  discount IS NULL AS missing,
  IFNULL(discount, 0) AS filled,
  COALESCE(discount, 5) AS dflt,
  NULLIF(discount, 0) AS zero_null
FROM promo;
```

OUTPUT · 3 ROWS

code	discount	missing	filled	dflt	zero_null
A	10	false	10	10	10
B	NULL	true	0	5	NULL
C	0	false	0	0	NULL

Reading the output. IFNULL(x, y) returns y only when x is NULL. COALESCE returns the first non NULL of any number of arguments. NULLIF(x, 0) turns zero into NULL, the classic guard before dividing: amount / NULLIF(count, 0) yields NULL instead of an error.

ORDER BY with several keys

QUERY

```
SELECT
  product_name,
  category,
  unit_price
FROM shop.products
ORDER BY
  category ASC,
  unit_price DESC;
```

OUTPUT · 6 ROWS

product_name	category	unit_price
Laptop Pro 14	Electronics	1200
Headphones	Electronics	300
Wireless Mouse	Electronics	25
Standing Desk	Furniture	450
Desk Chair	Furniture	180
Coffee Beans 1kg	Grocery	18.5

Reading the output. Rows are sorted alphabetically by category first (shaded bands); within each category, the most expensive product comes first. NULLs sort first in ascending order and last in descending order by default; override with NULLS LAST or NULLS FIRST.

PART 6 · SQL FOUNDATIONS

LIMIT controls rows returned, not bytes billed

SELECT * FROM events LIMIT 10 on a 2 TB table

Bytes scanned

2 TB: every column, every row

Rows returned

10 rows

In a classic database, LIMIT can stop a scan early. In BigQuery, columns are read in parallel by thousands of workers before LIMIT is applied, so the bill is the same as without LIMIT.

Exception: on clustered tables BigQuery can sometimes stop reading early, and the estimate shown is then an upper bound.

Also: without ORDER BY, which 10 rows you get is arbitrary.

To reduce cost, reduce columns and partitions: select only needed columns, filter on the partition column, or use the free table Preview to inspect data.

PART 6 · SQL FOUNDATIONS

DISTINCT: removing duplicate rows

QUERY A

```
SELECT DISTINCT country
FROM shop.customers
ORDER BY country;
```

OUTPUT A · 4 ROWS

country
France
Ghana
Italy
UK

QUERY B

```
SELECT DISTINCT country, city
FROM shop.customers
ORDER BY country, city;
```

OUTPUT B · 5 ROWS

country	city
France	Paris
Ghana	Accra
Italy	Milan
UK	London
UK	Manchester

Reading the output. DISTINCT applies to the whole selected row. Three UK customers collapse to one row in A; in B, the two Manchester customers collapse but London stays separate. DISTINCT requires a shuffle, so it is not free on large tables.

PART 6 · SQL FOUNDATIONS

CASE: conditional logic inside a query

QUERY

```
SELECT
  product_name,
  unit_price,
  CASE
    WHEN unit_price >= 400 THEN 'Premium'
    WHEN unit_price >= 100 THEN 'Mid range'
    ELSE 'Budget'
  END AS price_band
FROM shop.products
ORDER BY unit_price DESC;
```

OUTPUT · 6 ROWS

product_name	unit_price	price_band
Laptop Pro 14	1200	Premium
Standing Desk	450	Premium
Headphones	300	Mid range
Desk Chair	180	Mid range
Wireless Mouse	25	Budget
Coffee Beans 1kg	18.5	Budget

Reading the output. WHEN branches are tested top to bottom and the first TRUE wins, so 1200 never reaches the Mid range test. Without ELSE, unmatched rows would get NULL. Order thresholds from strictest to loosest.

PART 6 · SQL FOUNDATIONS

IF and the simple CASE form

QUERY

```
SELECT
  order_id,
  status,
  IF(status = 'cancelled', 0, 1) AS is_sale,
  CASE channel
    WHEN 'web' THEN 'Website'
    WHEN 'app' THEN 'Mobile app'
  END AS channel_name
FROM shop.orders
WHERE order_id <= 5004
ORDER BY order_id;
```

OUTPUT · 4 ROWS

order_id	status	is_sale	channel_name
5001	delivered	1	Website
5002	delivered	1	Mobile app
5003	delivered	1	Mobile app
5004	cancelled	0	Website

Reading the output. IF(condition, then, else) is shorthand for a two branch CASE. The simple CASE compares one expression to listed values. A flag of 1 and 0 is handy: SUM(is_sale) counts sales.

Related: IFNULL, NULLIF, COALESCE and the comparison functions GREATEST and LEAST.

Quotes, literals and query parameters

Syntax	Meaning
'text' or "text"	A STRING literal; both quote styles are equivalent
`name`	A quoted identifier: table, column or alias name
r'\d+'	A raw string: backslashes kept as typed, ideal for regex
'''multi line'''	Triple quoted string that may span lines
b'abc'	A BYTES literal
DATE(2026, 1, 5)	Typed constructor; typed literals such as NUMERIC '1.5' also exist
@min_price	Named query parameter, supplied by the client
?	Positional parameter

Security rule. Never build SQL by gluing user input into strings. Pass values as parameters, as on the Python slide. Parameters cannot replace table or column names; for those, validate against an allow list.

Comments and a professional style guide

```
# Monthly revenue, excluding cancellations
/* Owner: analytics team
   Source: shop.orders, shop.order_items */
SELECT
  DATE_TRUNC(o.order_date, MONTH) AS month,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.orders AS o
JOIN shop.order_items AS i
  ON i.order_id = o.order_id
WHERE o.status != 'cancelled'
GROUP BY month
ORDER BY month;
```

GoogleSQL accepts # and two dash line comments, and /* */ block comments.

1. Keywords in UPPERCASE, names in lowercase snake_case.
2. One selected column per line; leading clause keywords flush left.
3. Short, meaningful table aliases; qualify every column in joins.
4. Explicit JOIN … ON, never comma joins for related tables.
5. CTEs for each logical step, named after what they contain.
6. A header comment stating purpose, owner and sources.

PART 6 · SQL FOUNDATIONS

The SAFE prefix: errors become NULL

QUERY

```
SELECT
  SAFE_DIVIDE(10, 0) AS a,
  SAFE.PARSE_DATE('%d/%m/%Y', '31/02/2026') AS b,
  SAFE.PARSE_DATE('%d/%m/%Y', '28/02/2026') IS NOT NULL AS c,
  IEEE_DIVIDE(1, 0) AS d;
```

Without SAFE: 10 / 0 fails with division by zero: 10 / 0, and PARSE_DATE fails on 31 February.

OUTPUT

a	b	c	d
NULL	NULL	true	Infinity

Reading the output. SAFE_DIVIDE returns NULL rather than failing. The SAFE. prefix works on most scalar functions: an impossible date becomes NULL, a valid one parses, so c is true. IEEE_DIVIDE follows floating point rules and returns Infinity.

One bad row in a billion should not kill a nightly pipeline. Use SAFE functions, then audit the NULLs they produce.

PART 6 · PRACTICE

Exercise: a friendly web order list

Task. List web orders that were not cancelled, newest first, with a readable date and a customer facing status label.

SOLUTION

```
SELECT
  order_id,
  FORMAT_DATE('%d %b %Y', order_date) AS order_day,
  CASE status
    WHEN 'delivered' THEN 'Complete'
    WHEN 'shipped' THEN 'On its way'
  END AS label
FROM shop.orders
WHERE channel = 'web' AND status != 'cancelled'
ORDER BY order_date DESC;
```

OUTPUT · 4 ROWS

order_id	order_day	label
5008	03 Apr 2026	On its way
5007	28 Mar 2026	Complete
5005	02 Mar 2026	Complete
5001	05 Jan 2026	Complete

Check. Five web orders exist; 5004 is cancelled, leaving four. We sort on the real DATE column, not on the formatted text, which would sort alphabetically.

07

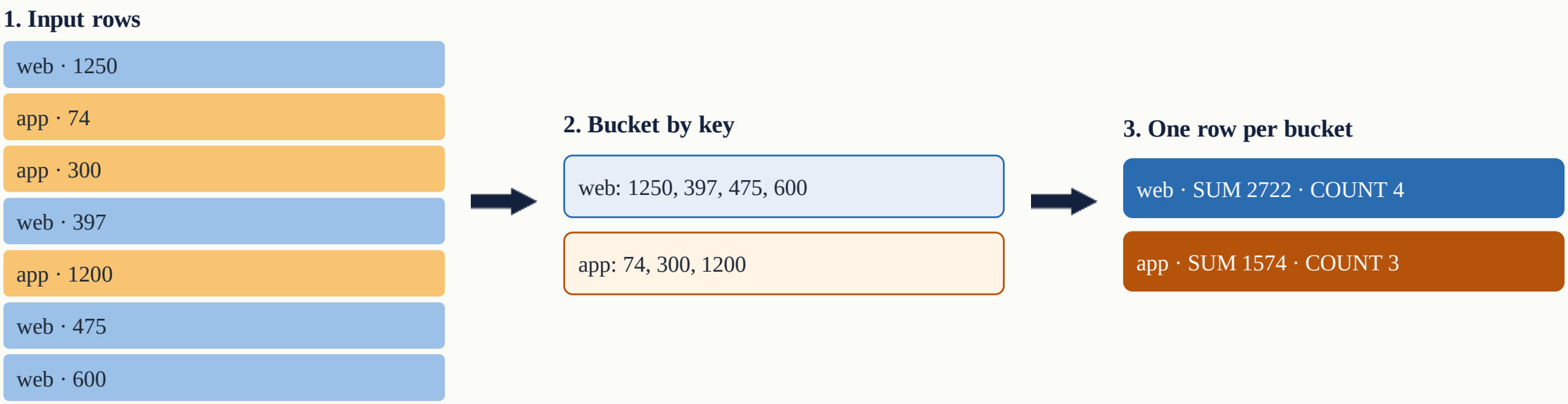
Aggregation and Grouping

Turning many rows into few: counting, summing and averaging, grouping by one or many keys, filtering groups, producing subtotals, approximating at scale and pivoting rows into columns.

COUNT variants · SUM, AVG, MIN, MAX · GROUP BY · GROUP BY ALL and STRING_AGG · HAVING · COUNTIF and conditional aggregation · ROLLUP · CUBE and GROUPING SETS · Approximate aggregation · PIVOT and UNPIVOT

PART 7 · AGGREGATION

What GROUP BY actually does



The golden rule. Every column in SELECT must either appear in GROUP BY or be wrapped in an aggregate function. Selecting a raw column that varies within a group raises an error: SELECT list expression references column which is neither grouped nor aggregated.

PART 7 · AGGREGATION

Three ways to COUNT

QUERY

```
SELECT
  COUNT(*) AS all_orders,
  COUNT(customer_id) AS with_customer,
  COUNT(DISTINCT customer_id) AS buyers,
  COUNT(DISTINCT channel) AS channels
FROM shop.orders;
```

OUTPUT · 1 ROW

all_orders	with_customer	buyers	channels
8	8	5	2

COUNT(*) counts rows, NULL or not.

COUNT(col) counts non NULL values only.

COUNT(DISTINCT col) counts unique non NULL values. Exact in BigQuery, at any scale.

Reading the output. Eight orders, all with a customer, so the first two match. Only five distinct customers bought: Amara and Ben ordered twice, Chloe ordered twice including a cancellation, and Farid never ordered. Two channels exist: web and app.

PART 7 · AGGREGATION

SUM, AVG, MIN and MAX over all order lines

QUERY

```
SELECT
  COUNT(*) AS lines,
  SUM(quantity) AS units,
  SUM(quantity * unit_price) AS revenue,
  ROUND(AVG(quantity * unit_price), 2) AS avg_line,
  MIN(quantity * unit_price) AS min_line,
  MAX(quantity * unit_price) AS max_line
FROM shop.order_items;
```

OUTPUT · 1 ROW

column	value
lines	11
units	18
revenue	4746
avg_line	431.45
min_line	25
max_line	1200

Reading the output. With no GROUP BY, the whole table is one group, so one row returns. Revenue 4746 includes the cancelled order; we exclude it from now on. The average is 4746 ÷ 11 = 431.4545…, rounded to 431.45. The smallest line is the single mouse in order 5007; the largest is a laptop. All of these aggregates ignore NULL inputs.

PART 7 · AGGREGATION

GROUP BY one key: revenue by category

QUERY

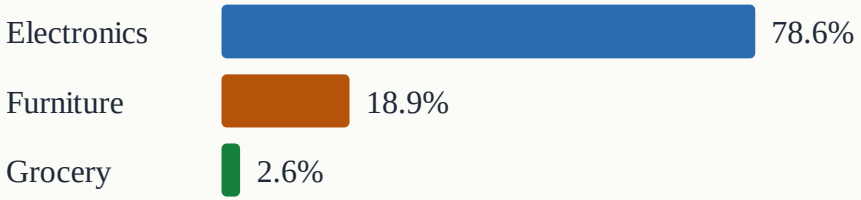
```
SELECT
  p.category,
  COUNT(*) AS lines,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.order_items AS i
JOIN shop.orders AS o USING (order_id)
JOIN shop.products AS p USING (product_id)
WHERE o.status != 'cancelled'
GROUP BY p.category
ORDER BY revenue DESC;
```

The joins are explained fully in Part 8.

OUTPUT · 3 ROWS

category	lines	revenue
Electronics	6	3375
Furniture	2	810
Grocery	2	111

The output as a chart



Reading the output. The three totals sum to 4296, the non cancelled revenue. Furniture has two lines because the cancelled standing desk was filtered out by WHERE before grouping.

PART 7 · AGGREGATION

GROUP BY two keys: month and channel

QUERY

```
SELECT
  FORMAT_DATE('%b %Y', o.order_date) AS month,
  o.channel,
  COUNT(DISTINCT o.order_id) AS orders,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.orders AS o
JOIN shop.order_items AS i USING (order_id)
WHERE o.status != 'cancelled'
GROUP BY month, o.channel
ORDER BY MIN(o.order_date);
```

OUTPUT · 6 ROWS

month	channel	orders	revenue
Jan 2026	web	1	1250
Jan 2026	app	1	74
Feb 2026	app	1	300
Mar 2026	web	2	872
Mar 2026	app	1	1200
Apr 2026	web	1	600

Reading the output. One row per existing month and channel combination; combinations with no sales, such as Feb web, simply do not appear. COUNT(DISTINCT order_id) counts orders, not lines: March web holds orders 5005 and 5007 (397 + 475 = 872). Ordering by MIN(order_date) gives true calendar order, which sorting the text label would not.

PART 7 · AGGREGATION

GROUP BY ALL and STRING_AGG

QUERY

```
SELECT
  country,
  COUNT(*) AS customers,
  STRING_AGG(name, ', ' ORDER BY name) AS names
FROM shop.customers
GROUP BY ALL
ORDER BY customers DESC, country;
```

OUTPUT · 4 ROWS

country	customers	names
UK	3	Amara Okafor, Ben Clarke, Farid Haddad
France	1	Chloe Martin
Ghana	1	David Osei
Italy	1	Elena Rossi

Reading the output. GROUP BY ALL tells BigQuery to group by every SELECT expression that is not an aggregate, here just country. It saves retyping long expressions and prevents the classic error of forgetting one. STRING_AGG concatenates the names within each group, sorted alphabetically by its own ORDER BY.

ARRAY_AGG does the same but builds an ARRAY instead of a string. Part 13.

ANY_VALUE(col) returns some value from the group when you know they are all equal.

PART 7 · AGGREGATION

HAVING: filtering groups, not rows

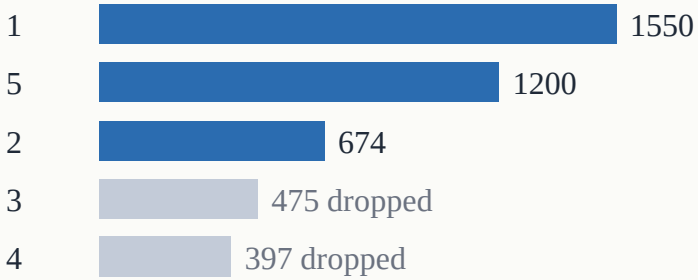
QUERY

```
SELECT
  o.customer_id,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.orders AS o
JOIN shop.order_items AS i USING (order_id)
WHERE o.status != 'cancelled' # row filter
GROUP BY o.customer_id
HAVING revenue > 500 # group filter
ORDER BY revenue DESC;
```

OUTPUT · 3 OF 5 GROUPS

customer_id	revenue
1	1550
5	1200
2	674

All five groups before HAVING



Reading the output. WHERE removed the cancelled order first, so Chloe’s total is only 475. HAVING then removed groups at or below 500. Put conditions on raw columns in WHERE: it shrinks the data before the expensive grouping.

PART 7 · AGGREGATION

Conditional aggregation with COUNTIF and LOGICAL_OR

QUERY

```
SELECT
  channel,
  COUNT(*) AS orders,
  COUNTIF(status = 'delivered') AS delivered,
  COUNTIF(status = 'shipped') AS shipped,
  COUNTIF(status = 'cancelled') AS cancelled,
  LOGICAL_OR(status = 'cancelled') AS any_cancel
FROM shop.orders
GROUP BY channel
ORDER BY orders DESC;
```

OUTPUT · 2 ROWS

channel	orders	delivered	shipped	cancelled	any
web	5	3	1	1	true
app	3	2	1	0	false

Reading the output. One pass over the table produces a status breakdown per channel. Each row's counts add up: web 3 + 1 + 1 = 5. LOGICAL_OR is TRUE if any row in the group satisfies the condition; LOGICAL_AND requires all rows.

Equivalent older pattern: SUM(IF(status = 'shipped', 1, 0)). COUNTIF is shorter and clearer.

PART 7 · AGGREGATION

ROLLUP: subtotals and a grand total in one query

QUERY

```
SELECT
  p.category,
  p.product_name,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.order_items AS i
JOIN shop.orders AS o USING (order_id)
JOIN shop.products AS p USING (product_id)
WHERE o.status != 'cancelled'
GROUP BY ROLLUP (p.category, p.product_name)
ORDER BY p.category NULLS LAST,
  p.product_name NULLS LAST;
```

Reading the output. ROLLUP(a, b) groups by (a, b), then (a), then (). NULL in a key column marks a subtotal row. Use GROUPING(col) to tell a subtotal NULL from a real NULL.

OUTPUT · 10 ROWS

category	product_name	revenue
Electronics	Headphones	900
Electronics	Laptop Pro 14	2400
Electronics	Wireless Mouse	75
Electronics	NULL	3375
Furniture	Desk Chair	360
Furniture	Standing Desk	450
Furniture	NULL	810
Grocery	Coffee Beans 1kg	111
Grocery	NULL	111
NULL	NULL	4296

PART 7 · AGGREGATION

GROUPING SETS, CUBE and the GROUPING function

QUERY

```
SELECT
  CASE
    WHEN GROUPING(channel) = 0 THEN 'channel'
    WHEN GROUPING(status) = 0 THEN 'status'
    ELSE 'total'
  END AS level,
  COALESCE(channel, status, 'all') AS value,
  COUNT(*) AS orders
FROM shop.orders
GROUP BY GROUPING SETS ((channel), (status), ())
ORDER BY level, value;
```

OUTPUT · 6 ROWS

level	value	orders
channel	app	3
channel	web	5
status	cancelled	1
status	delivered	5
status	shipped	2
total	all	8

GROUPING SETS lists exactly which groupings to compute, here two independent breakdowns plus a total.

CUBE(a, b) computes every combination: (a, b), (a), (b) and (). For n keys that is 2 to the power n groupings.

GROUPING(col) returns 0 when col is part of the current grouping, 1 when it was rolled up.

PART 7 · AGGREGATION

Approximate aggregation for very large data

QUERY

```
SELECT
  APPROX_COUNT_DISTINCT(customer_id) AS buyers,
  APPROX_TOP_COUNT(channel, 2) AS top_channels
FROM shop.orders;
```

OUTPUT · JSON VIEW

```
{ "buyers": 5,
  "top_channels": [
    { "value": "web", "count": 5 },
    { "value": "app", "count": 3 } ] }
```

Why approximate? Exact COUNT(DISTINCT) must shuffle every distinct value to one place. On billions of user IDs that is slow and slot hungry.

HyperLogLog++ keeps a small fixed size sketch instead, giving answers typically within about one percent using a fraction of the memory.

Sketches compose: HLL_COUNT.INIT builds daily sketches you store; HLL_COUNT.MERGE combines them into weekly or monthly uniques without rescanning raw data.

Also: APPROX_QUANTILES for medians and percentiles.

Reading the output. On tiny data the approximation is exact: 5 buyers, matching COUNT(DISTINCT). APPROX_TOP_COUNT returns an ARRAY of STRUCTs, which is why the JSON view is the clearest way to read it.

PART 7 · AGGREGATION

PIVOT: turning rows into columns

QUERY

```
WITH order_rev AS (
  SELECT o.channel,
    FORMAT_DATE('%b', o.order_date) AS mon,
    i.quantity * i.unit_price AS revenue
  FROM shop.orders AS o
  JOIN shop.order_items AS i USING (order_id)
  WHERE o.status != 'cancelled'
)
SELECT *
FROM order_rev
PIVOT (SUM(revenue)
  FOR mon IN ('Jan', 'Feb', 'Mar', 'Apr'))
ORDER BY channel DESC;
```

OUTPUT · 2 ROWS

channel	Jan	Feb	Mar	Apr
web	1250	NULL	872	600
app	74	300	1200	NULL

Reading the output. Every column not named in PIVOT, here channel, becomes a grouping key. Each IN value becomes a column holding SUM(revenue) for that month. Feb web is NULL because its only order was cancelled; wrap with COALESCE to show 0.

UNPIVOT reverses this, turning the Jan to Apr columns back into month and revenue rows. The IN list must be written out; it cannot come from a subquery.

08

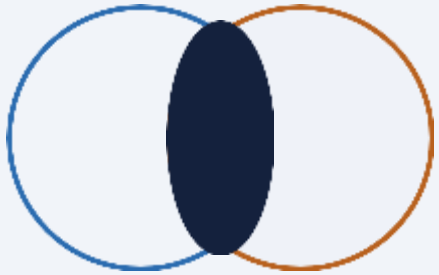
Joins

Combining tables is where analytical SQL becomes powerful and where most mistakes happen. This part covers every join type, the patterns built on them, the fan out trap, and how BigQuery physically executes a join.

Join types visualised · INNER · LEFT · RIGHT · FULL OUTER · CROSS · Self join · Multi table joins · USING versus ON · Semi and anti joins · Fan out · Broadcast and hash joins · Best practice

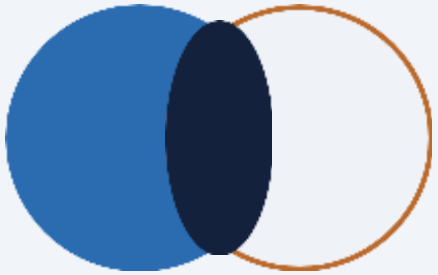
PART 8 · JOINS

Four join types, four shapes



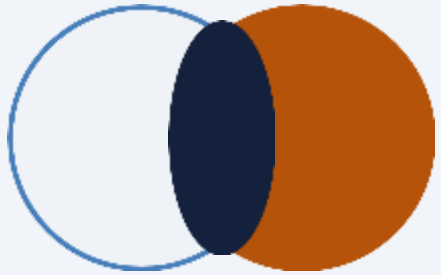
INNER JOIN

Only rows with a match on both sides. Unmatched rows from either table vanish.



LEFT JOIN

All left rows; right columns are NULL where no match exists.



RIGHT JOIN

All right rows; left columns are NULL where no match exists.



FULL OUTER

Everything from both sides, matched where possible, NULL padded otherwise.

Blue circle: left table. Amber circle: right table. Navy lens: matched rows. Filled areas are returned.

PART 8 · JOINS

INNER JOIN: orders with customer names

QUERY

```
SELECT
  o.order_id,
  c.name,
  o.status
FROM shop.orders AS o
INNER JOIN shop.customers AS c
  ON c.customer_id = o.customer_id
ORDER BY o.order_id;
```

Reading the output. Each order found exactly one customer, so 8 orders yield 8 rows. Farid has no order, so he is absent: INNER JOIN silently drops non matching rows on both sides. JOIN on its own means INNER JOIN.

OUTPUT · 8 ROWS

order_id	name	status
5001	Amara Okafor	delivered
5002	Ben Clarke	delivered
5003	Amara Okafor	delivered
5004	Chloe Martin	cancelled
5005	David Osei	delivered
5006	Elena Rossi	shipped
5007	Chloe Martin	delivered
5008	Ben Clarke	shipped

PART 8 · JOINS

LEFT JOIN: keep every customer, even without orders

QUERY

```
SELECT
  c.name,
  COUNT(o.order_id) AS orders,
  MAX(o.order_date) IS NULL AS never_ordered
FROM shop.customers AS c
LEFT JOIN shop.orders AS o
  ON o.customer_id = c.customer_id
GROUP BY c.name
ORDER BY orders DESC, c.name;
```

OUTPUT · 6 ROWS

name	orders	never_ordered
Amara Okafor	2	false
Ben Clarke	2	false
Chloe Martin	2	false
David Osei	1	false
Elena Rossi	1	false
Farid Haddad	0	true

Reading the output. Farid survives the LEFT JOIN with NULLs in every orders column. COUNT(o.order_id) ignores those NULLs and gives 0, whereas COUNT(*) would wrongly give 1. **Trap:** a WHERE condition on the right table, such as o.status = 'delivered', removes the NULL rows and silently turns the LEFT JOIN back into an INNER JOIN; put such conditions in the ON clause instead.

PART 8 · JOINS

RIGHT JOIN is a LEFT JOIN written backwards

```
SELECT c.name, o.order_id
FROM shop.orders AS o
RIGHT JOIN shop.customers AS c
  ON o.customer_id = c.customer_id;
```

=

same 9 rows

```
SELECT c.name, o.order_id
FROM shop.customers AS c
LEFT JOIN shop.orders AS o
  ON o.customer_id = c.customer_id;
```

Why 9 rows?

8 matched order rows plus 1 row for Farid with a NULL order_id.

House style

Most teams standardise on LEFT JOIN, putting the table to preserve first. Queries read top down as “start with all customers, add orders”.

When RIGHT helps

Occasionally, appending one more preserved table at the end of a long join chain without reordering everything.

PART 8 · JOINS

FULL OUTER JOIN: reconciling targets with actuals

QUERY

```
WITH targets AS (
  SELECT 'UK' AS country, 4 AS target UNION ALL
  SELECT 'France', 2 UNION ALL
  SELECT 'Spain', 1),
actual AS (
  SELECT country, COUNT(*) AS customers
  FROM shop.customers GROUP BY country)
SELECT
  COALESCE(t.country, a.country) AS country,
  t.target, a.customers
FROM targets AS t
FULL OUTER JOIN actual AS a
  ON a.country = t.country
ORDER BY country;
```

OUTPUT · 5 ROWS

country	target	customers
France	2	1
Ghana	NULL	1
Italy	NULL	1
Spain	1	NULL
UK	4	3

Reading the output. France and UK matched on both sides. Ghana and Italy exist only in actuals, Spain only in targets. COALESCE picks whichever side has the key, so no row shows a blank country. FULL OUTER JOIN is the standard tool for reconciliation and data comparison.

PART 8 · JOINS

CROSS JOIN: every combination, by design

QUERY

```
SELECT channel, mon
FROM UNNEST(['web', 'app']) AS channel
CROSS JOIN UNNEST(
  ['Jan', 'Feb', 'Mar', 'Apr']) AS mon;
```

Reading the output. 2 channels × 4 months = 8 rows, every pairing. This is a **spine**: LEFT JOIN actual sales onto it and missing combinations such as Feb web appear as explicit zeros instead of disappearing from a report.

Danger. A CROSS JOIN of two million row tables produces a trillion rows. A comma between tables in FROM also means CROSS JOIN.

OUTPUT · 8 ROWS AS A GRID

	Jan	Feb	Mar	Apr
web	web Jan	web Feb	web Mar	web Apr
app	app Jan	app Feb	app Mar	app Apr

The query returns these as 8 rows of (channel, mon); the grid simply shows that each row is one cell of the product.

PART 8 · JOINS

Self join: a table joined to itself

QUERY: CUSTOMERS SHARING A CITY

```
SELECT
  a.name AS customer,
  b.name AS neighbour,
  a.city
FROM shop.customers AS a
JOIN shop.customers AS b
  ON a.city = b.city
 AND a.customer_id < b.customer_id;
```

OUTPUT · 1 ROW

customer	neighbour	city
Amara Okafor	Farid Haddad	Manchester

Reading the output. Two aliases, a and b, let one table play two roles. Without a.customer_id < b.customer_id we would get 8 rows: every customer paired with themselves (6), plus Amara with Farid and Farid with Amara. The inequality removes self pairs and mirror duplicates.

Hierarchies: employee joined to manager row.

Sequences: each event joined to the previous one (though LAG is usually better, Part 10).

Pairs: products bought together in the same order.

PART 8 · JOINS

Joining four tables: customer value report

QUERY

```
SELECT
  c.name,
  COUNT(DISTINCT o.order_id) AS orders,
  COUNT(DISTINCT p.category) AS categories,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.customers AS c
JOIN shop.orders AS o
  ON o.customer_id = c.customer_id
JOIN shop.order_items AS i
  ON i.order_id = o.order_id
JOIN shop.products AS p
  ON p.product_id = i.product_id
WHERE o.status != 'cancelled'
GROUP BY c.name
ORDER BY revenue DESC;
```

OUTPUT · 5 ROWS

name	orders	categories	revenue
Amara Okafor	2	1	1550
Elena Rossi	1	1	1200
Ben Clarke	2	2	674
Chloe Martin	1	2	475
David Osei	1	2	397

Reading the output. Each join walks one relationship arrow of the schema. Amara bought only electronics across two orders. Chloe now shows 1 order because the cancelled one was filtered out. Totals still sum to 4296, proving no row was lost or duplicated.

PART 8 · JOINS

ON versus USING

ON condition

```
JOIN shop.orders AS o
  ON o.customer_id = c.customer_id
```

- Any boolean condition, including ranges and inequalities
- Columns may have different names
- Both key columns stay in SELECT *

USING (columns)

```
JOIN shop.orders AS o
  USING (customer_id)
```

- Equality only, on identically named columns
- Shorter to write
- The key appears once in SELECT *

Subtle difference. In a FULL OUTER JOIN with USING, the merged key is non NULL whenever either side has it, like an automatic COALESCE. With ON, you must COALESCE the two key columns yourself, as on the reconciliation slide.

PART 8 · JOINS

Semi joins and anti joins with EXISTS

SEMI JOIN: HAS A DELIVERED ORDER

```
SELECT c.name
FROM shop.customers AS c
WHERE EXISTS (
  SELECT 1 FROM shop.orders AS o
  WHERE o.customer_id = c.customer_id
    AND o.status = 'delivered')
ORDER BY c.name;
```

name · 4 rows
Amara Okafor
Ben Clarke
Chloe Martin
David Osei

ANTI JOIN: HAS NO ORDERS AT ALL

```
SELECT c.name
FROM shop.customers AS c
WHERE NOT EXISTS (
  SELECT 1 FROM shop.orders AS o
  WHERE o.customer_id = c.customer_id);
```

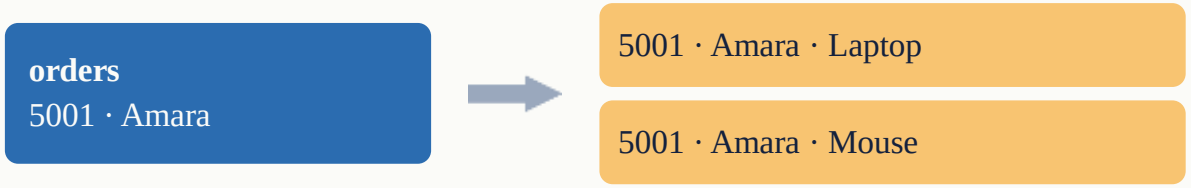
name · 1 row
Farid Haddad

Why not JOIN? A semi join returns each customer once, however many matching orders exist. Amara has two delivered orders but appears once. Elena is missing: her only order is shipped.

Beware NOT IN: if the subquery returns any NULL, x NOT IN (…) is never TRUE and the query silently returns nothing. NOT EXISTS has no such trap.

PART 8 · JOINS

The fan out trap: when joins multiply rows



After joining order_items, one order becomes two rows. Anything counted or summed at order level is now doubled.

```
SELECT c.name,
  COUNT(*) AS wrong_orders,
  COUNT(DISTINCT o.order_id) AS right_orders
FROM shop.customers AS c
JOIN shop.orders AS o USING (customer_id)
JOIN shop.order_items AS i USING (order_id)
GROUP BY c.name ORDER BY c.name;
```

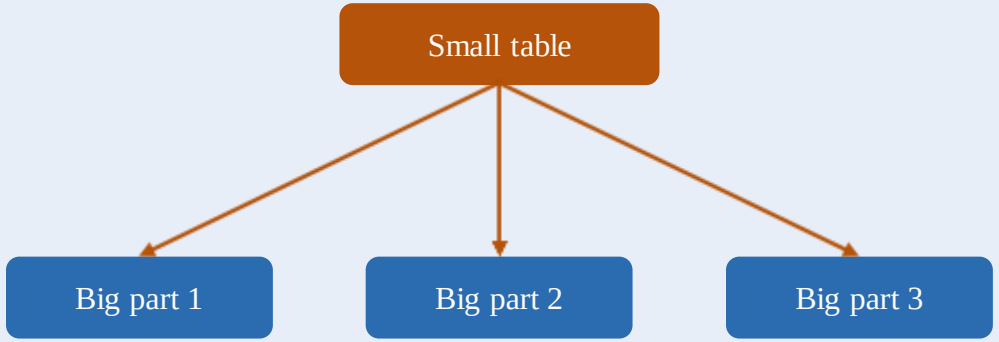
name	wrong_orders	right_orders
Amara Okafor	3	2
Ben Clarke	2	2
Chloe Martin	3	2
David Osei	2	1
Elena Rossi	1	1

Reading the output. COUNT(*) counts order lines, not orders; red values are inflated. Fixes: count distinct keys, or aggregate each table to the right grain in a CTE before joining. Always ask: what does one row represent after this join?

PART 8 · JOINS

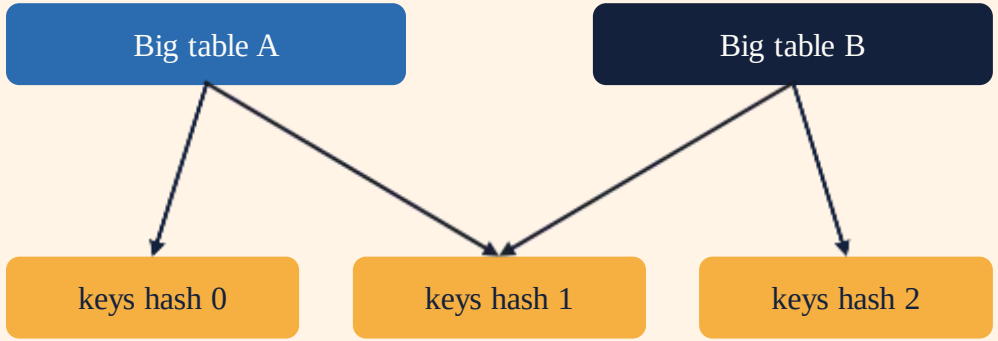
How BigQuery executes a join

Broadcast join



The small table is copied to every worker holding a piece of the big table. The big table never moves: fast and cheap.

Hash (shuffle) join



Both sides are shuffled so equal keys meet on the same worker. Needed when neither side is small; costs network and shuffle bytes.

The optimiser picks the strategy automatically from table statistics and can switch at run time. Skewed keys, where one value holds most rows, overload a single worker and show up as a long tail in the execution graph.

PART 8 · RECAP

Eight rules for correct, fast joins

1. Know the grain of each table before joining it.

2. Reconcile row counts and totals after every new join.

3. Filter and aggregate before joining to shrink shuffles.

4. Join on keys of the same type; casting inside ON blocks optimisations.

5. Put outer table conditions in ON, not WHERE, to keep LEFT JOIN semantics.

6. Prefer EXISTS and NOT EXISTS for presence and absence tests.

7. Watch for skewed keys and NULL keys in large joins.

8. Consider nesting child rows as ARRAYs to avoid the join entirely (Part 13).

09

Subqueries, CTEs and Set Operations

Queries inside queries, named building blocks that turn tangled SQL into readable pipelines, recursion for hierarchies, and the operators that stack or compare whole result sets.

Scalar subqueries · IN and correlated subqueries · Derived tables · WITH clauses · Chained CTE pipelines · Recursive CTEs · UNION, INTERSECT, EXCEPT

Scalar subqueries: orders above the average

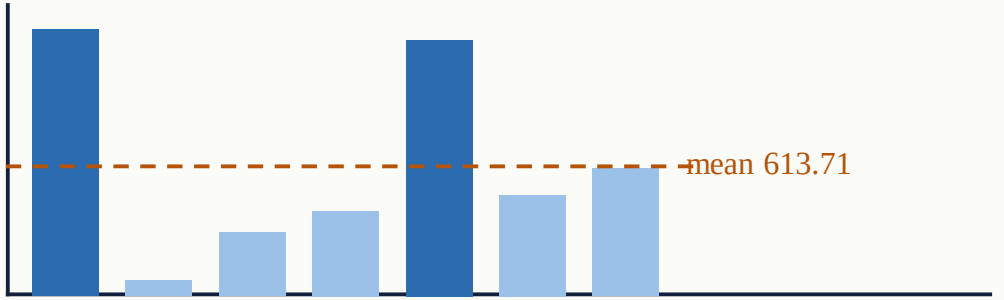
QUERY

```
WITH order_value AS (
  SELECT order_id, SUM(quantity * unit_price) AS value
  FROM shop.order_items
  JOIN shop.orders USING (order_id)
  WHERE status != 'cancelled'
  GROUP BY order_id)
SELECT
  order_id, value,
  ROUND((SELECT AVG(value) FROM order_value), 2) AS avg_value
FROM order_value
WHERE value > (SELECT AVG(value) FROM order_value)
ORDER BY value DESC;
```

OUTPUT · 2 ROWS

order_id	value	avg_value
5001	1250	613.71
5006	1200	613.71

All seven order values against the mean



Reading the output. A scalar subquery returns exactly one value and can appear wherever a single value is allowed. Seven valid orders total 4296, so the mean is 613.71; only the two laptop orders exceed it. If a scalar subquery returns more than one row, the query fails.

Correlated subqueries: one evaluation per outer row

QUERY

```
SELECT
  c.name,
  FORMAT_DATE('%d %b %Y', (
    SELECT MAX(o.order_date)
    FROM shop.orders AS o
    WHERE o.customer_id = c.customer_id
  )) AS last_order
FROM shop.customers AS c
ORDER BY c.customer_id;
```

OUTPUT · 6 ROWS

name	last_order
Amara Okafor	14 Feb 2026
Ben Clarke	03 Apr 2026
Chloe Martin	28 Mar 2026
David Osei	02 Mar 2026
Elena Rossi	15 Mar 2026
Farid Haddad	NULL

Reading the output. The inner query refers to c.customer_id from the outer row, so conceptually it runs once per customer. Farid has no orders, MAX over zero rows is NULL, and FORMAT_DATE of NULL is NULL. BigQuery rewrites most correlated subqueries into joins internally, so they perform well; still, a LEFT JOIN with GROUP BY is often clearer.

Derived tables: a subquery in FROM

QUERY: AVERAGE LINES PER ORDER

```
SELECT
  COUNT(*) AS orders,
  AVG(n_lines) AS avg_lines,
  MAX(n_lines) AS max_lines
FROM (
  SELECT order_id, COUNT(*) AS n_lines
  FROM shop.order_items
  GROUP BY order_id
) AS per_order;
```

INNER RESULT · 8 ROWS

5001: 2	5002: 1	5003: 1	5004: 1
5005: 2	5006: 1	5007: 2	5008: 1

FINAL OUTPUT · 1 ROW

orders	avg_lines	max_lines
8	1.375	2

Reading the output. Two stages of aggregation: first to one row per order, then across orders. 11 lines over 8 orders gives 1.375. AVG of an INT64 column returns FLOAT64. Aggregating an aggregate always needs a subquery or CTE like this.

The WITH clause: name each step

Nested: read inside out

```
SELECT tier, SUM(rev)
FROM (
  SELECT c.tier, x.rev
  FROM shop.customers c
  JOIN (
    SELECT customer_id, SUM(…) rev
    FROM (… more nesting …)
    GROUP BY customer_id
  ) x USING (customer_id)
)
GROUP BY tier
```

CTEs: read top down

```
WITH
line_rev AS (… one row per line …),
customer_rev AS (
  SELECT customer_id, SUM(rev) AS rev
  FROM line_rev GROUP BY customer_id),
tier_rev AS (
  SELECT c.tier, SUM(r.rev) AS rev
  FROM shop.customers c
  LEFT JOIN customer_rev r USING (customer_id)
  GROUP BY c.tier)
SELECT * FROM tier_rev
```

Readable: each step has a name describing its grain.

Testable: run SELECT * FROM any CTE to debug that step.

Not cached: a CTE referenced twice may be computed twice. Use a temp table if it is expensive.

A CTE pipeline: revenue share by loyalty tier

line_rev
10 valid lines

→

customer_rev
5 customers

→

tier_rev
3 tiers

→

final SELECT
share of total

```
# line_rev, customer_rev, tier_rev as on the previous slide
SELECT
  tier,
  IFNULL(rev, 0) AS revenue,
  ROUND(100 * IFNULL(rev, 0)
    / SUM(rev) OVER (), 1) AS pct
FROM tier_rev
ORDER BY revenue DESC;
```

tier	revenue	pct
Gold	2025	47.1
Silver	1874	43.6
Bronze	397	9.2



Gold · Silver · Bronze as one 100% bar

Reading the output. Gold is Amara 1550 plus Chloe 475; Silver is Ben 674 plus Elena 1200; Bronze is David 397 plus Farid, whose NULL revenue is ignored by SUM. SUM(rev) OVER () is a window total, a preview of Part 10.

Recursive CTEs: walking a hierarchy

QUERY

```
WITH RECURSIVE
cat AS (
  SELECT 1 AS id, 'All products' AS name,
    CAST(NULL AS INT64) AS parent UNION ALL
  SELECT 2, 'Electronics', 1 UNION ALL
  SELECT 3, 'Audio', 2 UNION ALL
  SELECT 4, 'Headphones', 3),
tree AS (
  SELECT id, 0 AS depth, name AS path
  FROM cat WHERE parent IS NULL
  UNION ALL
  SELECT c.id, t.depth + 1,
    CONCAT(t.path, ' > ', c.name)
  FROM cat c JOIN tree t ON c.parent = t.id)
SELECT depth, path FROM tree ORDER BY depth;
```

OUTPUT · 4 ROWS

depth	path
0	All products
1	All products > Electronics
2	All products > Electronics > Audio
3	All products > Electronics > Audio > Headphones

Reading the output. The first branch is the anchor: the root row. The second branch runs repeatedly, each time joining children to rows found in the previous round, until no new rows appear. BigQuery caps recursion at 500 iterations by default, protecting you from infinite loops.

PART 9 · SET OPERATIONS

UNION, INTERSECT and EXCEPT

```
SELECT customer_id FROM shop.orders
WHERE channel = 'web'
INTERSECT DISTINCT # or EXCEPT, UNION
SELECT customer_id FROM shop.orders
WHERE channel = 'app';
```

web buyers
1, 3, 4, 3, 2 → {1, 2, 3, 4}

app buyers
2, 1, 5 → {1, 2, 5}

Both queries must return the same number of columns with compatible types, matched by position, not by name.

Operator	Result	Rows
UNION ALL	every row kept	8
UNION DISTINCT	1, 2, 3, 4, 5	5
INTERSECT DISTINCT	1, 2	2
EXCEPT DISTINCT	3, 4	2

Reading the output. Amara and Ben used both channels. Chloe and David bought only on the web. UNION ALL keeps all 5 + 3 rows including duplicates and is the cheapest; the DISTINCT forms must deduplicate. BigQuery requires the ALL or DISTINCT keyword to be written out.

10

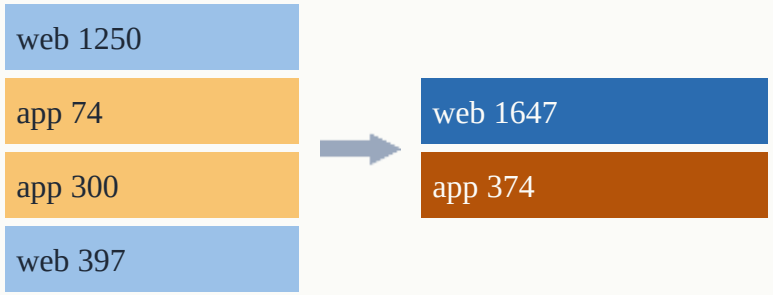
Window Functions

Calculations across related rows without collapsing them: running totals, rankings, comparisons with the previous period, moving averages and deduplication. The single most valuable skill for analytical SQL.

Window versus aggregate · OVER anatomy · Running totals · ROW_NUMBER, RANK, DENSE_RANK · QUALIFY top N · LAG and LEAD · Percent of total · Moving averages · Frames · FIRST_VALUE and LAST_VALUE · NTILE and distribution · Named windows · Deduplication

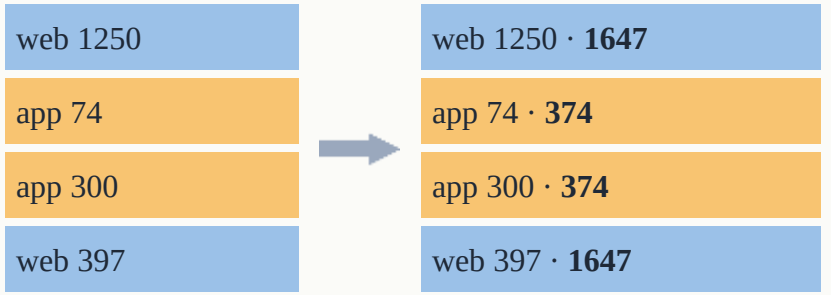
Aggregate versus window: rows collapse or rows stay

GROUP BY channel



4 rows in, 2 rows out. Detail is lost.

SUM() OVER (PARTITION BY channel)



4 rows in, 4 rows out. Each row gains its group’s total.

A window function looks through a **window** of related rows to compute a value, then writes that value onto the current row. Nothing is collapsed, so you can compare each row with its group.

Structure of the OVER clause

Function: what to compute

```
SUM(value) OVER (PARTITION BY customer_id ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

PARTITION BY: split rows into independent groups. Omit it and the whole result is one window.

ORDER BY: sequence inside each partition. Required for ranking, LAG and running totals.

Frame: which rows relative to the current one are included. Optional, with defaults.

Aggregate windows
SUM, AVG, COUNT, MIN, MAX and friends with OVER

Ranking windows
ROW_NUMBER, RANK, DENSE_RANK, NTILE, PERCENT_RANK, CUME_DIST

Navigation windows
LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

PART 10 · WINDOW FUNCTIONS

Running totals, overall and per customer

QUERY

```
# order_value: one row per valid order
SELECT
  order_id, customer_id, value,
  SUM(value) OVER (
    ORDER BY order_date) AS running_total,
  SUM(value) OVER (
    PARTITION BY customer_id
    ORDER BY order_date) AS customer_running
FROM order_value
ORDER BY order_date;
```

OUTPUT · 7 ROWS

order	cust	value	running	cust_run
5001	1	1250	1250	1250
5002	2	74	1324	74
5003	1	300	1624	1550
5005	4	397	2021	397
5006	5	1200	3221	1200
5007	3	475	3696	475
5008	2	600	4296	674

Reading the output. With ORDER BY and no frame, the default frame runs from the first row to the current row, so the sum accumulates. running_total ends at 4296. customer_running restarts for each customer: Amara reaches 1550 on her second order, Ben 674 on his.

PART 10 · WINDOW FUNCTIONS

ROW_NUMBER, RANK and DENSE_RANK with ties

QUERY

```
# cust_orders: name, orders (from the LEFT JOIN slide)
SELECT
  name, orders,
  ROW_NUMBER() OVER w AS row_num,
  RANK() OVER w AS rnk,
  DENSE_RANK() OVER w AS dense
FROM cust_orders
WINDOW w AS (ORDER BY orders DESC)
ORDER BY row_num;
```

OUTPUT · 6 ROWS

name	orders	row_num	rnk	dense
Amara Okafor	2	1	1	1
Ben Clarke	2	2	1	1
Chloe Martin	2	3	1	1
David Osei	1	4	4	2
Elena Rossi	1	5	4	2
Farid Haddad	0	6	6	3

ROW_NUMBER always 1, 2, 3… Ties are broken arbitrarily, so add a tiebreaker column for repeatable results.

RANK gives ties the same number, then skips: three firsts, so the next is 4, like an Olympic podium.

DENSE_RANK gives ties the same number without gaps: 1, 1, 1, 2, 2, 3.

PART 10 · WINDOW FUNCTIONS

Top N per group with QUALIFY

QUERY: TWO BEST SELLERS PER CATEGORY

```
SELECT
  p.category,
  p.product_name,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.order_items AS i
JOIN shop.orders AS o USING (order_id)
JOIN shop.products AS p USING (product_id)
WHERE o.status != 'cancelled'
GROUP BY p.category, p.product_name
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY p.category
  ORDER BY revenue DESC) <= 2
ORDER BY p.category, revenue DESC;
```

OUTPUT · 5 ROWS

category	product_name	revenue
Electronics	Laptop Pro 14	2400
Electronics	Headphones	900
Furniture	Standing Desk	450
Furniture	Desk Chair	360
Grocery	Coffee Beans 1kg	111

Reading the output. The window ranks products within each category after GROUP BY has computed revenue. QUALIFY keeps rank 1 and 2. Wireless Mouse (75) is third in Electronics and is dropped. Grocery has only one product, so it contributes one row.

Without QUALIFY you would wrap the query in a subquery and filter the rank in an outer WHERE.

PART 10 · WINDOW FUNCTIONS

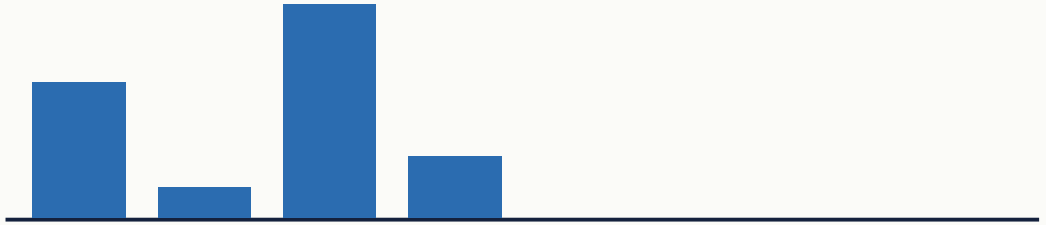
LAG and LEAD: comparing with neighbouring rows

QUERY

```
# monthly: month_start, revenue
SELECT
  FORMAT_DATE('%b', month_start) AS mon,
  revenue,
  LAG(revenue) OVER w AS prev_rev,
  ROUND(revenue / LAG(revenue) OVER w, 2) AS ratio,
  LEAD(revenue) OVER w AS next_rev
FROM monthly
WINDOW w AS (ORDER BY month_start)
ORDER BY month_start;
```

OUTPUT · 4 ROWS

mon	revenue	prev_rev	ratio	next_rev
Jan	1324	NULL	NULL	300
Feb	300	1324	0.23	2072
Mar	2072	300	6.91	600
Apr	600	2072	0.29	NULL



Jan · Feb · Mar · Apr revenue

Reading the output. LAG looks one row back, LEAD one row forward. The first month has no predecessor, so prev_rev and ratio are NULL. A ratio of 6.91 means March revenue was almost seven times February's. LAG(revenue, 12) would compare with the same month last year.

PART 10 · WINDOW FUNCTIONS

Percent of total and percent of group

QUERY

```
# customer_rev: name, country, revenue
SELECT
  name, country, revenue,
  ROUND(100 * revenue /
    SUM(revenue) OVER (), 1) AS pct_total,
  ROUND(100 * revenue /
    SUM(revenue) OVER (
      PARTITION BY country), 1) AS pct_country
FROM customer_rev
ORDER BY revenue DESC;
```

OUTPUT · 5 ROWS

name	country	revenue	pct_total	pct_country
Amara Okafor	UK	1550	36.1	69.7
Elena Rossi	Italy	1200	27.9	100
Ben Clarke	UK	674	15.7	30.3
Chloe Martin	France	475	11.1	100
David Osei	Ghana	397	9.2	100

Reading the output. OVER () with nothing inside makes the entire result one window, so the denominator is 4296 for every row and pct_total sums to 100. PARTITION BY country changes the denominator per country: the UK total is 2224, so Amara holds 69.7% of UK revenue. The other countries have a single buyer each, hence 100.

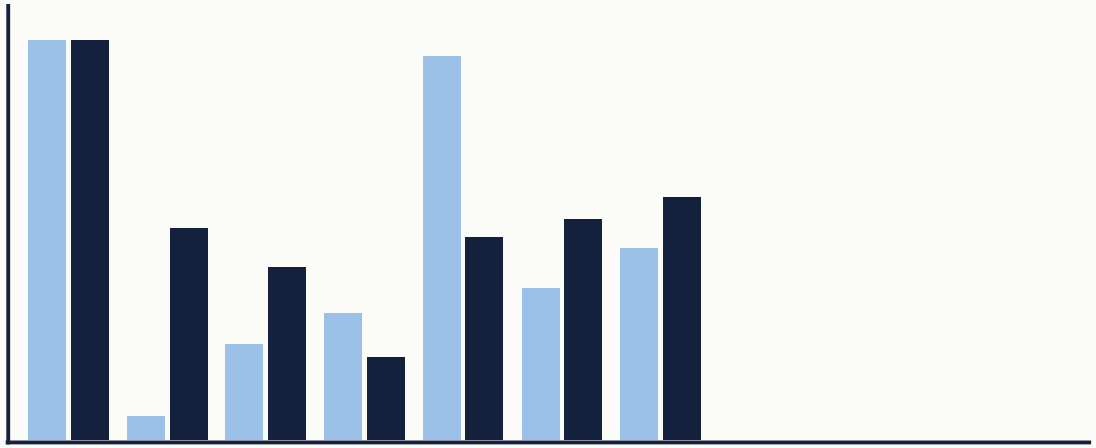
Moving averages with an explicit frame

QUERY

```
SELECT
  order_id, value,
  ROUND(AVG(value) OVER (
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING
      AND CURRENT ROW), 2) AS avg3
FROM order_value
ORDER BY order_date;
```

order_id	value	avg3
5001	1250	1250
5002	74	662
5003	300	541.33
5005	397	257
5006	1200	632.33
5007	475	690.67
5008	600	758.33

Raw values (light) versus 3 order moving average (dark)



Reading the output. Each avg3 averages the current order and up to two before it. The first two rows have fewer predecessors, so 5002 averages only two values: (1250 + 74) ÷ 2 = 662. The dark bars swing far less than the raw values: smoothing reveals the trend.

PART 10 · WINDOW FUNCTIONS

Window frames: ROWS versus RANGE

Seven ordered rows; the current row is row 4



Shaded: ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

Frame clause	Rows included for row 4
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	1, 2, 3, 4 (running total)
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW	2, 3, 4 (moving window)
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING	4, 5, 6, 7 (remaining)
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	All 7
RANGE BETWEEN INTERVAL 7 DAY PRECEDING AND CURRENT ROW	Rows whose date is within 7 days

The key difference. ROWS counts physical rows. RANGE uses the ORDER BY value, so tied values are included together and gaps in dates are respected. With ORDER BY and no frame, the default is RANGE UNBOUNDED PRECEDING, which includes all peers tied with the current row.

PART 10 · WINDOW FUNCTIONS

FIRST_VALUE, LAST_VALUE and the frame trap

QUERY

```
SELECT
  customer_id, order_id, value,
  FIRST_VALUE(value) OVER w AS first_val,
  LAST_VALUE(value) OVER (w
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND UNBOUNDED FOLLOWING) AS last_val
FROM order_value
WINDOW w AS (PARTITION BY customer_id
          ORDER BY order_date)
ORDER BY customer_id, order_date;
```

OUTPUT · 7 ROWS

cust	order	value	first_val	last_val
1	5001	1250	1250	300
1	5003	300	1250	300
2	5002	74	74	600
2	5008	600	74	600
3	5007	475	475	475
4	5005	397	397	397
5	5006	1200	1200	1200

Reading the output. first_val is each customer's first order value on every row. For last_val we widened the frame to the whole partition. With the default frame, LAST_VALUE would only see up to the current row and simply return the current value, a very common bug. NTH_VALUE(value, 2) fetches the second.

PART 10 · WINDOW FUNCTIONS

NTILE, PERCENT_RANK and CUME_DIST

QUERY

```
SELECT
  order_id, value,
  NTILE(3) OVER w AS tercile,
  ROUND(PERCENT_RANK() OVER w, 2) AS pct_rank,
  ROUND(CUME_DIST() OVER w, 2) AS cume
FROM order_value
WINDOW w AS (ORDER BY value)
ORDER BY value;
```

PERCENT_RANK = (rank minus 1) ÷ (rows minus 1)
CUME_DIST = rows at or below ÷ rows

OUTPUT · 7 ROWS

order	value	tercile	pct_rank	cume
5002	74	1	0	0.14
5003	300	1	0.17	0.29
5005	397	1	0.33	0.43
5007	475	2	0.5	0.57
5008	600	2	0.67	0.71
5006	1200	3	0.83	0.86
5001	1250	3	1	1

Reading the output. NTILE(3) splits 7 rows into buckets as equal as possible: 3, 2 and 2, earlier buckets taking the extra row. The cheapest order has percent rank 0 and the dearest 1. CUME_DIST of 0.43 for 5005 means 3 of 7 orders are worth 397 or less.

PART 10 · WINDOW FUNCTIONS

Named windows keep complex queries tidy

Repeated definitions

```
SUM(v) OVER (PARTITION BY c ORDER BY d),
AVG(v) OVER (PARTITION BY c ORDER BY d),
LAG(v) OVER (PARTITION BY c ORDER BY d),
ROW_NUMBER() OVER (PARTITION BY c ORDER BY d)
```

One named window

```
SUM(v) OVER w,
AVG(v) OVER w,
LAG(v) OVER w,
ROW_NUMBER() OVER w
…
WINDOW w AS (PARTITION BY c ORDER BY d)
```

Where it goes: the WINDOW clause sits after HAVING and before QUALIFY and ORDER BY.

Extending: OVER (w ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) adds a frame to a named window.

Performance: functions sharing one window can share one sort, which BigQuery exploits.

The deduplication pattern: latest version per key

QUERY

```
WITH status_feed AS (
  SELECT 5001 AS order_id, 'pending' AS status,
         1 AS seq UNION ALL
  SELECT 5001, 'delivered', 2 UNION ALL
  SELECT 5002, 'delivered', 1 UNION ALL
  SELECT 5003, 'pending', 1 UNION ALL
  SELECT 5003, 'shipped', 2 UNION ALL
  SELECT 5003, 'delivered', 3)
SELECT order_id, status, seq
FROM status_feed
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id ORDER BY seq DESC) = 1
ORDER BY order_id;
```

OUTPUT · 3 OF 6 ROWS KEPT

order_id	status	seq
5001	delivered	2
5002	delivered	1
5003	delivered	3

Reading the output. Within each order_id, rows are numbered from the highest seq down. Row number 1 is the newest version, and QUALIFY keeps only that. This is the standard way to clean at least once streams, change data capture feeds and duplicate loads.

Use a timestamp such as ingested_at instead of seq in real pipelines, with a tiebreaker for identical timestamps.

PART 10 · RECAP

Window function cheat sheet

Business question	Pattern
Cumulative revenue	SUM(x) OVER (ORDER BY date)
Rank within a group	RANK() OVER (PARTITION BY g ORDER BY x DESC)
Top N per group	QUALIFY ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC) <= N
Change from last period	x / LAG(x) OVER (ORDER BY period)
Share of total	x / SUM(x) OVER ()
Moving average	AVG(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
First event per user	FIRST_VALUE(event) OVER (PARTITION BY user ORDER BY ts)
Deduplicate	QUALIFY ROW_NUMBER() OVER (PARTITION BY key ORDER BY ts DESC) = 1
Quartiles	NTILE(4) OVER (ORDER BY x)

Remember: windows run after WHERE, GROUP BY and HAVING, so they can operate on aggregated values; filter their results with QUALIFY.

11

Strings and Regular Expressions

Text is the messiest data in any warehouse. These functions build labels, slice and search text, split strings into arrays, validate and extract with regular expressions, and normalise inconsistent input.

Core functions · CONCAT, ||, FORMAT · SUBSTR and STRPOS · SPLIT · REGEXP_CONTAINS, EXTRACT, REPLACE · Cleaning pipelines · Collation and CONTAINS_SUBSTR

PART 11 · STRINGS

Twelve everyday string functions and their results

Expression	Result
UPPER('okafor')	OKAFOR
LOWER('LONDON')	london
INITCAP('ben clarke')	Ben Clarke
LENGTH('Milan')	5
TRIM(' Paris ')	Paris
LPAD('7', 3, '0')	007

Expression	Result
LEFT('Standing Desk', 5)	Stand
RIGHT('Laptop Pro 14', 2)	14
REPLACE('web_app', '_', ' ')	web app
REPEAT('ab', 3)	ababab
STARTS_WITH('Laptop', 'Lap')	true
REVERSE('Accra')	arccA

Reading the results. Positions in GoogleSQL strings start at 1, not 0. Functions count Unicode characters, not bytes. Most return NULL when any argument is NULL, so LENGTH(NULL) is NULL, not 0. INITCAP capitalises the first letter of every word, handy for names but careful with surnames such as McDonald.

PART 11 · STRINGS

Building text: CONCAT, the || operator and FORMAT

QUERY

```
SELECT
  CONCAT(name, ' (', city, ')') AS label,
  LEFT(name, 1) ||
    LEFT(SPLIT(name, ' ')[OFFSET(1)], 1) AS initials,
  FORMAT('C%03d', customer_id) AS code
FROM shop.customers
WHERE customer_id <= 3
ORDER BY customer_id;
```

OUTPUT · 3 ROWS

label	initials	code
Amara Okafor (Manchester)	AO	C001
Ben Clarke (London)	BC	C002
Chloe Martin (Paris)	CM	C003

Reading the output. CONCAT joins any number of strings; || does the same between two. FORMAT uses printf style specifiers: %03d pads the integer to three digits with zeros. If any argument to CONCAT or || is NULL the whole result is NULL, so wrap optional parts in IFNULL.

%s any value as text

%d integer

%.2f two decimals

%'d thousands separator

PART 11 · STRINGS

Slicing text: LENGTH, STRPOS and SUBSTR

QUERY

```
SELECT
  product_name,
  LENGTH(product_name) AS len,
  STRPOS(product_name, ' ') AS space_at,
  SUBSTR(product_name,
    STRPOS(product_name, ' ') + 1) AS rest
FROM shop.products
ORDER BY product_id;
```

SUBSTR(s, start, length): start is 1 based; omit length to take everything to the end.

OUTPUT · 6 ROWS

product_name	len	space_at	rest
Laptop Pro 14	13	7	Pro 14
Wireless Mouse	14	9	Mouse
Desk Chair	10	5	Chair
Standing Desk	13	9	Desk
Coffee Beans 1kg	16	7	Beans 1kg
Headphones	10	0	Headphones

Reading the output. STRPOS returns the 1 based position of the first match. For Headphones there is no space, so STRPOS returns 0, SUBSTR starts at position 1, and the whole name comes back. Always test edge cases like this: the highlighted row is exactly where text logic breaks in production.

PART 11 · STRINGS

SPLIT: from a string to an ARRAY

QUERY

```
SELECT
  name,
  SPLIT(name, ' ') AS parts,
  ARRAY_LENGTH(SPLIT(name, ' ')) AS n,
  SPLIT(name, ' ')[SAFE_OFFSET(1)] AS surname,
  SPLIT(name, ' ')[SAFE_OFFSET(5)] AS sixth
FROM shop.customers
WHERE customer_id IN (1, 4);
```

OUTPUT · 2 ROWS

name	parts	n	surname	sixth
Amara Okafor	["Amara", "Okafor"]	2	Okafor	NULL
David Osei	["David", "Osei"]	2	Osei	NULL

Reading the output. SPLIT returns ARRAY<STRING>. OFFSET is 0 based, so OFFSET(1) is the second element. SAFE_OFFSET(5) asks for an element that does not exist and returns NULL; plain OFFSET(5) would fail with Array index 5 is out of bounds.

OFFSET(n): 0 based index

ORDINAL(n): 1 based index

SAFE_ versions: NULL instead of error

ARRAY_TO_STRING: the reverse of SPLIT

PART 11 · REGULAR EXPRESSIONS

Validate, extract and mask with REGEXP functions

QUERY

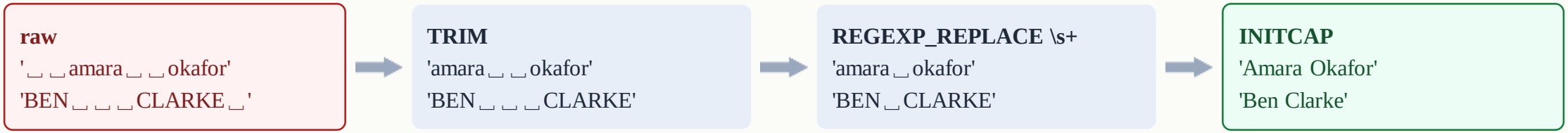
```
SELECT
  email,
  REGEXP_CONTAINS(email,
    r'^[\w.]+@[\w.]+\.\w+$') AS valid,
  REGEXP_EXTRACT(email, r'@(.+)$') AS domain,
  REGEXP_REPLACE(email, r'^[^@]+', '***') AS masked
FROM UNNEST([
  'amara@acme.co.uk',
  'ben.clarke@gmail.com',
  'chloe@exemple.fr',
  'not an email']) AS email;
```

OUTPUT · 4 ROWS

email	valid	domain	masked
amara@acme.co.uk	true	acme.co.uk	***@acme.co.uk
ben.clarke@gmail.com	true	gmail.com	***@gmail.com
chloe@exemple.fr	true	exemple.fr	***@exemple.fr
not an email	false	NULL	***

Reading the output. REGEXP_CONTAINS tests a pattern. REGEXP_EXTRACT returns the first capture group, or NULL when nothing matches. REGEXP_REPLACE replaced everything before @; in the invalid row there is no @, so the whole text was replaced. Use r'' raw strings so backslashes reach the regex engine untouched.

A text cleaning pipeline, step by step




```
SELECT
  raw,
  INITCAP(REGEXP_REPLACE(
    TRIM(raw), r'\s+', ' ')) AS clean
FROM UNNEST([' amara okafor',
  'BEN CLARKE ', 'chloe martin']) AS raw;
```

raw	clean
␣ ␣amara␣ ␣okafor	Amara Okafor
BEN␣ ␣ ␣CLARKE␣	Ben Clarke
chloe martin	Chloe Martin

Reading the output. The symbol ␣ makes invisible spaces visible here. Functions nest from the inside out: TRIM removes edge spaces, \s+ collapses inner runs of whitespace to one space, INITCAP fixes case. Wrap such logic in a SQL UDF (Part 20) so every pipeline cleans names identically.

Case insensitive matching and searching everywhere

```
SELECT
  'Paris' = 'PARIS' AS plain,
  LOWER('Paris') = LOWER('PARIS') AS lowered,
  COLLATE('Paris', 'und:ci') = 'PARIS' AS collated;
```

plain	lowered	collated
false	true	true

und:ci is the Unicode root collation, case insensitive. Set it as a column or dataset default to make every comparison and GROUP BY ignore case.

```
SELECT customer_id, name
FROM shop.customers AS c
WHERE CONTAINS_SUBSTR(c, 'manchester');
```

customer_id	name
1	Amara Okafor
6	Farid Haddad

CONTAINS_SUBSTR on a whole row searches every column, case insensitively. Ideal for exploration; combine with a search index (Part 15) on large tables.

Reading the outputs. Plain string equality is exact and case sensitive. LOWER on both sides works but must be repeated everywhere. CONTAINS_SUBSTR found manchester inside the city column of two customers, even though the stored value is capitalised.

12

Dates, Times and Time Zones

Almost every analytical question has a time dimension. This part covers constructing temporal values, extracting parts, truncating to periods, date arithmetic, formatting and parsing, time zones and filling gaps in time series.

Current values and constructors · EXTRACT · DATE_TRUNC · DATE_ADD and DATE_DIFF · FORMAT and PARSE · Time zones · GENERATE_DATE_ARRAY spines · Pitfalls

One instant, four representations

TIMESTAMP the order was placed at 13:30:00 UTC on Thursday 22 October 2026

An absolute point on the global timeline. Same value whichever city you stand in.

in Europe/London

DATETIME

22 Oct 2026, 14:30:00

The wall clock reading in London (BST, one hour ahead of UTC)

DATE

22 Oct 2026

The calendar day only. Perfect for daily reporting and partitioning.

TIME

14:30:00

The clock time only. Useful for opening hours and shift patterns.

The conversion rule. Moving from TIMESTAMP to DATETIME, DATE or TIME always requires a time zone, explicit or defaulted to UTC. Moving back requires one too. Most date bugs come from forgetting this.

PART 12 · DATES AND TIMES

Current values and constructors

QUERY · RUN ON 22 OCTOBER 2026 AT 14:30 LONDON TIME

```
SELECT
  FORMAT_DATE('%A %d %B %Y',
    CURRENT_DATE('Europe/London')) AS today,
  FORMAT_TIMESTAMP('%H:%M',
    CURRENT_TIMESTAMP(), 'Europe/London') AS now_london,
  FORMAT_TIMESTAMP('%H:%M',
    CURRENT_TIMESTAMP()) AS now_utc,
  FORMAT_DATE('%d %b %Y',
    DATE(2026, 12, 25)) AS built;
```

OUTPUT

column	value
today	Thursday 22 October 2026
now_london	14:30
now_utc	13:30
built	25 Dec 2026

Reading the output. In October London is on British Summer Time, one hour ahead of UTC, hence 14:30 versus 13:30. CURRENT_DATE without a zone uses UTC, which near midnight can be a different day from London. Constructors DATE(y, m, d), DATETIME(…) and TIME(h, m, s) build values from parts.

EXTRACT: pulling parts out of a date

QUERY

```
SELECT
  order_id,
  EXTRACT(MONTH FROM order_date) AS m,
  EXTRACT(QUARTER FROM order_date) AS q,
  EXTRACT(DAYOFWEEK FROM order_date) AS dow,
  FORMAT_DATE('%A', order_date) AS day_name,
  EXTRACT(ISOWEEK FROM order_date) AS iso_wk
FROM shop.orders
WHERE order_id IN (5001, 5003, 5008);
```

OUTPUT · 3 ROWS

order	m	q	dow	day_name	iso_wk
5001	1	1	2	Monday	2
5003	2	1	7	Saturday	7
5008	4	2	6	Friday	14

Reading the output. DAYOFWEEK counts Sunday as 1, so Monday is 2 and Saturday is 7. 5 January 2026 falls in ISO week 2 because ISO week 1 is the week containing the year's first Thursday (1 January 2026). Other parts: YEAR, DAY, DAYOFYEAR, WEEK, HOUR, MINUTE, SECOND, MILLISECOND.

PART 12 · DATES AND TIMES

DATE_TRUNC: rolling dates up to periods

QUERY

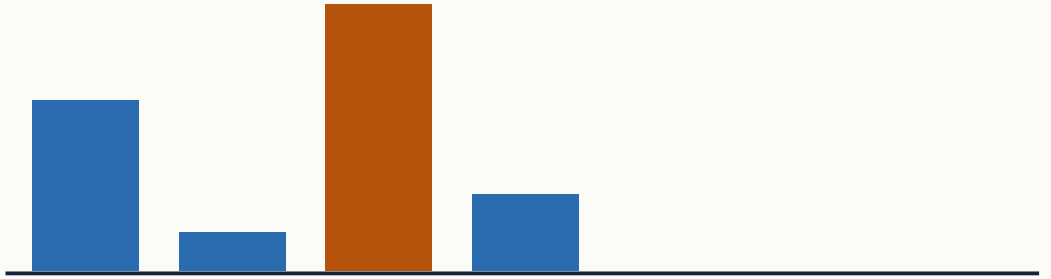
```
SELECT
  DATE_TRUNC(o.order_date, MONTH) AS month_start,
  COUNT(DISTINCT o.order_id) AS orders,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.orders AS o
JOIN shop.order_items AS i USING (order_id)
WHERE o.status != 'cancelled'
GROUP BY month_start
ORDER BY month_start;
```

DATE_TRUNC(d, WEEK) starts on Sunday; WEEK(MONDAY) or ISOWEEK on Monday.

Granularities: DAY, WEEK, MONTH, QUARTER, YEAR, ISOYEAR.

OUTPUT · 4 ROWS

month_start	orders	revenue
1 Jan 2026	2	1324
1 Feb 2026	1	300
1 Mar 2026	3	2072
1 Apr 2026	1	600



March is the peak at 2072, driven by the 1200 laptop order.

Reading the output. Every date is replaced by the first day of its month, and grouping on that gives one row per month that sorts correctly as a real DATE. Dates are shown in long form here; the console prints them as year, month and day digits.

PART 12 · DATES AND TIMES

Date arithmetic: DATE_ADD and DATE_DIFF

QUERY

```
SELECT
  c.name,
  DATE_DIFF(MIN(o.order_date), c.signup_date, DAY)
    AS days_to_first_order,
  FORMAT_DATE('%d %b %Y',
    DATE_ADD(c.signup_date, INTERVAL 1 YEAR))
    AS anniversary
FROM shop.customers AS c
JOIN shop.orders AS o USING (customer_id)
GROUP BY c.name, c.signup_date
ORDER BY days_to_first_order;
```

OUTPUT · 5 ROWS

name	days_to_first	anniversary
Elena Rossi	124	11 Nov 2026
David Osei	153	30 Sep 2026
Chloe Martin	214	21 Jul 2026
Ben Clarke	221	02 Jun 2026
Amara Okafor	297	14 Mar 2026

Reading the output. DATE_DIFF(later, earlier, DAY) counts whole days between them. Elena converted fastest: 124 days from signing up on 11 November to ordering on 15 March. DATE_ADD with INTERVAL 1 YEAR gives the signup anniversary. Farid is absent because the inner JOIN found no orders for him.

Format elements, shown for 5 January 2026 at 09:15 UTC

Element	Meaning	Output
%Y	four digit year	2026
%y	two digit year	26
%m	month number	01
%B / %b	month name	January / Jan
%d	day of month	05
%e	day, space padded	␣5

Element	Meaning	Output
%A / %a	weekday name	Monday / Mon
%j	day of year	005
%V	ISO week number	02
%Q	quarter	1
%H:%M	24 hour time	09:15
%Z	time zone name	UTC

```
SELECT PARSE_DATE('%d/%m/%Y', '05/01/2026') AS d;
# result: the DATE 5 January 2026
```

PARSE uses the same elements in reverse. The format must match the text exactly; use SAFE.PARSE_DATE for unreliable input. UK style 05/01 is 5 January; US style would read it as 1 May.

Time zones: the late night order problem

QUERY

```
WITH e AS (
  SELECT TIMESTAMP(
    DATETIME(2026, 6, 30, 23, 30, 0), 'UTC') AS ts)
SELECT
  FORMAT_DATE('%d %b', DATE(ts)) AS utc_day,
  FORMAT_DATE('%d %b',
    DATE(ts, 'Europe/London')) AS london_day,
  FORMAT_TIMESTAMP('%d %b %H:%M', ts,
    'Asia/Tokyo') AS tokyo,
  FORMAT_TIMESTAMP('%d %b %H:%M', ts,
    'America/New_York') AS new_york
FROM e;
```

OUTPUT · 1 ROW

column	value
utc_day	30 Jun
london_day	01 Jul
tokyo	01 Jul 08:30
new_york	30 Jun 19:30

Reading the output. One order at 23:30 UTC on 30 June belongs to June in a UTC report but to July for a London business, because BST is UTC plus one hour. Tokyo is nine hours ahead; New York four hours behind in summer. Always state the business time zone in DATE(ts, zone) when building daily or monthly reports.

PART 12 · DATES AND TIMES

Date spines: showing the months with no sales

QUERY

```
WITH spine AS (
  SELECT month_start
  FROM UNNEST(GENERATE_DATE_ARRAY(
    DATE(2026, 1, 1), DATE(2026, 6, 1),
    INTERVAL 1 MONTH)) AS month_start),
monthly AS (… revenue per month_start …)
SELECT
  FORMAT_DATE('%b %Y', s.month_start) AS month,
  IFNULL(m.revenue, 0) AS revenue
FROM spine AS s
LEFT JOIN monthly AS m USING (month_start)
ORDER BY s.month_start;
```

OUTPUT · 6 ROWS

month	revenue
Jan 2026	1324
Feb 2026	300
Mar 2026	2072
Apr 2026	600
May 2026	0
Jun 2026	0

Reading the output. GENERATE_DATE_ARRAY builds every month from January to June; UNNEST turns the array into rows. The LEFT JOIN keeps all six, and IFNULL turns missing months into honest zeros. Charts built from this never skip a month or draw a misleading line across a gap.

PART 12 · RECAP

Six date pitfalls and how BigQuery behaves

Expression	Result	Lesson
DATE_ADD(DATE(2026, 1, 31), INTERVAL 1 MONTH)	28 Feb 2026	Month ends are clamped
LAST_DAY(DATE(2026, 2, 10))	28 Feb 2026	Month end without arithmetic
DATE_DIFF(DATE(2026, 1, 1), DATE(2025, 12, 31), YEAR)	1	Counts boundaries crossed, not full years
DATE_TRUNC(DATE(2026, 1, 7), WEEK)	4 Jan 2026	Weeks start on Sunday by default
DATE(ts) near midnight	UTC day	Pass the business time zone
BETWEEN on TIMESTAMP with a day end	misses 23:59:59.5	Use ts >= start AND ts < next_day

Reading the table. 7 January 2026 is a Wednesday; the preceding Sunday is 4 January. The half open interval pattern, greater than or equal to the start and strictly less than the next period, is the safest way to filter any time range and also lets BigQuery prune partitions.

13

Nested and Repeated Data

BigQuery’s signature modelling technique. Instead of splitting parents and children into separate tables, store the children inside the parent row as an ARRAY of STRUCTs, and avoid expensive joins entirely.

Why nest · STRUCT · ARRAY basics · UNNEST visualised · Building nested rows with ARRAY_AGG · Schema of ARRAY of STRUCT · Querying children · Aggregating inside a row · EXISTS over arrays · Array functions · Best practice

PART 13 · NESTED DATA

Normalised versus nested: the same order, two shapes

Normalised: two tables and a join

orders: 5001 · customer 1 · delivered



order_items
5001 · 101 · qty 1 · 1200
5001 · 102 · qty 2 · 25

Every query needing both pays for a shuffle join, and the order ID is stored on every line.

Nested: one row carries its lines

orders_nested: 5001 · customer 1 · delivered

items: ARRAY of STRUCT

{ 101, qty 1, 1200 }

{ 102, qty 2, 25 }

Children are physically stored with their parent. Reading an order and its lines needs no join and no shuffle.

When to nest. Children that are always read with their parent, bounded in number and owned by one parent: order lines, event parameters, address history. Keep shared entities such as products and customers as separate dimension tables.

PART 13 · NESTED DATA

STRUCT: creating and reading records

QUERY 1: BUILD

```
SELECT
  name,
  STRUCT(city, country, tier) AS profile
FROM shop.customers
WHERE customer_id = 3;
```

OUTPUT 1 · JSON VIEW


```
{ "name": "Chloe Martin",
  "profile": {
    "city": "Paris",
    "country": "France",
    "tier": "Gold" } }
```


QUERY 2: READ WITH DOT NOTATION

```
WITH p AS (… query 1 …)
SELECT
  name,
  profile.city,
  profile.tier
FROM p;
```

OUTPUT 2

name	city	tier
Chloe Martin	Paris	Gold

STRUCT field names default to the source column names. Because storage is columnar, reading profile.city reads only that leaf, not the whole struct.

PART 13 · NESTED DATA

ARRAY basics: literals, length, indexing, membership

QUERY

```
WITH t AS (SELECT [10, 20, 30] AS a)
SELECT
  a,
  ARRAY_LENGTH(a) AS n,
  a[OFFSET(0)] AS first,
  a[ORDINAL(3)] AS third,
  20 IN UNNEST(a) AS has_20,
  a[SAFE_OFFSET(9)] AS missing
FROM t;
```

OUTPUT · 1 ROW

column	value
a	[10, 20, 30]
n	3
first	10
third	30
has_20	true
missing	NULL



Under each element: OFFSET · ORDINAL

Reading the output. Square brackets make an array literal. OFFSET counts from 0 and ORDINAL from 1, so both return the same element in different notations. value IN UNNEST(array) is the membership test. Arrays preserve order and may contain duplicates, but not NULL elements when written to a table.

PART 13 · NESTED DATA

UNNEST visualised: one row with an array becomes many rows



Parent columns repeat on every child row, exactly as a join would produce.

Empty arrays vanish with a comma or CROSS JOIN. Use LEFT JOIN UNNEST to keep the parent.

WITH OFFSET adds each element’s position as an extra column.

PART 13 · NESTED DATA

Building nested rows with ARRAY_AGG and STRUCT

DDL

```
CREATE OR REPLACE TABLE shop.orders_nested AS
SELECT
  o.order_id, o.customer_id,
  o.order_date, o.status,
  ARRAY_AGG(
    STRUCT(i.product_id, i.quantity, i.unit_price)
    ORDER BY i.product_id) AS items
FROM shop.orders AS o
JOIN shop.order_items AS i
  USING (order_id)
GROUP BY o.order_id, o.customer_id,
  o.order_date, o.status;
```

RESULT · ROW 5005 IN JSON VIEW


```
{ "order_id": 5005,
  "customer_id": 4,
  "status": "delivered",
  "items": [
    { "product_id": 103, "quantity": 2,
      "unit_price": "180" },
    { "product_id": 105, "quantity": 2,
      "unit_price": "18.5" } ] }
```


Reading the output. 11 item rows collapse into 8 order rows, each holding its lines. The JSON view prints NUMERIC as quoted strings to avoid precision loss. ARRAY_AGG's own ORDER BY fixes element order.

PART 13 · NESTED DATA

How the nested schema looks in the console

Field name	Type	Mode
order_id	INTEGER	NULLABLE
customer_id	INTEGER	NULLABLE
order_date	DATE	NULLABLE
status	STRING	NULLABLE
items	RECORD	REPEATED
items.product_id	INTEGER	NULLABLE
items.quantity	INTEGER	NULLABLE
items.unit_price	NUMERIC	NULLABLE

RECORD is the legacy display name for STRUCT.

REPEATED means ARRAY. Together: ARRAY<STRUCT<product_id INT64, quantity INT64, unit_price NUMERIC>>.

Leaf columns are stored separately, so SUM over items.quantity reads only quantities.

Limit: nesting can go 15 levels deep.

PART 13 · NESTED DATA

Flattening children with UNNEST

QUERY

```
SELECT
  o.order_id,
  pos,
  item.product_id,
  item.quantity * item.unit_price AS line_total
FROM shop.orders_nested AS o
CROSS JOIN UNNEST(o.items) AS item
  WITH OFFSET AS pos
WHERE o.order_id IN (5001, 5005)
ORDER BY o.order_id, pos;
```

OUTPUT · 4 ROWS

order_id	pos	product_id	line_total
5001	0	101	1200
5001	1	102	50
5005	0	103	360
5005	1	105	37

Reading the output. Each order expands into one row per item; item is the STRUCT, so its fields are reached with dots. pos is the 0 based position inside the array. The comma form, FROM t, UNNEST(t.items), means exactly the same as CROSS JOIN UNNEST.

PART 13 · NESTED DATA

Aggregating inside each row: no join, no shuffle

QUERY

```
SELECT
  order_id,
  ARRAY_LENGTH(items) AS n_items,
  (SELECT SUM(quantity) FROM UNNEST(items)) AS units,
  (SELECT SUM(quantity * unit_price)
    FROM UNNEST(items)) AS value
FROM shop.orders_nested
WHERE status != 'cancelled'
ORDER BY value DESC;
```

The subqueries over UNNEST(items) run inside each row independently: a correlated array subquery that never leaves the worker holding the row.

OUTPUT · 7 ROWS

order_id	n_items	units	value
5001	2	3	1250
5006	1	1	1200
5008	1	2	600
5007	2	2	475
5005	2	4	397
5003	1	1	300
5002	1	4	74

Reading the output. The same order values we computed earlier with joins, now from a single table. Values sum to 4296 again. On billions of orders, this pattern is dramatically cheaper than joining a separate items table.

PART 13 · NESTED DATA

Filtering parents by their children with EXISTS

QUERY: ORDERS THAT CONTAIN A LAPTOP

```
SELECT order_id, status
FROM shop.orders_nested
WHERE EXISTS (
  SELECT 1
  FROM UNNEST(items) AS it
  WHERE it.product_id = 101
)
ORDER BY order_id;
```

Common error: WHERE items.product_id = 101 fails with Cannot access field product_id on a value with type ARRAY. Fields live on elements, not on the array.

OUTPUT · 2 ROWS

order_id	status
5001	delivered
5006	shipped

Reading the output. The parent row is kept whole, once, if any element matches. Unlike UNNEST in FROM, this does not multiply rows, so no DISTINCT is needed. Order 5001 also contains a mouse, but it still appears only once.

Shorthand for scalar arrays: 'gift' IN UNNEST(tags).

PART 13 · NESTED DATA

Array functions and their results

Expression	Result
ARRAY_CONCAT([1, 2], [3])	[1, 2, 3]
ARRAY_TO_STRING(['web', 'app'], ' | ')	web | app
GENERATE_ARRAY(1, 9, 4)	[1, 5, 9]
ARRAY_REVERSE([1, 2, 3])	[3, 2, 1]
ARRAY(SELECT x * 2 FROM UNNEST([1, 2, 3]) AS x)	[2, 4, 6]
ARRAY(SELECT DISTINCT x FROM UNNEST([3, 1, 3]) AS x ORDER BY x)	[1, 3]
ARRAY_LENGTH([])	0
ARRAY_AGG(DISTINCT channel IGNORE NULLS)	an array of unique channels per group

Reading the table. The ARRAY(subquery) constructor is the universal tool: UNNEST an array, transform, filter or sort its elements with ordinary SQL, then rebuild it. GENERATE_ARRAY(start, end, step) creates numeric sequences, the integer cousin of GENERATE_DATE_ARRAY.

PART 13 · RECAP

Nested data: when it wins and when it hurts

Wins

- Parent and children always read together
- Joins between huge tables eliminated
- Event data with variable parameters, as in Google Analytics 4 exports
- Fewer bytes than repeating parent keys on every child row
- Natural fit for JSON and Avro sources

Hurts

- Children updated independently and often
- Children shared by many parents
- Unbounded arrays: a row has a 100 MB limit
- BI tools that struggle with nested columns
- Analysts unfamiliar with UNNEST

A practical compromise: keep normalised raw tables, publish nested reporting tables for heavy analytical use, and expose flattened views for BI tools.

14

JSON, Geography and Pipe Syntax

Three powerful extensions of GoogleSQL: the native JSON type for schema free documents, GEOGRAPHY for spatial analytics on the globe, and pipe query syntax for writing queries in the order you think.

JSON_VALUE and JSON_QUERY · Field access and conversion · ST_GEOGPOINT and ST_DISTANCE · Spatial predicates · Pipe syntax versus standard · Pipe operators · A pipe query traced step by step · When to use pipes

PART 14 · JSON

Reading JSON: scalars, objects and arrays

QUERY

```
WITH meta AS (
  SELECT 5001 AS order_id, JSON
    '{"gift": true, "note": "Happy birthday",
     "tags": ["red", "large"]}' AS j
  UNION ALL
  SELECT 5002, JSON '{"gift": false, "tags": []}')
SELECT
  order_id,
  JSON_VALUE(j, '$.note') AS note,
  BOOL(j.gift) AS gift,
  JSON_QUERY(j, '$.tags') AS tags,
  ARRAY_LENGTH(JSON_QUERY_ARRAY(j, '$.tags')) AS n_tags
FROM meta;
```

OUTPUT · 2 ROWS

order	note	gift	tags	n
5001	Happy birthday	true	["red","large"]	2
5002	NULL	false	[]	0

Reading the output. JSON_VALUE returns a scalar as STRING, or NULL when the path is missing, as for 5002's note. JSON_QUERY returns JSON, keeping arrays and objects. Dot access j.gift returns JSON, and BOOL() converts it to a real BOOL. JSON_QUERY_ARRAY turns a JSON array into a SQL ARRAY.

PART 14 · JSON

The JSON toolkit

Function	Purpose and example result
PARSE_JSON(text)	String to JSON; fails on invalid text, use SAFE.PARSE_JSON
TO_JSON(struct)	SQL value to JSON: TO_JSON(STRUCT(1 AS id)) gives {"id":1}
TO_JSON_STRING(value)	Any value serialised as JSON text, for export and APIs
JSON_VALUE / JSON_QUERY	Extract a scalar as STRING, or a fragment as JSON
INT64(j), FLOAT64(j), STRING(j), BOOL(j)	Strict conversion from a JSON scalar
LAX_INT64(j) and friends	Lenient conversion: "42" as text still becomes 42
JSON_SET, JSON_REMOVE	Modify documents inside SQL
JSON_KEYS(j)	Array of the keys present, ideal for profiling messy data

Why a native JSON type? BigQuery shreds JSON into virtual columns on write, so reading j.gift scans only that path instead of parsing the whole document. Store semistructured payloads as JSON, then promote frequently used fields to typed columns for speed and clarity. Older JSON_EXTRACT functions still work on STRING columns.

PART 14 · GEOGRAPHY

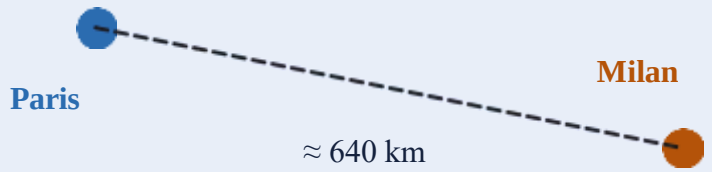
Distances on the globe: Paris to Milan

QUERY

```
WITH cities AS (
  SELECT
    ST_GEOGPOINT(2.3522, 48.8566) AS paris,
    ST_GEOGPOINT(9.1900, 45.4642) AS milan)
SELECT
  ST_ASTEXT(paris) AS paris_wkt,
  ROUND(ST_DISTANCE(paris, milan) / 1000, 1) AS km,
  ST_DWITHIN(paris, milan, 500000) AS within_500km
FROM cities;
```

OUTPUT · 1 ROW

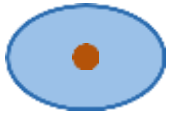
column	value
paris_wkt	POINT(2.3522 48.8566)
km	about 639.5
within_500km	false



Reading the output. ST_GEOGPOINT takes longitude first, then latitude. ST_DISTANCE returns metres along the Earth’s surface, so dividing by 1000 gives kilometres: roughly 640 km. ST_DWITHIN with a 500,000 metre radius is therefore false.

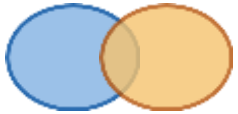
PART 14 · GEOGRAPHY

Spatial building blocks



ST_CONTAINS

Is this store inside that delivery zone polygon?



ST_INTERSECTS

Do two shapes share any area, such as a route crossing a flood zone?



ST_BUFFER, ST_DWITHIN

Everything within a radius, such as customers within 5 km of a shop.

Function	Use
ST_GEOGFROMTEXT, ST_GEOGFROMGEOJSON	Load shapes from WKT or GeoJSON
ST_AREA, ST_LENGTH, ST_PERIMETER	Square metres and metres on the spheroid
ST_CENTROID, ST_UNION_AGG	Centres and merged regions
ST_CLUSTERDBSCAN	Spatial clustering as a window function

Visualise results in BigQuery Geo Viz or Looker Studio maps. Spatial joins on ST_DWITHIN and ST_INTERSECTS are optimised with internal spatial indexing.

PART 14 · PIPE SYNTAX

Pipe syntax: write the query in the order it runs

Standard syntax

```
SELECT channel, COUNT(*) AS n
FROM shop.orders
WHERE status != 'cancelled'
GROUP BY channel
ORDER BY n DESC;
```

Clauses have a fixed order, and SELECT is written first though it runs late.

Pipe syntax

```
FROM shop.orders
|> WHERE status != 'cancelled'
|> AGGREGATE COUNT(*) AS n
   GROUP BY channel
|> ORDER BY n DESC;
```

Each |> takes the previous result and applies one operation, top to bottom.

channel	n
web	4
app	3

Reading the output. Both queries return the same two rows and are priced, optimised and executed identically. Pipe syntax has been generally available in BigQuery since February 2025 and can be mixed freely with standard syntax.

The pipe operators

Operator	Effect
|> WHERE	Filter rows, anywhere in the chain
|> SELECT	Choose and compute columns
|> EXTEND	Add computed columns, keep the rest
|> SET	Replace a column's value
|> DROP	Remove columns
|> RENAME	Rename columns
|> AS	Give the current result an alias

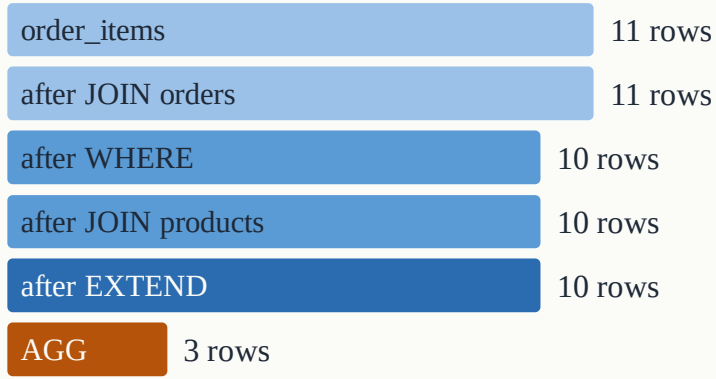
Operator	Effect
|> AGGREGATE	Aggregate, with optional GROUP BY
|> JOIN	Join the result to another table
|> ORDER BY	Sort
|> LIMIT	Cap rows, with optional OFFSET
|> DISTINCT	Remove duplicate rows
|> WITH	Define CTEs mid pipeline
|> PIVOT, UNPIVOT, CALL	Reshape, or call a table function

Key differences. There is no HAVING or QUALIFY: just add another WHERE after AGGREGATE or after a window EXTEND. GROUP BY lives inside AGGREGATE. A bare FROM table is itself a complete, valid query.

PART 14 · PIPE SYNTAX

A pipe query traced step by step

```
FROM shop.order_items AS i
|> JOIN shop.orders AS o USING (order_id)
|> WHERE o.status != 'cancelled'
|> JOIN shop.products AS p USING (product_id)
|> EXTEND i.quantity * i.unit_price AS revenue
|> AGGREGATE SUM(revenue) AS revenue,
           COUNT(*) AS lines
   GROUP BY p.category
|> ORDER BY revenue DESC;
```



category	revenue	lines
Electronics	3375	6
Furniture	810	2
Grocery	111	2

Reading the output. The bars show how many rows flow out of each step: the cancelled line disappears at WHERE and aggregation collapses ten lines into three categories. The answer matches the standard syntax version from Part 7 exactly. To debug, delete the trailing operators and run the prefix.

PART 14 · PIPE SYNTAX

Window functions and filters in a pipe

QUERY: CUSTOMERS ABOVE 20% OF REVENUE

```
FROM shop.orders AS o
|> WHERE o.status != 'cancelled'
|> JOIN shop.order_items AS i USING (order_id)
|> AGGREGATE SUM(i.quantity * i.unit_price) AS rev
   GROUP BY o.customer_id
|> EXTEND ROUND(100 * rev / SUM(rev) OVER (), 1) AS pct
|> WHERE pct > 20
|> ORDER BY pct DESC;
```

OUTPUT · 2 ROWS

customer_id	rev	pct
1	1550	36.1
5	1200	27.9

Reading the output. In standard SQL this needs either QUALIFY or a wrapping subquery. In a pipe, the window column is added by EXTEND and then filtered by an ordinary WHERE. Amara and Elena each exceed a fifth of total revenue.

Best uses: exploratory analysis, long transformation chains, log analytics, and teaching, because each line is one idea.

PART 14 · RECAP

Three extensions, three decisions

JSON

Use for payloads whose shape varies or evolves quickly. Extract hot fields into typed columns. Prefer the native JSON type over JSON in STRING columns.

GEOGRAPHY

Use whenever location matters: catchments, delivery zones, nearest store. Remember longitude first and distances in metres.

Pipe syntax

Use when a query is a long chain of steps. Same engine, same cost. Agree a team convention so code reviews stay consistent.

15

DDL and Table Management

Data Definition Language creates and shapes the objects that hold data. This part covers tables, views, materialized views, snapshots, clones, time travel, expiration and indexes, all managed with SQL.

CREATE TABLE · CREATE TABLE AS SELECT · Replace and if not exists · ALTER TABLE · Views · Materialized views · Snapshots and clones · Time travel and fail safe · Expiration · Dropping and recovering · Search and vector indexes

PART 15 · DDL

CREATE TABLE with an explicit schema

DDL

```
CREATE TABLE shop.returns (
  return_id INT64 NOT NULL,
  order_id INT64 NOT NULL,
  return_date DATE,
  reason STRING OPTIONS (
    description = 'Free text from the customer'),
  refund NUMERIC(10, 2) DEFAULT 0,
  PRIMARY KEY (return_id) NOT ENFORCED
)
OPTIONS (
  description = 'Product returns',
  labels = [('domain', 'sales')]
);
```

OUTPUT

This statement created a new table named acmeretail.shop.returns.

NOT NULL makes the column REQUIRED.

DEFAULT fills the value when an INSERT omits it.

PRIMARY KEY NOT ENFORCED documents uniqueness and helps the optimiser with joins, but BigQuery does not check it: duplicates are your responsibility.

Descriptions appear in the console and in Gemini’s understanding of your data.

CREATE TABLE AS SELECT: materialising a result

DDL

```
CREATE OR REPLACE TABLE shop.customer_value
OPTIONS (expiration_timestamp =
  TIMESTAMP_ADD(CURRENT_TIMESTAMP(), INTERVAL 30 DAY))
AS
SELECT
  c.customer_id, c.name, c.tier,
  SUM(i.quantity * i.unit_price) AS revenue
FROM shop.customers AS c
JOIN shop.orders AS o USING (customer_id)
JOIN shop.order_items AS i USING (order_id)
WHERE o.status != 'cancelled'
GROUP BY ALL;
```

OUTPUT

This statement created a new table named acmeretail.shop.customer_value.

SELECT * FROM shop.customer_value

id	name	tier	revenue
1	Amara Okafor	Gold	1550
2	Ben Clarke	Silver	674
3	Chloe Martin	Gold	475
4	David Osei	Bronze	397
5	Elena Rossi	Silver	1200

The schema is inferred from the query. The table deletes itself after 30 days, ideal for intermediate results.

PART 15 · DDL

Three ways to create, three behaviours

Statement	If the table is absent	If the table exists
CREATE TABLE	Creates it	Error: Already Exists
CREATE TABLE IF NOT EXISTS	Creates it	Does nothing, keeps old data
CREATE OR REPLACE TABLE	Creates it	Replaces schema and data atomically

Idempotent pipelines

CREATE OR REPLACE makes a rebuild safe to rerun: running it twice leaves the same result as running it once.

Watch out

Replacing a table discards its old data and can reset properties set outside the statement, such as some access policies. Use DML when you mean to change rows, not rebuild.

Temporary tables

CREATE TEMP TABLE inside a script or session lives only for that script or session.

Copying structure

CREATE TABLE new LIKE old copies the schema without data; COPY copies both.

ALTER TABLE: evolving a schema safely

```
# 1 add a column (metadata only, instant)
ALTER TABLE shop.customers
  ADD COLUMN IF NOT EXISTS email STRING;
# 2 rename a column
ALTER TABLE shop.returns
  RENAME COLUMN reason TO return_reason;
# 3 widen a type
ALTER TABLE shop.returns
  ALTER COLUMN refund SET DATA TYPE NUMERIC(12, 2);
# 4 change table options
ALTER TABLE shop.returns
  SET OPTIONS (description = 'Returns, all channels');
```

OUTPUT FOR EACH

This statement modified the table named acmeretail.shop.customers.

Change	Allowed in place?
Add NULLABLE column	Yes, instant
Rename column	Yes
Widen NUMERIC or STRING(n)	Yes
Drop column	Yes, storage freed later
INT64 to STRING	No: rebuild with CTAS

Existing rows get NULL in a new column, or its DEFAULT if one is defined. Renames break downstream SQL, so add a view with the old name during migration.

PART 15 · DDL

Views: saved queries that behave like tables

DDL AND USE

```
CREATE OR REPLACE VIEW shop.v_order_value AS
SELECT
  o.order_id, o.customer_id, o.order_date,
  SUM(i.quantity * i.unit_price) AS value
FROM shop.orders AS o
JOIN shop.order_items AS i USING (order_id)
WHERE o.status != 'cancelled'
GROUP BY ALL;

SELECT order_id, value
FROM shop.v_order_value
ORDER BY value DESC LIMIT 3;
```

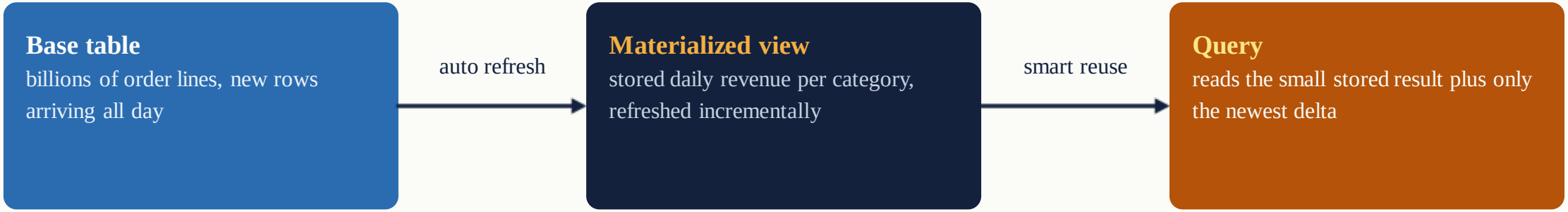
OUTPUT · 3 ROWS

order_id	value
5001	1250
5006	1200
5008	600

Reading the output. The view stored no data. Querying it ran its SQL against orders and order_items at that moment, then sorted. Bytes billed are those of the underlying tables, so a view is a convenience and a security boundary, not a performance optimisation.

Views centralise business logic such as “exclude cancelled orders”, so every analyst uses one definition.

Materialized views: precomputed and self maintaining




```
CREATE MATERIALIZED VIEW shop.mv_daily_cat
OPTIONS (enable_refresh = TRUE,
  refresh_interval_minutes = 30)
AS SELECT order_date, product_id,
  SUM(quantity * unit_price) AS revenue
FROM shop.order_lines_wide
GROUP BY order_date, product_id;
```

Smart tuning: queries on the base table can be rewritten automatically to use the view.

Always fresh: results combine stored data with changes since the last refresh.

Limits: SQL is restricted; aggregation over one table is the classic case.

PART 15 · DDL

Snapshots and clones: copies that cost almost nothing

Table snapshot

```
CREATE SNAPSHOT TABLE
  shop.orders_snap_q1
CLONE shop.orders
OPTIONS (expiration_timestamp =
  TIMESTAMP(DATE(2027, 4, 1)));
```

- Read only, frozen at a point in time
- Ideal for audits and month end records
- Restore later with CREATE TABLE … CLONE snapshot

Table clone

```
CREATE TABLE sandbox.orders_test
CLONE shop.orders;
```

- Writable, independent from its base
- Perfect for testing destructive DML
- Experiments never touch production



Only bytes that later differ from the base are billed to the copy.

PART 15 · DDL

Time travel: querying the past to undo mistakes

← going back in time

Time travel window: 2 to 7 days, default 7, queryable with SQL

Fail safe: 7 more days, emergency recovery

now

SCENARIO: 3 ORDERS DELETED BY MISTAKE AN HOUR AGO

```
SELECT
  (SELECT COUNT(*) FROM shop.orders) AS now_rows,
  (SELECT COUNT(*) FROM shop.orders
    FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(
      CURRENT_TIMESTAMP(), INTERVAL 2 HOUR)) AS before_rows;
```

OUTPUT

now_rows	before_rows
5	8

Reading the output. Two hours ago the table still had all 8 orders. Recover with INSERT INTO shop.orders SELECT the missing rows FROM the table FOR SYSTEM_TIME AS OF that moment, or CREATE TABLE … CLONE … FOR SYSTEM_TIME AS OF to restore a full copy.

PART 15 · DDL

Expiration: let data tidy itself up

Dataset default

```
default_table_expiration_days = 7
```

Every new table in a scratch dataset deletes itself after a week.

Table expiration

```
expiration_timestamp = …
```

One table disappears at a chosen instant, such as an extract shared for a project.

Partition expiration

```
partition_expiration_days = 400
```

Old partitions drop automatically while the table lives on: a rolling retention window.

Governance angle. Expiration is how you implement retention policies required by UK GDPR and similar laws: personal data is kept only as long as needed, enforced by the platform rather than by memory.

PART 15 · DDL

Dropping objects, and getting them back

```
DROP TABLE IF EXISTS shop.returns;
DROP VIEW IF EXISTS shop.v_order_value;
DROP SCHEMA scratch CASCADE;

# restore a deleted dataset
UNDROP SCHEMA scratch;
```

CASCADE drops a dataset with everything inside; without it, a non empty dataset cannot be dropped.

UNDROP SCHEMA recovers a deleted dataset within its time travel window.

A deleted table can be restored within the window by copying it at a timestamp before deletion.

Production safeguard. Grant DROP rights sparingly, take scheduled snapshots of critical tables, and require code review for any DDL that runs against production datasets.

Search indexes and vector indexes

Search index: needle in a haystack

```
CREATE SEARCH INDEX idx_logs
ON ops.app_logs(ALL COLUMNS);

SELECT *
FROM ops.app_logs AS l
WHERE SEARCH(l, 'order5004');
```

Tokenises text so a lookup for one ID touches a tiny fraction of the table instead of scanning every byte. Ideal for logs and security analytics.

Vector index: nearest meaning

```
CREATE VECTOR INDEX idx_reviews
ON shop.review_embeddings(embedding)
OPTIONS (index_type = 'IVF',
  distance_type = 'COSINE');
```

Speeds up VECTOR_SEARCH over embeddings for semantic search and retrieval augmented generation. Covered with ML in Part 22.

How these differ from database indexes. BigQuery maintains them in the background, they never block writes, and the optimiser decides automatically when using one is cheaper than a scan. Partitioning and clustering remain the first tools; indexes target specific lookup patterns.

16

DML, MERGE and Transactions

Data Manipulation Language changes rows: inserting, updating, deleting and upserting. BigQuery supports full DML and multi statement transactions, with behaviour tuned for analytical batches rather than single row traffic.

The examples here change a working copy of the sample data; every later part uses the original eight orders again.

INSERT · UPDATE · DELETE and TRUNCATE · MERGE upserts · MERGE clauses · Transactions · Concurrency and quotas · Change data capture

PART 16 · DML

INSERT: adding rows by value or by query

```
# by value
INSERT INTO shop.returns
  (return_id, order_id, return_date, return_reason, refund)
VALUES
  (1, 5007, DATE(2026, 4, 2), 'Wrong colour', 25),
  (2, 5005, DATE(2026, 3, 10), 'Damaged', 180);
```

```
# by query
INSERT INTO shop.order_audit (order_id, note)
SELECT order_id, 'cancelled order'
FROM shop.orders WHERE status = 'cancelled';
```

OUTPUTS

This statement added 2 rows to returns.

This statement added 1 row to order_audit.

return_id	order_id	return_reason	refund
1	5007	Wrong colour	25
2	5005	Damaged	180

Only the cancelled order 5004 matched, so the audit gained one row. Batch many rows into one INSERT: thousands of tiny inserts are slow and hit quotas.

PART 16 · DML

UPDATE: changing values in matching rows

DML

```
# mark old shipments as delivered
UPDATE shop.orders
SET status = 'delivered'
WHERE status = 'shipped'
  AND order_date < DATE(2026, 4, 1);
```

OUTPUT

This statement modified 1 row in orders.

BEFORE → AFTER

5006 · 15 Mar · shipped	5006 · delivered
5008 · 3 Apr · shipped	unchanged

Reading the output. Two orders were shipped, but only 5006 was placed before April, so one row changed. BigQuery requires a WHERE clause on UPDATE; write WHERE TRUE deliberately if you really mean every row.

UPDATE … FROM another table lets you set values from a join.

PART 16 · DML

DELETE versus TRUNCATE, practised on a clone

DELETE

```
DELETE FROM sandbox.orders_test
WHERE status = 'cancelled';
```

This statement removed 1 row from orders_test.

Removes matching rows. Scans to find them, so it is billed like a query. A WHERE clause is mandatory.

TRUNCATE

```
TRUNCATE TABLE sandbox.orders_test;
```

This statement removed all rows from orders_test.

Empties the whole table but keeps its schema, partitioning and options. A metadata operation: fast and free of scan charges.

Partition shortcut. Deleting a whole day from a partitioned table with a filter on the partition column only, such as WHERE order_date = DATE(2026, 1, 5), is executed as a cheap metadata partition drop rather than a row by row rewrite.

PART 16 · DML

MERGE: the upsert in one atomic statement




```
MERGE shop.orders AS t
USING shop.orders_staging AS s
ON t.order_id = s.order_id
WHEN MATCHED AND t.status != s.status THEN
  UPDATE SET status = s.status
WHEN NOT MATCHED THEN
  INSERT ROW;
```

This statement modified 2 rows in orders.

Reading the output. One row matched and changed; one did not match and was inserted: 2 rows modified. INSERT ROW copies every source column by position. The extra condition on MATCHED skips rows whose status is already identical.

PART 16 · DML

The three MERGE clauses

WHEN MATCHED

Key exists in both. Allowed actions: UPDATE or DELETE.



target ∩ source

WHEN NOT MATCHED [BY TARGET]

Key only in source. Allowed action: INSERT.



source only

WHEN NOT MATCHED BY SOURCE

Key only in target. Allowed actions: UPDATE or DELETE.



target only

```
# full synchronisation: make target identical to source
WHEN NOT MATCHED BY SOURCE THEN DELETE
```

Rules. Clauses are checked in order and each row takes the first that applies. If two source rows match one target row, MERGE fails, so deduplicate the source first with the QUALIFY pattern from Part 10. Filter the target on its partition column inside ON to avoid scanning the entire table.

Multi statement transactions: all or nothing

```
BEGIN
  BEGIN TRANSACTION;

  INSERT INTO shop.returns
    (return_id, order_id, return_date, return_reason, refund)
  VALUES (3, 5008, DATE(2026, 4, 9), 'Faulty', 300);

  UPDATE shop.orders
  SET status = 'partially_returned'
  WHERE order_id = 5008;

  COMMIT TRANSACTION;
EXCEPTION WHEN ERROR THEN
  ROLLBACK TRANSACTION;
  RAISE USING message = @@error.message;
END;
```



Both succeed: COMMIT makes both visible at once

Either fails: ROLLBACK undoes both

Reading the script. A refund and the order status must never disagree. Inside the transaction, other users keep seeing the old snapshot until COMMIT. The exception handler rolls back and rethrows the original error so the failure is not silent.

Transactions use snapshot isolation and may span multiple tables and datasets in one project.

PART 16 · DML

DML at warehouse scale: concurrency and habits



Illustrative relative elapsed time. Each DML job has fixed overhead and rewrites storage blocks, so many small statements cost far more than one set based statement.

Concurrency: mutating DML on one table runs a couple at a time, with more queued; conflicting changes to the same partitions are retried or fail.

Batch it: collect changes in a staging table, then apply one MERGE per interval.

Prune it: filter on partition and cluster columns so DML rewrites only the affected blocks.

Prefer append: for event data, append and deduplicate in views rather than update in place.

PART 16 · RECAP

Change data capture and the DML summary

Streaming CDC

The Storage Write API can apply upserts and deletes directly: each streamed row carries a change type of UPSERT or DELETE, and BigQuery applies it to a table that declares a primary key. Source databases replicate into BigQuery with no MERGE jobs to schedule, typically via Datastream.

Statement	Remember
INSERT	Batch rows; INSERT SELECT for copies
UPDATE	WHERE required; FROM for joins
DELETE	Partition filters make it cheap
TRUNCATE	Free, keeps schema
MERGE	Deduplicate source first
Transactions	Snapshot isolation, rollback on error

Cost note. On demand, DML is billed on the bytes scanned to find and rewrite affected data, like any query. Combine DML with good partitioning, the subject of the next part, to keep it cheap.

17

Partitioning and Clustering

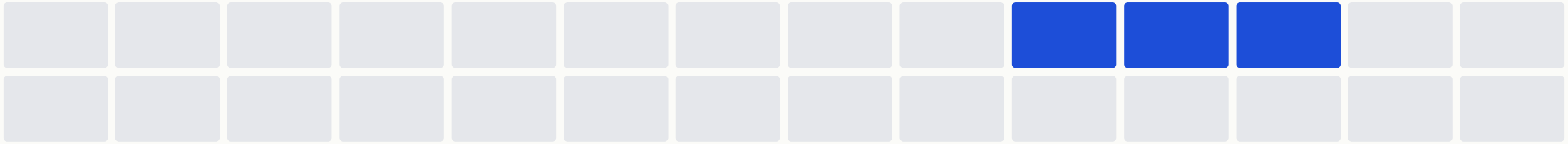
The two physical design levers that decide how much data a query must read. Used well, they cut cost and time by orders of magnitude; used badly, they do nothing at all.

Partitioning explained · Partition types · Creating partitioned tables · Pruning in numbers · Filters that prune · Clustering explained · Creating clustered tables · Combining both · Choosing · Inspecting partitions

PART 17 · PARTITIONING

Partitioning: a table split into independent segments

A table partitioned by day, shown as 28 daily segments. Query: **WHERE event_date BETWEEN 10th AND 12th**.



Blue: 3 partitions read **Grey:** 25 partitions pruned, never opened, never billed

Pruning

The planner reads partition metadata and skips segments that cannot match, before any slot touches data.

Exact estimates

Because pruning happens at planning time, the validator’s byte estimate already reflects it.

Management

Partitions can be expired, deleted or overwritten individually as cheap metadata operations.

Three kinds of partitioning

Time unit column

```
PARTITION BY order_date
```

On a DATE, TIMESTAMP or DATETIME column, by HOUR, DAY, MONTH or YEAR. The most common and most useful choice.

Ingestion time

```
PARTITION BY _PARTITIONDATE
```

By the time rows arrived, via pseudo columns _PARTITIONTIME and _PARTITIONDATE. For data without a reliable event date.

Integer range

```
RANGE_BUCKET(store_id,
GENERATE_ARRAY(0, 1000, 10))
```

Buckets of an INT64 column, such as store or customer ranges.

Rule	Detail
One partition column	A table is partitioned on a single column
Partition count limit	Up to 10,000 partitions per table, so choose granularity accordingly
Ideal size	Partitions of roughly 1 GB or more; tiny partitions add overhead
Special partitions	__NULL__ for NULL keys, __UNPARTITIONED__ for out of range values

Creating a partitioned and clustered table

DDL

```
CREATE OR REPLACE TABLE shop.orders_part
PARTITION BY order_date
CLUSTER BY customer_id, channel
OPTIONS (
  require_partition_filter = TRUE,
  partition_expiration_days = 730
)
AS
SELECT * FROM shop.orders;
```

OUTPUT

This statement created a new table named acmeretail.shop.orders_part.

PARTITION BY order_date: one partition per day. For TIMESTAMP columns use DATE(event_ts) or TIMESTAMP_TRUNC(event_ts, HOUR).

CLUSTER BY: within each day, rows are sorted by customer, then channel.

require_partition_filter: every query must filter on order_date, a guard against accidental full scans.

partition_expiration_days: days older than two years drop automatically.

Partitioning cannot be added to an existing table in place; rebuild with CTAS as shown.

Pruning in numbers: a worked example

Illustrative events table: **2 TiB** spread evenly over **3 years (1,095 daily partitions)**. Question: last 7 days of activity.



Measure	Unpartitioned	Partitioned
Bytes scanned	2,048 GiB	2,048 × 7 ÷ 1,095 ≈ 13.1 GiB
On demand cost at $6.25 per TiB	$12.50	about $0.08
Run 100 times a day for a month	about $37,500	about $240

Reading the numbers. Pruning reduced bytes by a factor of about 156, which is 1,095 ÷ 7. The free tier and minimum billing are ignored for clarity. This is why partitioning the event date is the single highest impact design decision for large tables.

Filters that prune, and filters that do not

WHERE clause on orders_part	Prunes?	Why
order_date = DATE(2026, 3, 2)	Yes	Constant on the column
order_date >= DATE(2026, 3, 1) AND order_date < DATE(2026, 4, 1)	Yes	Constant range
order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY)	Yes	Evaluated once before planning
order_date >= (SELECT MAX(d) FROM other)	Often not	Value unknown at planning; declare a variable first
FORMAT_DATE('%Y%m', order_date) = '202603'	No	Function hides the column
customer_id = 1 only	No	Not the partition column

With require_partition_filter, the last query fails: Cannot query over table without a filter over column(s) 'order_date' that can be used for partition elimination.

Rule. Compare the raw partition column with constants, parameters or scripting variables. Check the validator estimate: if it does not drop, pruning is not happening.

PART 17 · CLUSTERING

Clustering: sorted storage blocks inside each partition

Unclustered: customers scattered



customer_id = 5 appears in every block, so all 4 blocks must be read.

Clustered by customer_id



Each block records its minimum and maximum customer_id. Only the 2 blue blocks can contain 5; the other 2 are skipped.

Block pruning happens at run time, so the estimate is an upper bound and the bill is often lower.

Helps filters, GROUP BY and joins on clustered columns, and high cardinality keys.

Automatic reclustering keeps blocks sorted as new data arrives, at no charge.

Cluster column order matters



Filter on CLUSTER BY customer_id, channel	Benefit
customer_id = 4	Strong: first key, like a phone book surname
customer_id = 4 AND channel = 'web'	Strongest: both keys in order
channel = 'web' only	Weak: second key without the first
status = 'shipped'	None: not a clustering column

Guidance. Up to four columns. Put the most frequently filtered, highest cardinality column first. Unlike partitioning, clustering can be changed on an existing table with ALTER TABLE SET OPTIONS; new data follows the new order and reclustering gradually applies it.

PART 17 · COMBINED

Partition then cluster: two levels of skipping

Query: **WHERE order_date = 2 Mar 2026 AND customer_id = 4**



Level 1, planning time: partition pruning removes whole days. Reflected in the estimate.

Level 2, run time: block pruning inside the day keeps only blocks whose range includes customer 4.

Result. Of 12 blocks in the drawing, one is read. The standard production pattern is: partition on the event date, cluster on the two or three most common filter keys.

Partitioning or clustering? A comparison

Question	Partitioning	Clustering
Columns allowed	One: date, timestamp or integer	Up to four, most types
Cardinality suited	Low: hundreds to thousands of values	High: millions of values welcome
Cost known before running	Yes, exact	No, upper bound only
Lifecycle management	Expire, drop or overwrite per partition	Not applicable
Can enforce a filter	Yes, require_partition_filter	No
Change later	Rebuild the table	ALTER TABLE SET OPTIONS
Small tables under about 1 GB	Rarely worthwhile	Rarely worthwhile

Recommendation engine. BigQuery’s partition and cluster recommender analyses your past queries and suggests columns with estimated savings, visible in the console and through the Recommender API.

PART 17 · INSPECTING

Looking inside partitions with INFORMATION_SCHEMA

QUERY

```
SELECT
  partition_id,
  total_rows,
  total_logical_bytes
FROM shop.INFORMATION_SCHEMA.PARTITIONS
WHERE table_name = 'orders_part'
ORDER BY partition_id;
```

Reading the output. The table built from the original eight orders has eight daily partitions, one order each. partition_id is the date as eight digits. This is a toy: in real tables, aim for partitions of a gigabyte or more, and monthly partitioning would suit data this small.

OUTPUT · 8 ROWS

partition_id	total_rows	logical_bytes
20260105	1	40
20260109	1	40
20260214	1	40
20260220	1	40
20260302	1	40
20260315	1	38
20260328	1	40
20260403	1	38

18

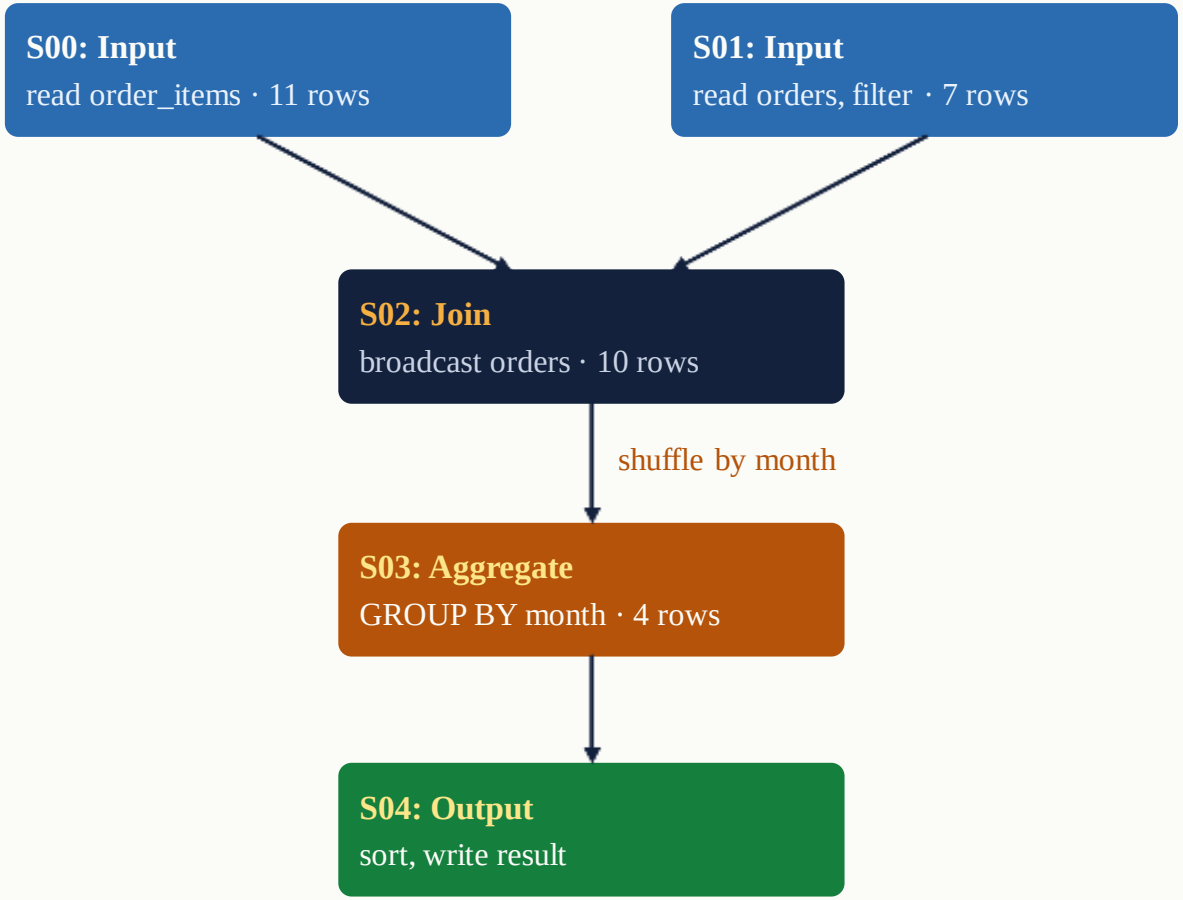
Performance and Query Plans

How to read what BigQuery actually did with your query, find the expensive stage, and apply the handful of techniques that deliver most real world speed ups.

Execution graph · Stage timings and skew · Column selection · Early filtering · Reducing before joining · Anti patterns · Caching, BI Engine and materialization · Specialised accelerators · Checklist

PART 18 · PERFORMANCE

Reading the execution graph



Stages are boxes; arrows are shuffles carrying rows between them.

Input stages read storage. Watch records read versus records written: a big drop means filters were pushed down effectively.

Join stages show broadcast or hash. A huge join output warns of fan out.

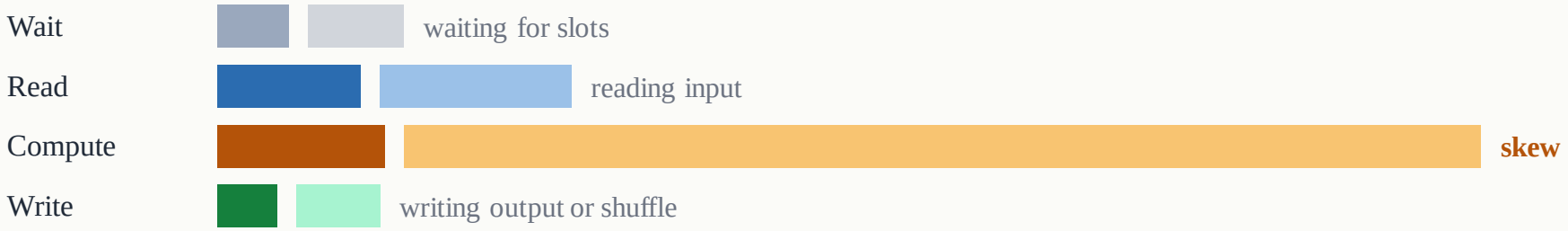
Hot stages are highlighted when they dominate slot time: start optimising there.

Plan for the monthly revenue query from Part 12. Stage numbers and row counts as BigQuery would report them for this tiny dataset.

PART 18 · PERFORMANCE

Stage timings: average versus maximum reveals skew

Each stage reports time spent in four phases, as the average worker and the slowest worker. Illustrative stage from a large join:



Dark segment: average worker. Light segment: additional time of the slowest worker.

High wait: not enough slots; contention with other jobs or a small reservation.

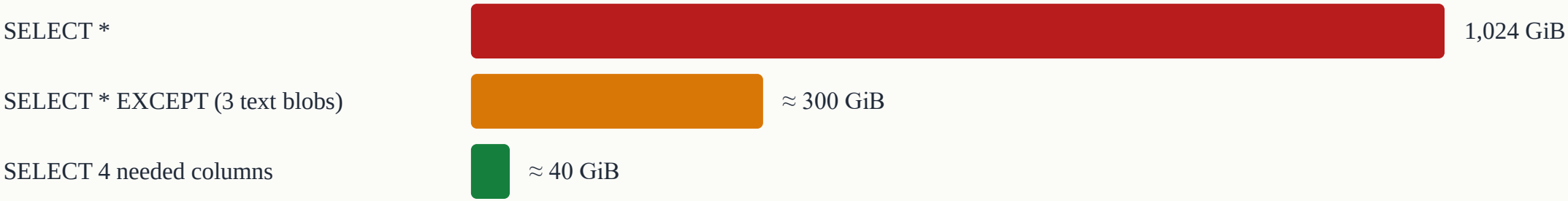
High read: scanning too much; add pruning or select fewer columns.

Max far above average compute: one key holds most rows, often NULL or a default value. Filter it out or handle it separately.

PART 18 · PERFORMANCE

Technique 1: read fewer columns

Illustrative 1 TiB table of 40 columns, where a few wide text columns hold most bytes.



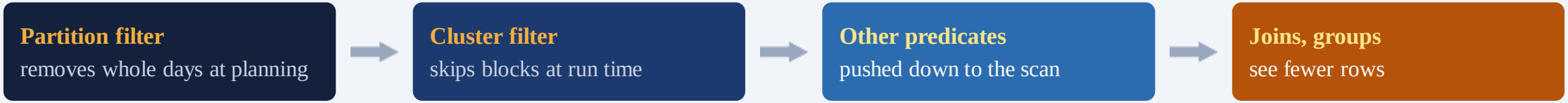

```
# costly habit
SELECT * FROM events LIMIT 100;
```

```
# targeted
SELECT event_date, user_id, event_name
FROM events WHERE event_date = @d;
```

Reading the chart. Columnar storage bills per column read, so the column list is a direct cost dial. Wide STRING and JSON columns dominate; leave them out unless needed. Remember LIMIT does not help, as shown in Part 6.

PART 18 · PERFORMANCE

Technique 2: filter early and filter on the right columns



Keep columns bare

Write order_date >= @start rather than wrapping the column in a function, so pruning still works.

Filter inside CTEs

The optimiser usually pushes filters down, but placing them in the first CTE makes intent clear and guarantees it.

Use variables for dynamic bounds

DECLARE d DATE DEFAULT (SELECT MAX…); then filter on d, so the partition bound is a constant.

Cheap predicates first

Integer and date comparisons before regular expressions or JSON parsing on large inputs.

Technique 3: aggregate before you join

Join first, then aggregate

```
SELECT c.country,
  COUNT(*) AS events
FROM events AS e
JOIN customers AS c USING (customer_id)
GROUP BY c.country;
```



10 billion rows shuffled

Aggregate first, then join

```
WITH per_cust AS (
  SELECT customer_id, COUNT(*) AS n
  FROM events GROUP BY customer_id)
SELECT c.country, SUM(p.n) AS events
FROM per_cust AS p
JOIN customers AS c USING (customer_id)
GROUP BY c.country;
```



5 million rows joined

Why it works. Illustrative sizes: 10 billion events from 5 million customers. Collapsing events to one row per customer first shrinks the join input two thousandfold, and the result is identical because SUM of per customer counts equals the total count. The optimiser sometimes does this itself; writing it explicitly makes it certain.

Anti patterns and their fixes

Anti pattern	Symptom	Fix
SELECT * on wide tables	Huge bytes billed	Name columns; use Preview
ORDER BY without LIMIT on big results	Final stage on one worker, resources exceeded	Sort only what you return
Self join to find the previous row	Exploding join	LAG window function
Unfiltered CROSS JOIN	Rows multiply	Add keys or pre aggregate
Heavy JavaScript UDFs	Slow compute stage	Rewrite as SQL UDF
Many tiny DML statements	Quota errors, slow pipelines	Batch into one MERGE
Date sharded tables events_2026xxxx	Metadata overhead, many reads	One partitioned table
Repeated CTE used three times	Same work done three times	Temp table once

Date sharded tables, one table per day, predate partitioning. Wildcard queries over them still work, but a single partitioned table is faster and simpler to govern.

PART 18 · PERFORMANCE

Accelerators: do the work once, reuse it often

Result cache
Identical deterministic queries within about 24 hours return instantly and free.

BI Engine
An in memory acceleration layer you reserve in GB; dashboards hitting hot tables answer in sub second time.

Materialized views
Precomputed aggregates, incrementally maintained and used automatically by the optimiser.

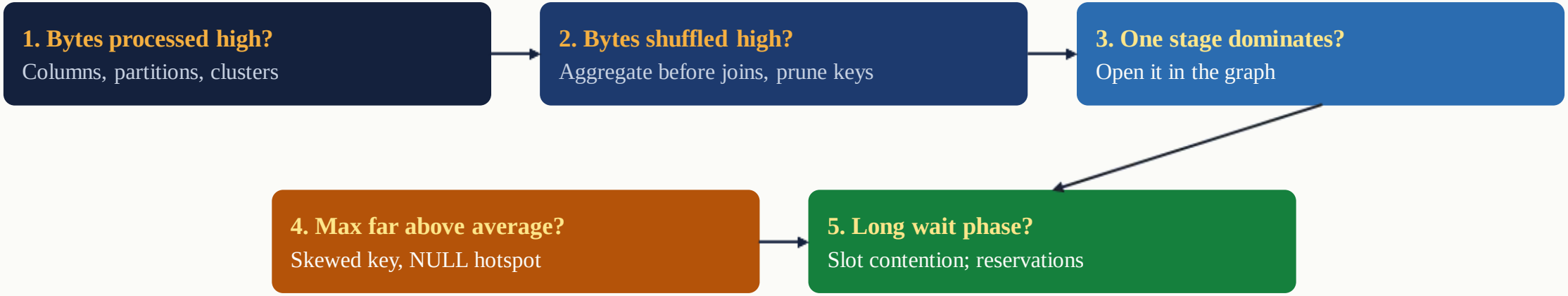
Summary tables
Scheduled CTAS or MERGE jobs that build daily rollups for reports.

History based optimisation
BigQuery learns from previous runs of similar queries to improve later plans automatically.

Short query optimisation
Small interactive queries can run without the full job overhead, lowering latency for applications.

PART 18 · PERFORMANCE

Diagnosing a slow query in five questions




```
# the job statistics behind those questions, for the last day
SELECT job_id, total_bytes_processed, total_slot_ms,
  TIMESTAMP_DIFF(end_time, start_time, SECOND) AS secs
FROM `region-eu`.INFORMATION_SCHEMA.JOBS
WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 DAY)
ORDER BY total_slot_ms DESC LIMIT 10;
```

PART 18 · RECAP

The performance checklist

- Only the columns you need
- Partition filter on every large table
- Cluster on frequent filter keys
- Aggregate before joining big tables
- Window functions over self joins
- Nest child rows that are read together
- Approximate functions where exactness is not needed
- Materialise expensive repeated logic
- Check the execution graph after changes
- Measure with cache disabled

19

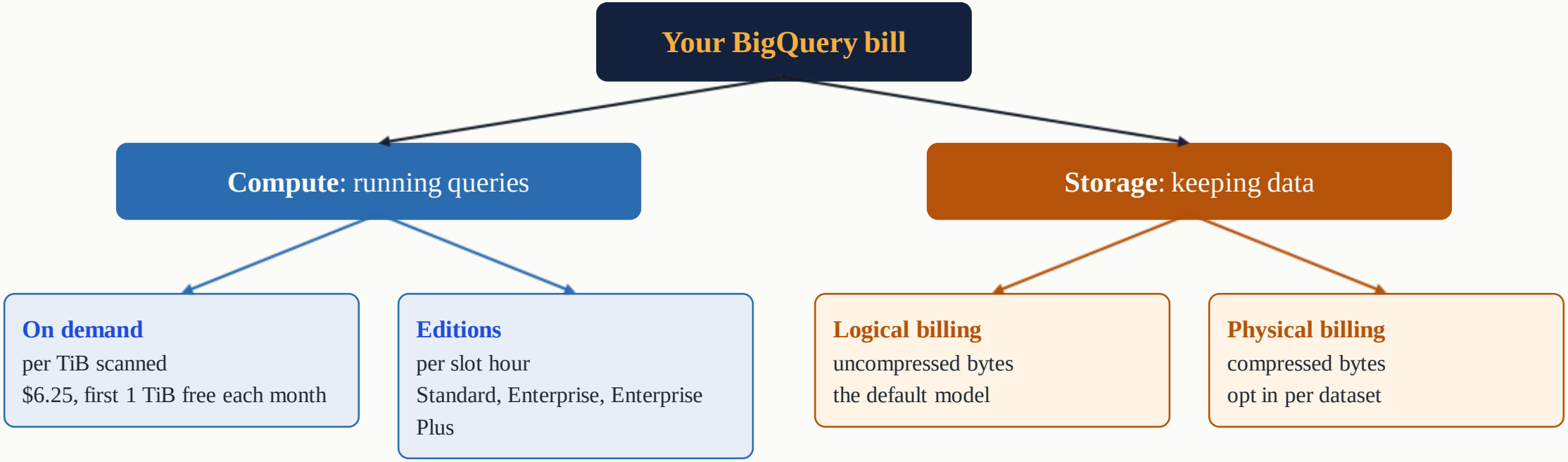
Pricing and Cost Control

BigQuery bills compute and storage separately. Understanding both models, when each wins, and the guard rails available is what separates a predictable bill from an unpleasant surprise.

The pricing map · On demand · Editions · Break even · Autoscaling · Storage pricing · Dry runs and byte limits · Cost attribution with INFORMATION_SCHEMA · Guard rails

PART 19 · PRICING

The BigQuery pricing map



Also billed: streaming ingestion, Storage Read and Write API usage, BI Engine capacity, and some ML and AI functions. **Free:** batch loading, copying, exporting on the shared pool, metadata queries, cached results and failed queries.

Prices are US list prices as published in 2026 and vary by region; always confirm on the official pricing page.

PART 19 · PRICING

On demand pricing, worked through

$6.25
per TiB of data scanned

1 TiB
free every month

10 MB
minimum per table referenced

Query	Bytes scanned	Cost after free tier
Any query on our tiny sample tables	bytes, rounded up to 10 MB per table	about $0.00006 per table
Three columns of a 2 TiB table	300 GiB = 0.293 TiB	0.293 × $6.25 ≈ $1.83
SELECT * on the same table	2 TiB	2 × $6.25 = $12.50
Cache hit or dry run	0	$0.00

How to read this. A TiB is 2 to the power 40 bytes, 1,024 GiB. You pay for bytes read, not rows returned and not time taken. On demand projects share a large slot pool, typically up to about 2,000 concurrent slots, with no guarantee: ideal for spiky, moderate workloads.

PART 19 · PRICING

BigQuery Editions: paying for slot capacity

Standard

$0.04

per slot hour, pay as you go

Entry level SQL analytics. Reservations capped at 1,600 slots; no commitments; fewer governance and ML features.

Enterprise

$0.06

per slot hour, pay as you go

The production default: BigQuery ML, BI Engine, customer managed keys, row and column security, one and three year commitments for discounts.

Enterprise Plus

$0.10

per slot hour, pay as you go

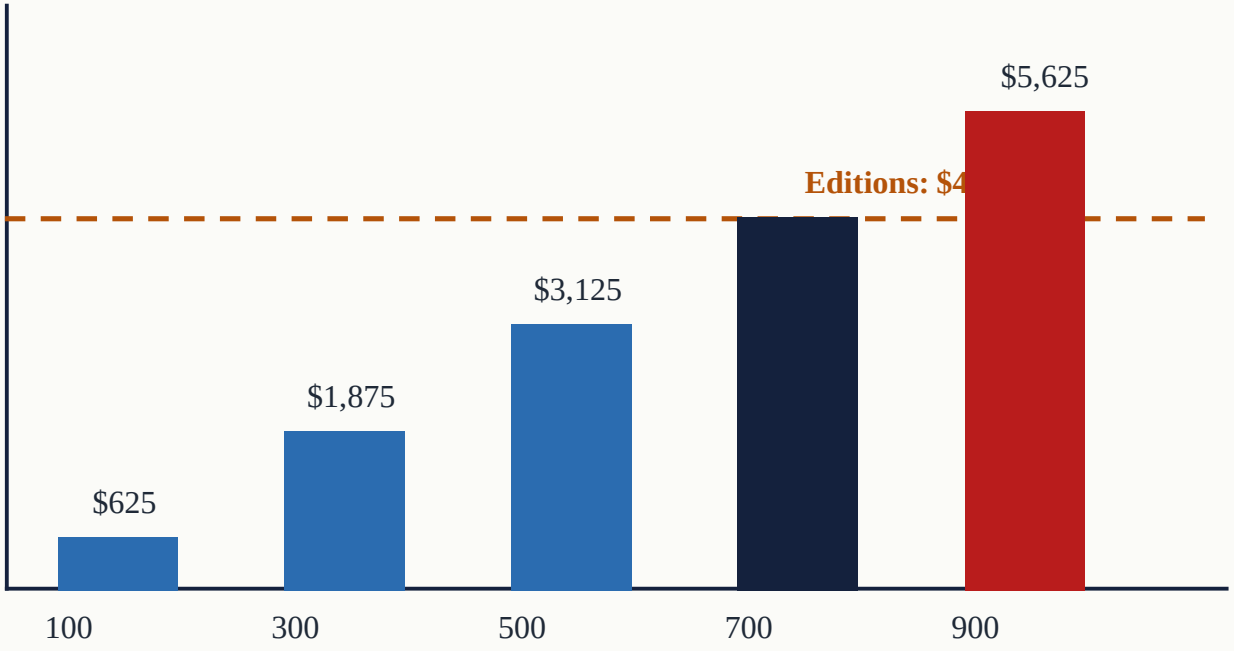
Mission critical: managed disaster recovery across regions and regulated compliance controls.

How it works. You create a reservation with a baseline and a maximum, then assign projects to it. Queries in those projects consume reservation slots instead of paying per TiB. Projects not assigned stay on demand, so both models can coexist in one organisation. Flat rate pricing was retired in 2023.

PART 19 · PRICING

Break even: on demand versus 100 Enterprise slots

Monthly cost in US dollars by TiB scanned per month



100 slots × 730 hours × $0.06
= **$4,380 per month**
$4,380 ÷ $6.25 per TiB
= **about 700 TiB**

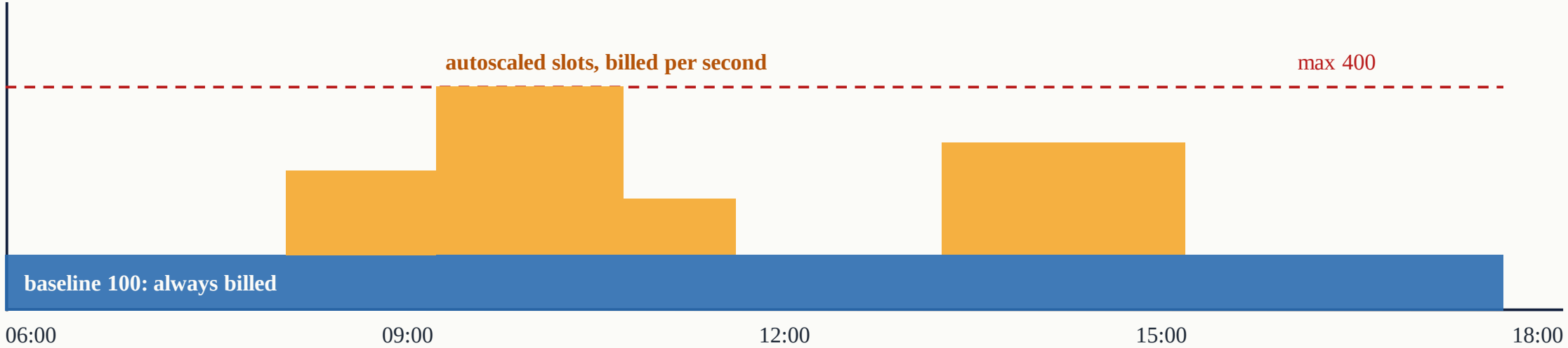
Reading the chart. Blue bars: on demand is cheaper. Around 700 TiB a month the two meet (navy). Beyond it, red, slots win. This assumes the 100 slots run all month; with autoscaling you pay only for slot time used, which moves the break even lower for bursty work.

Also weigh predictability: Editions cap spending by capacity, while on demand cost grows with every byte.

PART 19 · PRICING

Reservations, baselines and autoscaling

Slots across one working day (illustrative): baseline 100, maximum 400



Baseline slots are reserved and billed continuously, whether used or not.

Autoscale adds slots in steps when demand rises, up to the maximum, with a one minute minimum.

Idle slot sharing lets reservations borrow each other’s unused baseline.

PART 19 · PRICING

Storage pricing: active, long term, logical, physical

Per GiB per month	Active (changed in last 90 days)	Long term (untouched 90 days)
Logical (uncompressed)	$0.02	$0.01
Physical (compressed)	$0.04	$0.02

Worked example: 10 TiB logical

All active: 10,240 × $0.02 = **$204.80**

Half long term: 5,120 × $0.02 + 5,120 × $0.01 = **$153.60**

Same data, physical billing

If it compresses 4 to 1: 2,560 GiB × $0.04 = **$102.40** all active. Physical billing also counts time travel and fail safe bytes.

Key facts. The long term discount is automatic, per table or partition, with no performance difference. Any modification resets the 90 day clock for that table or partition, which is one more reason to partition. Physical billing pays off when compression ratios exceed about two to one and churn is low; compare using INFORMATION_SCHEMA.TABLE_STORAGE before switching.

PART 19 · COST CONTROL

Dry runs and maximum bytes billed

PYTHON

```python
# 1 estimate without running or paying
cfg = bigquery.QueryJobConfig(dry_run=True,
  use_query_cache=False)
job = client.query(sql, job_config=cfg)
print(job.total_bytes_processed)

# 2 run, but refuse if it would bill over 10 GB
cfg = bigquery.QueryJobConfig(
  maximum_bytes_billed=10_000_000_000)
client.query(sql, job_config=cfg).result()
```

OUTPUTS

```
1: 25368709120
```

```
2: Query exceeded limit for bytes billed: 10000000000. 25368709120 or higher required.
```

Reading the outputs. The dry run reports about 23.6 GiB without charge. With a 10 GB ceiling the real run is rejected before it starts, and a rejected query costs nothing. Set the same limit in the console under Query settings, Advanced options.

PART 19 · COST CONTROL

Who spent what? Cost attribution with JOBS

QUERY

```
SELECT
  user_email,
  COUNT(*) AS jobs,
  ROUND(SUM(total_bytes_billed) / POW(1024, 4), 2) AS tib,
  ROUND(SUM(total_bytes_billed) / POW(1024, 4)
    * 6.25, 2) AS est_usd
FROM `region-eu`.INFORMATION_SCHEMA.JOBS
WHERE job_type = 'QUERY'
  AND creation_time >= TIMESTAMP_TRUNC(CURRENT_TIMESTAMP(),
MONTH)
GROUP BY user_email
ORDER BY est_usd DESC;
```

OUTPUT · ILLUSTRATIVE

user_email	jobs	tib	est_usd
dashboards@acme	4210	18.4	115.00
etl@acme	930	9.6	60.00
amara@acme	212	1.28	8.00

Reading the output. A dashboard service account with thousands of small refreshes dominates spend: the classic case for caching, BI Engine or a materialized view. The region qualifier is required because job history is stored per region. The estimate ignores the free tier.

PART 19 · RECAP

Eight guard rails for a predictable bill

1
Custom quotas on query bytes per day, per project and per user

2
Cloud Billing budgets with email and Pub/Sub alerts

3
maximum_bytes_billed on scheduled and application queries

4
require_partition_filter on large tables

5
Labels on jobs and datasets for chargeback reports

6
Reservations with sensible maximums for heavy teams

7
Expiration on scratch datasets and old partitions

8
A monthly review of INFORMATION_SCHEMA.JOBS

Mindset. Cost is a design property, not an accident. The table design choices of Part 17 and the query habits of Part 18 do more for the bill than any discount negotiation.

20

Scripting, Procedures and Functions

GoogleSQL is also a procedural language: variables, branches, loops, dynamic SQL, stored procedures, reusable functions in SQL or JavaScript, table functions, remote functions and structured error handling.

DECLARE and SET · IF, WHILE, LOOP, FOR · EXECUTE IMMEDIATE · Stored procedures · SQL UDFs · JavaScript UDFs · Table functions · Remote functions · Exception handling

PART 20 · SCRIPTING

Variables with DECLARE and SET, and branching with IF

SCRIPT

```
DECLARE min_value NUMERIC DEFAULT 500;
DECLARE n_big INT64;

SET n_big = (
  SELECT COUNT(*)
  FROM shop.v_order_value
  WHERE value > min_value);

IF n_big >= 3 THEN
  SELECT n_big AS big_orders, 'Strong period' AS verdict;
ELSE
  SELECT n_big AS big_orders, 'Quiet period' AS verdict;
END IF;
```

OUTPUT

big_orders	verdict
3	Strong period

Reading the output. Three orders exceed 500: 1250, 1200 and 600. The script stored that count in a variable, then chose a branch. A script runs as one parent job with a child job per statement; the console shows each child's result under its own tab.

DECLARE statements must come first. Variables can hold any type, including ARRAY and STRUCT.

Loops: WHILE, LOOP and FOR … IN

WHILE

```
DECLARE i INT64 DEFAULT 0;
WHILE i < 3 DO
  SET i = i + 1;
END WHILE;
SELECT i;
```

Output: i = 3

LOOP with LEAVE

```
DECLARE x INT64 DEFAULT 1;
LOOP
  SET x = x * 2;
  IF x > 50 THEN LEAVE; END IF;
END LOOP;
SELECT x;
```

Output: x = 64

FOR over query rows

```
FOR rec IN (
  SELECT DISTINCT channel
  FROM shop.orders)
DO
  SELECT rec.channel,
    COUNT(*) AS n
  FROM shop.orders
  WHERE channel = rec.channel;
END FOR;
```

Two result sets: web 5, app 3

Reading the outputs. WHILE ran three times. LOOP doubled x through 2, 4, 8, 16, 32, 64 and left once it passed 50. The FOR loop ran one query per channel. Loops are for orchestration, such as processing a list of tables; never loop over rows to do what one set based query can.

PART 20 · SCRIPTING

Dynamic SQL with EXECUTE IMMEDIATE

```
# 1 table name chosen at run time
DECLARE tbl STRING DEFAULT 'orders';
EXECUTE IMMEDIATE FORMAT(
  'SELECT COUNT(*) AS n FROM shop.%s', tbl);
```

```
# 2 values passed safely as parameters
EXECUTE IMMEDIATE
  'SELECT COUNT(*) AS n FROM shop.orders WHERE channel = @c'
USING 'web' AS c;
```

OUTPUTS

statement	n
1	8
2	5

Reading the outputs. Statement 1 built SQL text with FORMAT and counted all eight orders. Statement 2 kept the SQL fixed and bound web as the parameter @c, giving five web orders.

Security. Identifiers cannot be parameters, so validate any table name against an allow list, for example with INFORMATION_SCHEMA.TABLES, before formatting it into SQL.

Stored procedures: packaged, parameterised scripts

```
CREATE OR REPLACE PROCEDURE
  shop.refresh_top_customers(
    IN min_rev NUMERIC, OUT n_rows INT64)
BEGIN
  CREATE OR REPLACE TABLE shop.top_customers AS
  SELECT customer_id, SUM(value) AS revenue
  FROM shop.v_order_value
  GROUP BY customer_id
  HAVING revenue >= min_rev;
  SET n_rows = (SELECT COUNT(*) FROM shop.top_customers);
END;
```

```
DECLARE n INT64;
CALL shop.refresh_top_customers(500, n);
SELECT n AS customers_kept;
```

OUTPUT

customers_kept
3

Reading the output. Customers with at least 500 in valid revenue are Amara (1550), Elena (1200) and Ben (674). The OUT parameter carried the count back to the caller's variable.

Parameter modes: IN (default), OUT, and INOUT.

Spark procedures can also be written in PySpark and called the same way.

SQL user defined functions

```
# temporary: lives for this query or script only
CREATE TEMP FUNCTION clean_name(s STRING) AS (
  INITCAP(REGEXP_REPLACE(TRIM(s), r'\s+', ' ')));

# persistent: shared through the dataset
CREATE OR REPLACE FUNCTION shop.line_total(
  qty INT64, price NUMERIC)
RETURNS NUMERIC AS (qty * price);

SELECT
  clean_name(' ben CLARKE ') AS name,
  shop.line_total(4, NUMERIC '18.5') AS total;
```

OUTPUT

name	total
Ben Clarke	74

Reading the output. The cleaning logic from Part 11 is now one named function, and 4 × 18.5 = 74 matches order 5002. SQL UDFs are inlined into the query plan, so they cost nothing extra in performance.

Templated arguments with ANY TYPE make one function work across types.

Aggregate UDFs are created with CREATE AGGREGATE FUNCTION.

JavaScript UDFs: when SQL is not enough

```
CREATE TEMP FUNCTION luhn_ok(card STRING)
RETURNS BOOL
LANGUAGE js AS r"""
  let sum = 0;
  card.split('').reverse().forEach((c, i) => {
    let d = Number(c);
    if (i % 2 === 1) { d = d * 2; if (d > 9) d = d % 10 + 1; }
    sum += d;
  });
  return sum % 10 === 0;
""";

SELECT luhn_ok('79927398713') AS valid;
```

OUTPUT

valid
true

Reading the output. The Luhn checksum, used to validate card and account numbers, needs a digit by digit loop that is awkward in SQL. 79927398713 is the standard textbook valid example.

Costs: JavaScript runs in a sandbox per row, far slower than SQL, with memory limits. Use only for logic SQL cannot express, and load shared libraries from Cloud Storage with OPTIONS (library = …).

PART 20 · FUNCTIONS

Table functions: parameterised views

DDL AND USE

```
CREATE OR REPLACE TABLE FUNCTION
  shop.orders_by_channel(ch STRING)
AS
  SELECT order_id, status
  FROM shop.orders
  WHERE channel = ch;

SELECT *
FROM shop.orders_by_channel('app')
ORDER BY order_id;
```

OUTPUT · 3 ROWS

order_id	status
5002	delivered
5003	delivered
5006	shipped

Reading the output. A table valued function returns a table and is used in FROM like one. Here it returns the three app orders. Unlike a view, it accepts arguments, and they are applied inside the query so partition filters still prune.

Combine with authorised routines (Part 21) to give users filtered access without granting the base table.

PART 20 · FUNCTIONS

Remote functions: calling your own code from SQL




```
CREATE FUNCTION shop.vat_rate(country STRING) RETURNS NUMERIC
REMOTE WITH CONNECTION `acmeretail.eu.fn_conn`
OPTIONS (endpoint = 'https://vat.acme.example/rate', max_batching_rows = 500);
```

Use for: enrichment from internal services, custom ML models, libraries unavailable in SQL.

Batching: rows are sent in groups, so the endpoint must accept a list and return a list in the same order.

Watch: latency, endpoint scaling and the endpoint's own charges; cache results in a table where possible.

PART 20 · ERROR HANDLING

Catching errors with BEGIN … EXCEPTION

SCRIPT

```
BEGIN
  SELECT 1 / 0;
EXCEPTION WHEN ERROR THEN
  INSERT INTO ops.error_log (logged_at, message, stmt)
  VALUES (CURRENT_TIMESTAMP(),
          @@error.message,
          @@error.statement_text);
  SELECT @@error.message AS caught;
END;
```

OUTPUT

caught
Query error: division by zero: 1 / 0 at [2:3]

Reading the output. The failing statement did not end the script; control jumped to the handler, which logged the details and returned the message. @@error.message, @@error.statement_text and @@error.stack_trace describe what went wrong and where.

Rethrow with RAISE; inside the handler so schedulers still see the job as failed. Raise your own errors with RAISE USING message = '…'.

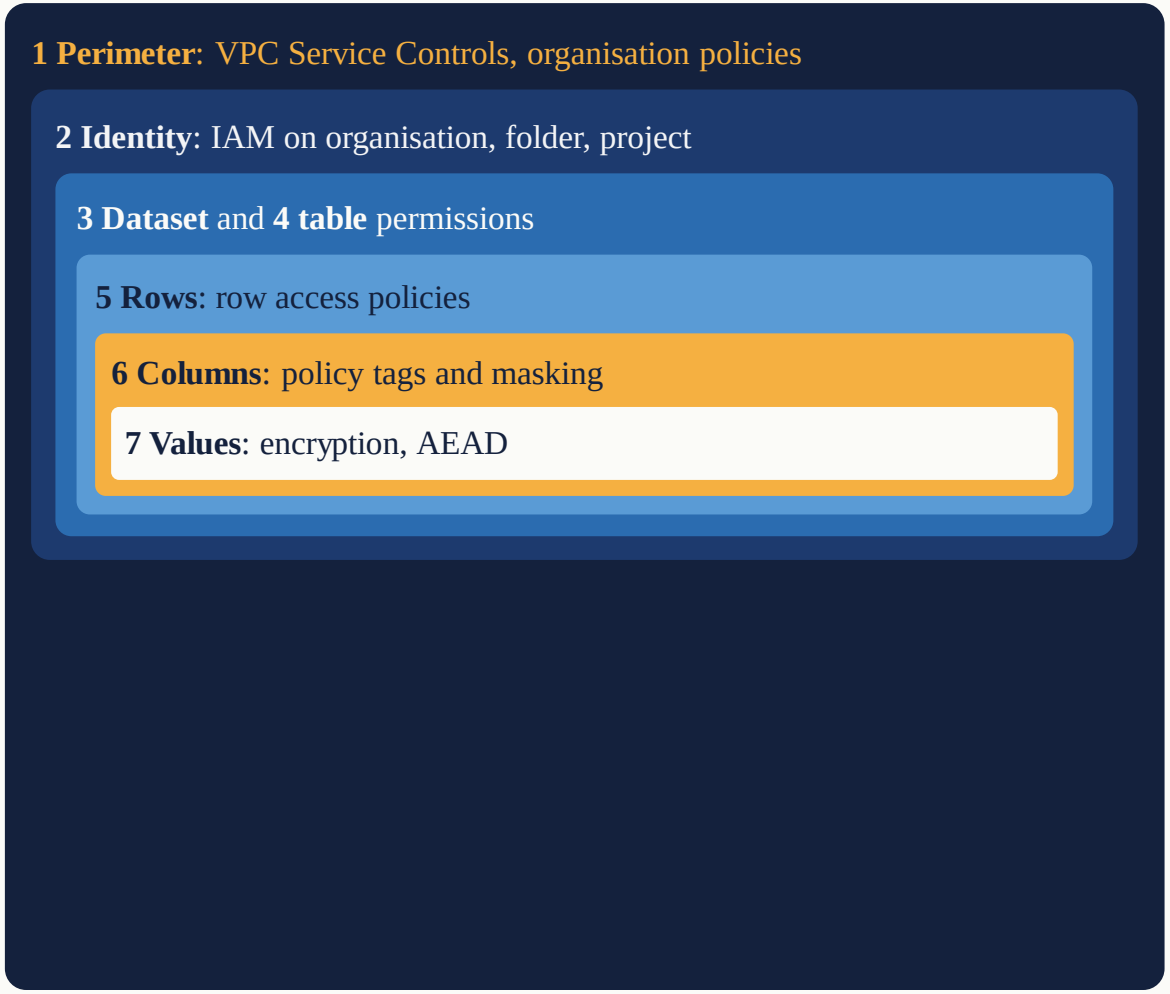
21

Security and Governance

Who can see which projects, datasets, tables, rows and columns, how sensitive values are masked or encrypted, and how every access is audited. Layered controls, all enforced by the platform.

Defence in depth · IAM roles · GRANT and REVOKE · Authorised views · Row level security · Column level security · Dynamic masking · Encryption · Auditing and lineage · Checklist

Defence in depth: seven nested layers



Every query passes through each layer from the outside in. A user must be allowed at every level to see a value.

Principle of least privilege: grant the narrowest role at the lowest level that does the job, to groups rather than individuals.

Separation of duties: those who manage data access are not automatically those who read the data.

The predefined IAM roles you will use most

Role	Can	Typical holder
bigquery.dataViewer	Read tables and metadata	Analysts, on datasets
bigquery.dataEditor	Read, insert, update, delete, create tables	Pipelines
bigquery.dataOwner	Everything on data plus sharing	Data product owners
bigquery.jobUser	Run jobs in a project, billed to it	Anyone who queries
bigquery.user	Run jobs, list and create datasets	Analysts, on projects
bigquery.metadataViewer	See schemas, not data	Catalogue browsers
bigquery.admin	Full control including reservations	Platform team only

The key insight. Querying needs two separate grants: permission to run a job somewhere (jobUser on a compute project) and permission to read the data (dataViewer on the dataset). That split is what lets one team pay for compute while another owns the data.

PART 21 · SECURITY

Granting access with SQL

```
# read access to a whole dataset, for a group
GRANT `roles/bigquery.dataViewer`
ON SCHEMA acmeretail.shop
TO "group:analysts@acme.co.uk";

# read access to one table, for one user
GRANT `roles/bigquery.dataViewer`
ON TABLE acmeretail.shop.products
TO "user:amara@acme.co.uk";

# take it away again
REVOKE `roles/bigquery.dataViewer`
ON TABLE acmeretail.shop.products
FROM "user:amara@acme.co.uk";
```

OUTPUTS

Access was granted to the schema shop.

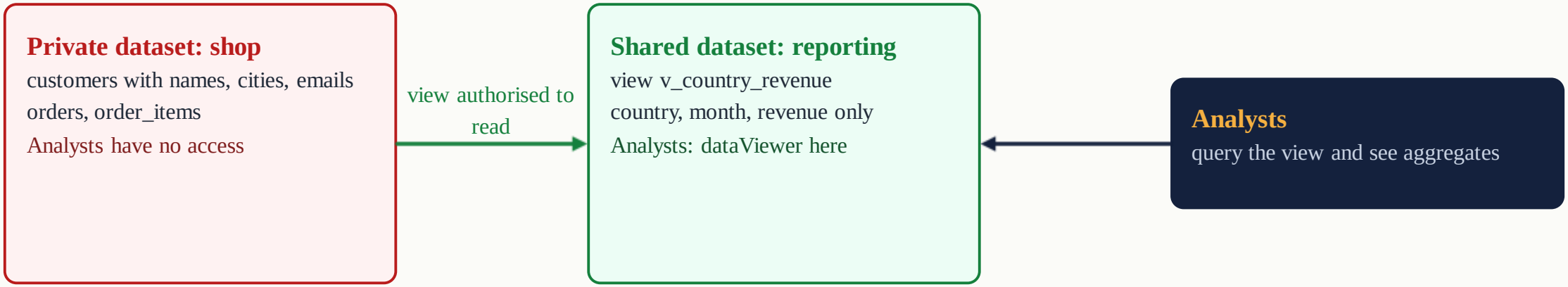
Access was granted to the table products.

Access was revoked from the table products.

Principals are written as user:, group:, serviceAccount: or domain: followed by the identity. Review effective grants with INFORMATION_SCHEMA.OBJECT_PRIVILEGES.

PART 21 · SECURITY

Authorised views: share results, not raw tables



How it works. You add the view to the private dataset's list of authorised views. The view can then read the source tables on behalf of its users, who never need, and never get, access to the private data. Authorised routines and authorised datasets extend the same idea to functions and to every view inside a dataset.

PART 21 · SECURITY

Row level security: same query, different rows

```
CREATE ROW ACCESS POLICY uk_only
ON shop.customers
GRANT TO ("group:uk_team@acme.co.uk")
FILTER USING (country = 'UK');
```

UK TEAM RUNS SELECT name, city FROM shop.customers

name	city
Amara Okafor	Manchester
Ben Clarke	London
Farid Haddad	Manchester

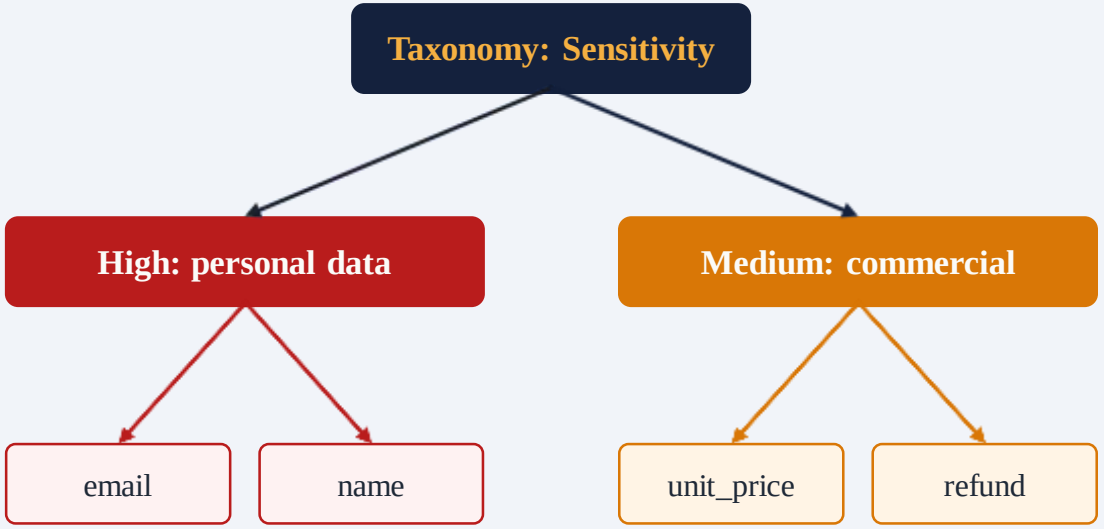
ANYONE NOT IN A POLICY RUNS THE SAME

name	city
(no rows)	

Reading the outputs. Once a table has any row access policy, users see only rows granted by a policy that includes them. The filter is applied invisibly to every query, view and export. Add a policy with FILTER USING (TRUE) for groups that need everything.

PART 21 · SECURITY

Column level security with policy tags



Tags are created once in a taxonomy and attached to columns in any table.

Without permission on a tag, SELECT email fails with Access Denied, while other columns stay readable. SELECT * EXCEPT (email) then works.

Granting the fine grained reader role on the High tag to the privacy team lets them read every High column everywhere.

Central control: one tag change instantly governs thousands of columns.

PART 21 · SECURITY

Dynamic data masking: see the shape, not the value

Data policies attached to a policy tag decide what a masked reader sees instead of an error. Example value: **amara@acme.co.uk**

Masking rule	Masked reader sees	Good for
Email mask	XXXXX@acme.co.uk	Domain level analysis
SHA256 hash	a stable 32 byte hash	Joining on the column without seeing it
Nullify	NULL	Removing the value entirely
Default value	an empty string	Keeping reports working
Last four characters	XXXXXXXXXXXXo.uk	Card or phone numbers
Custom routine	whatever your UDF returns	Bespoke policies

Reading the table. The same query returns real values to fine grained readers and masked values to masked readers, with no code change. Hashing preserves equality, so masked analysts can still count distinct customers or join tables on the hashed email.

PART 21 · SECURITY

Encryption: default, customer managed and in SQL

Default

All data is encrypted at rest and in transit with Google managed keys. Nothing to configure.

CMEK

Customer managed keys in Cloud KMS. You control rotation, and disabling the key makes the data unreadable.

AEAD functions

Encrypt individual values inside SQL with your own keysets.

```
# crypto shredding: one keyset per customer
SELECT customer_id,
  AEAD.ENCRYPT(k.keyset, c.email, CAST(c.customer_id AS STRING)) AS email_enc
FROM shop.customers AS c
JOIN secure.customer_keys AS k USING (customer_id);
```

Why per customer keys? To honour a deletion request, delete that customer’s keyset row. Every encrypted value for them, in every table and every backup, becomes permanently unreadable without rewriting history. The output column is BYTES ciphertext.

PART 21 · GOVERNANCE

Auditing, cataloguing and lineage



Data lineage, captured automatically from the jobs that read and write each table, answers “where did this number come from?” and “what breaks if I change this column?”

Cloud Audit Logs

Admin activity is always logged; data access logs record who read what. Route them into BigQuery itself for analysis.

INFORMATION_SCHEMA

JOBS, OBJECT_PRIVILEGES and TABLE_OPTIONS answer audit questions with plain SQL.

Dataplex catalog

Search, business glossary, data profiling and data quality scans across BigQuery and the lake.

Sharing

Analytics Hub and data clean rooms share data across organisations with controls on what can be queried.

PART 21 · RECAP

Governance checklist

Need	Control
Team sees its datasets only	Dataset level IAM granted to groups
Share aggregates, hide detail	Authorised views or authorised datasets
Regional managers see their region	Row access policies
Protect personal data columns	Policy tags with fine grained readers
Analysts work with but cannot see PII	Dynamic masking: hash or email mask
Own the encryption keys	CMEK; AEAD for crypto shredding
Stop data leaving the perimeter	VPC Service Controls
Prove who accessed what	Audit logs exported to BigQuery
Keep data only as long as allowed	Table and partition expiration

22

BigQuery ML and Generative AI

Train, evaluate, explain and serve machine learning models with SQL, forecast time series, and call Gemini models for text generation, classification and embeddings, all where the data already lives.

Why in database ML · Model types · Workflow · CREATE MODEL · ML.EVALUATE · Confusion matrix · ML.PREDICT · Explainability · Forecasting · Generative AI · Embeddings and vector search · Tuning and MLOps

Model outputs in this part are illustrative: they show the shape and meaning of real results.

PART 22 · BIGQUERY ML

Bring the model to the data, not the data to the model

Traditional path



- Data copied, governed twice
- Memory limits on large tables
- Separate skills and infrastructure

BigQuery ML path

CREATE MODEL → ML.EVALUATE → ML.PREDICT
all inside BigQuery

- No data movement; same IAM and audit
- Scales with BigQuery slots
- Analysts who know SQL can build models

Where it shines. Fast baselines, classic tabular problems such as churn, propensity and demand forecasting, batch scoring of millions of rows, and using foundation models over warehouse text. For bespoke deep learning, export or register models to Vertex AI.

The model menu

Problem	model_type	Example
Predict a number	LINEAR_REG, BOOSTED_TREE_REGRESSOR, DNN_REGRESSOR	Order value, delivery days
Predict a class	LOGISTIC_REG, BOOSTED_TREE_CLASSIFIER, RANDOM_FOREST_CLASSIFIER	Churn yes or no
Group similar rows	KMEANS	Customer segments
Recommend	MATRIX_FACTORIZATION	Products to suggest
Forecast time series	ARIMA_PLUS, ARIMA_PLUS_XREG	Daily revenue
Reduce or detect anomalies	PCA, AUTOENCODER	Fraud signals
Bring your own	TENSORFLOW, ONNX, XGBOOST imports	Models trained elsewhere
Foundation models	Remote models over Vertex AI and Gemini	Summaries, embeddings

Choosing. Start with a simple linear or logistic baseline, then try boosted trees, which usually win on tabular data. Model creation in BigQuery ML requires an Editions reservation tier that includes it, or on demand billing for training bytes.

The workflow in five SQL statements

1

Prepare

A feature table or view: one row per example, a label column

2

CREATE MODEL

Train with OPTIONS; automatic train and evaluation split

3

ML.EVALUATE

Metrics on held out data

4

ML.EXPLAIN

Global and per row feature attributions

5

ML.PREDICT

Score new rows in batch, write to a table

```
# the feature table used on the next slides
shop.customer_features (
  customer_id, tenure_days, orders_90d,
  avg_order_value, days_since_last_order,
  support_tickets, churned # label
)
```

Imagine this built from a much larger real customer base than our six sample customers: tens of thousands of rows, of which about 30% churned.

PART 22 · BIGQUERY ML

CREATE MODEL: training a churn classifier

SQL

```
CREATE OR REPLACE MODEL shop.churn_model
OPTIONS (
  model_type = 'LOGISTIC_REG',
  input_label_cols = ['churned'],
  auto_class_weights = TRUE,
  data_split_method = 'AUTO_SPLIT',
  enable_global_explain = TRUE
) AS
SELECT * EXCEPT (customer_id)
FROM shop.customer_features;
```

TRAINING TAB · ILLUSTRATIVE

iteration	train loss	eval loss
0	0.6931	0.6931
2	0.4512	0.4620
4	0.3468	0.3551
6	0.3274	0.3309

Reading the output. Loss starts at 0.6931, the log loss of guessing 50/50, and falls as the model learns. Training and evaluation loss stay close, a sign the model is not overfitting. Training stops when improvement becomes negligible.

auto_class_weights balances the 70/30 class imbalance; customer_id is excluded because an ID carries no predictive meaning.

PART 22 · BIGQUERY ML

ML.EVALUATE: how good is the model?

```
SELECT *
FROM ML.EVALUATE(MODEL shop.churn_model);
```

OUTPUT · ILLUSTRATIVE

metric	value
precision	0.81
recall	0.74
accuracy	0.87
f1_score	0.77
log_loss	0.33
roc_auc	0.90

Precision 0.81: of customers flagged as churners, 81% really churned.

Recall 0.74: of actual churners, the model caught 74%.

Accuracy 0.87: correct overall; misleading alone when classes are imbalanced.

F1 0.77: harmonic mean of precision and recall, 2PR ÷ (P + R).

ROC AUC 0.90: a random churner outranks a random loyal customer 90% of the time; 0.5 is guessing.

Metrics use the default 0.5 threshold; ML.ROC_CURVE shows the trade off at every threshold.

PART 22 · BIGQUERY ML

The confusion matrix behind those numbers

	Predicted churn	Predicted stay
Actual churn 300	**222** true positives	**78** false negatives
Actual stay 700	**52** false positives	**648** true negatives

Precision = 222 ÷ (222 + 52) = **0.81**

Recall = 222 ÷ (222 + 78) = **0.74**

Accuracy = (222 + 648) ÷ 1,000 = **0.87**

Business reading. 78 churners slip through unnoticed; 52 loyal customers receive a retention offer they did not need. Lowering the threshold catches more churners at the price of more false alarms.

From SELECT * FROM ML.CONFUSION_MATRIX(MODEL shop.churn_model), on an illustrative evaluation set of 1,000 customers.

PART 22 · BIGQUERY ML

ML.PREDICT: scoring customers

SQL

```
SELECT
  customer_id,
  predicted_churned,
  ROUND(p.prob, 2) AS churn_prob
FROM ML.PREDICT(
  MODEL shop.churn_model,
  (SELECT * FROM shop.customer_features_today)),
UNNEST(predicted_churned_probs) AS p
WHERE p.label = TRUE
ORDER BY churn_prob DESC;
```

OUTPUT · ILLUSTRATIVE

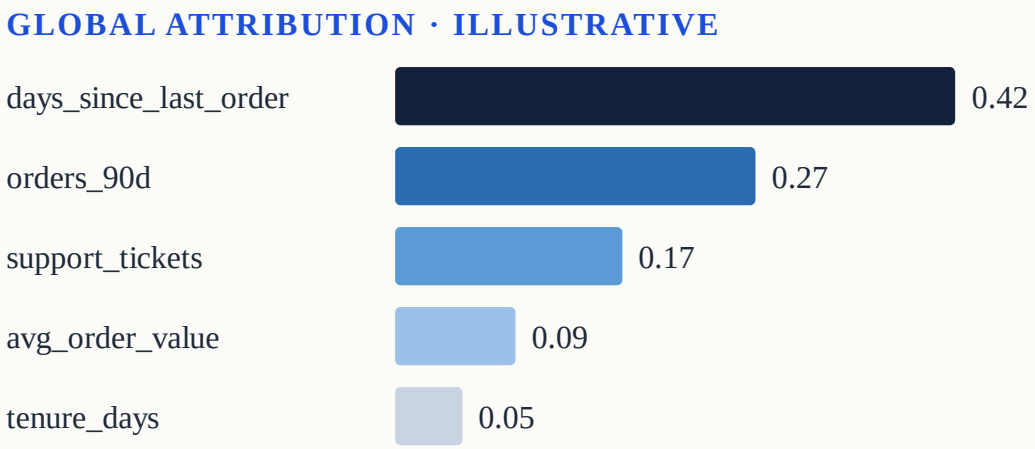
customer	predicted	churn_prob
4	true	0.71
6	true	0.64
3	false	0.46
5	false	0.31
2	false	0.22
1	false	0.08

Reading the output. ML.PREDICT adds predicted_churned and an array of class probabilities. UNNEST picks the probability of TRUE. Customers above 0.5 are flagged; Chloe at 0.46 is borderline and worth watching.

Explainability: which features drive churn?

```
SELECT feature, attribution
FROM ML.GLOBAL_EXPLAIN(MODEL shop.churn_model)
ORDER BY attribution DESC;
```

Reading the chart. Recency dominates: customers who have not ordered for a long time are the most likely to leave. Support tickets matter too. Tenure barely helps. Use ML.EXPLAIN_PREDICT for the same breakdown per individual customer, to tell an account manager why one person was flagged.



PART 22 · BIGQUERY ML

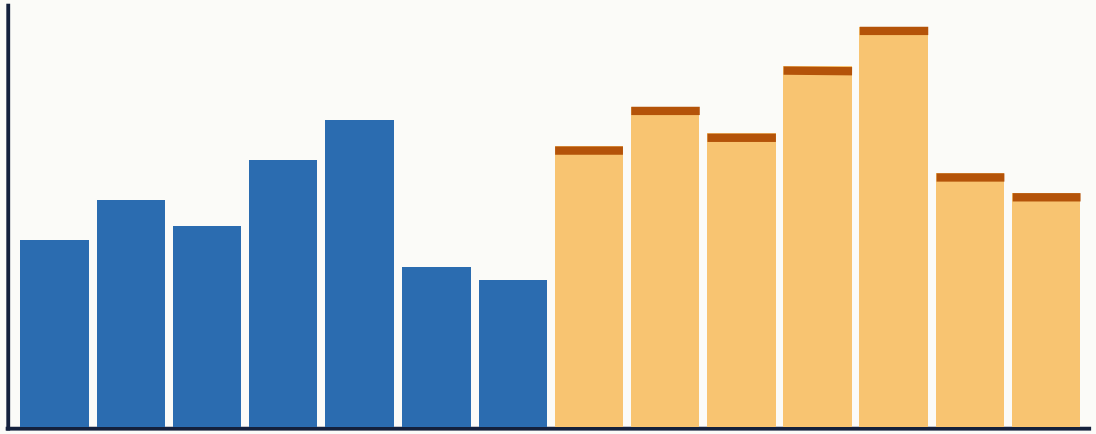
Forecasting daily revenue with ARIMA_PLUS

```
CREATE OR REPLACE MODEL shop.revenue_fc
OPTIONS (
  model_type = 'ARIMA_PLUS',
  time_series_timestamp_col = 'day',
  time_series_data_col = 'revenue',
  holiday_region = 'GB',
  horizon = 14
) AS SELECT day, revenue FROM shop.daily_revenue;

SELECT * FROM ML.FORECAST(MODEL shop.revenue_fc,
  STRUCT(14 AS horizon, 0.9 AS confidence_level));
```

Automatically handles trend, weekly and yearly seasonality, UK holidays, outliers and gaps.

HISTORY AND FORECAST · ILLUSTRATIVE



Blue: last 7 days actual. Amber: next 7 days forecast.

Each forecast row includes forecast_value plus lower and upper prediction interval bounds at 90% confidence. AI.FORECAST, backed by Google's TimesFM foundation model, can forecast without training a model first.

PART 22 · GENERATIVE AI

Gemini inside SQL: classifying review sentiment

```
# 1 a remote model pointing at a Gemini endpoint
CREATE OR REPLACE MODEL shop.gemini
REMOTE WITH CONNECTION `acmeretail.eu.vertex_conn`
OPTIONS (endpoint = '[current Gemini model name]');

# 2 one prompt per row
SELECT review_id, ml_generate_text_llm_result AS sentiment
FROM ML.GENERATE_TEXT(MODEL shop.gemini,
 (SELECT review_id, CONCAT(
  'Answer positive, neutral or negative: ',
  review_text) AS prompt
 FROM shop.reviews),
 STRUCT(0.0 AS temperature,
      TRUE AS flatten_json_output));
```

OUTPUT · ILLUSTRATIVE

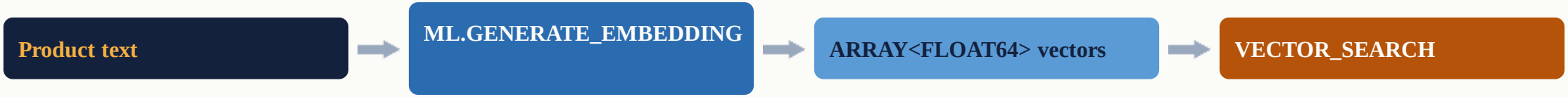
review_text	sentiment
Laptop arrived early, superb screen	positive
Chair squeaks after a week	negative
Coffee is fine, nothing special	neutral

Reading the output. Each row's prompt went to Gemini and the answer came back as a column. Temperature 0 makes answers as consistent as possible. Newer AI functions such as AI.GENERATE and AI.IF offer the same power with simpler syntax.

Costs are per token on the Vertex AI side; test prompts on a LIMITed sample first.

PART 22 · GENERATIVE AI

Embeddings and VECTOR_SEARCH: search by meaning




```
SELECT base.product_name, ROUND(distance, 2) AS distance
FROM VECTOR_SEARCH(
 TABLE shop.product_embeddings, 'embedding',
 (SELECT ml_generate_embedding_result AS embedding
  FROM ML.GENERATE_EMBEDDING(MODEL shop.embedder,
   (SELECT 'something to type my reports on' AS content))),
 top_k => 3,
 distance_type => 'COSINE');
```

OUTPUT · ILLUSTRATIVE

product_name	distance
Laptop Pro 14	0.18
Wireless Mouse	0.31
Standing Desk	0.44

Reading the output. No product mentions “type” or “reports”, yet the laptop is nearest in meaning, with the smallest cosine distance. This is the retrieval step of retrieval augmented generation; a vector index from Part 15 keeps it fast on millions of rows.

PART 22 · MLOPS

From experiment to production model

TRANSFORM clause
Bundle feature engineering such as ML.STANDARD_SCALER and ML.BUCKETIZE into the model, so training and prediction always match.

Hyperparameter tuning
num_trials and HPARAM_RANGE options search learning rates, depths and regularisation automatically.

Vertex AI Model Registry
model_registry = 'VERTEX_AI' registers versions for online endpoints and monitoring.

EXPORT MODEL
Writes the trained model to Cloud Storage for serving anywhere.

Scheduled scoring
A scheduled query writes ML.PREDICT output to a table every night for CRM and dashboards.

Drift checks
ML.VALIDATE_DATA_DRIFT compares new feature distributions with training data before you trust new scores.

23

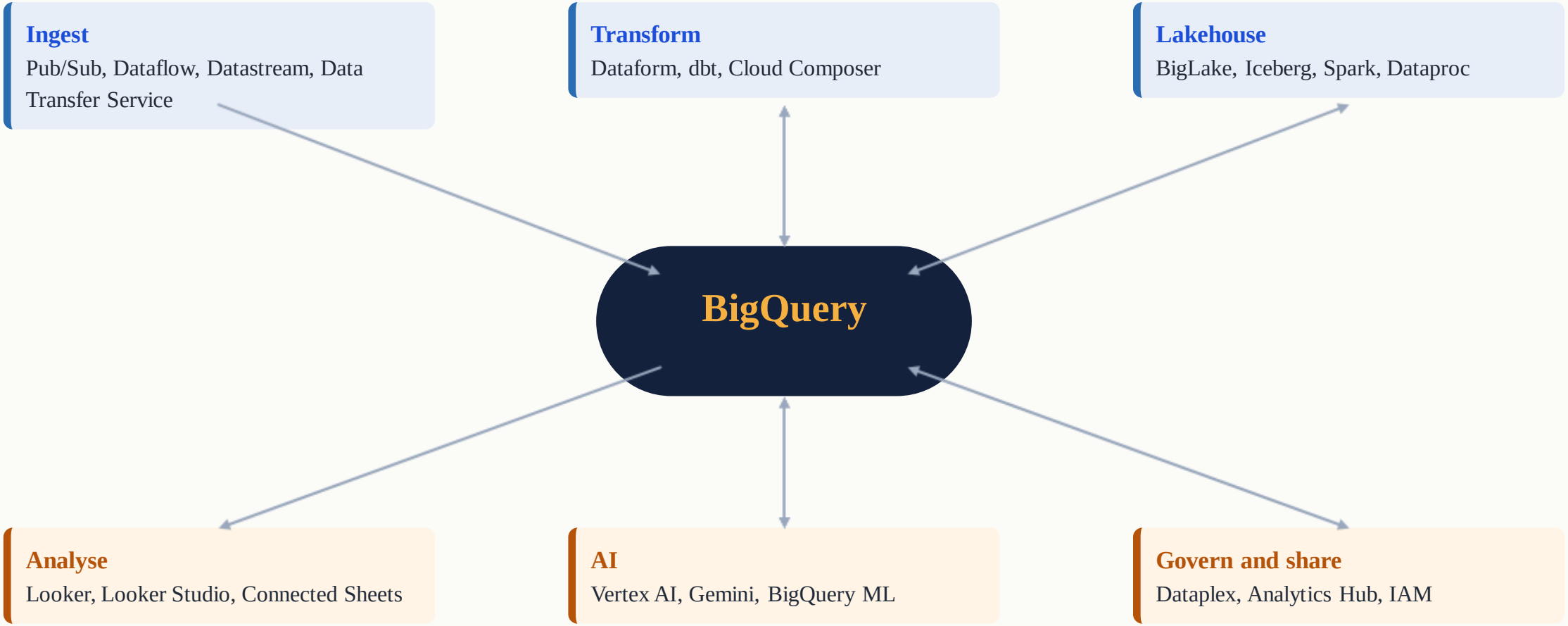
The Ecosystem and Automation

BigQuery rarely works alone. This part maps the services around it and shows how to automate pipelines with scheduled queries, Dataform, continuous queries, BI integrations and data sharing.

Ecosystem map · Scheduled queries · Dataform pipelines · Continuous queries and Pub/Sub · BI and dashboards · Sharing and multicloud

PART 23 · ECOSYSTEM

BigQuery at the centre of the data platform



PART 23 · AUTOMATION

Scheduled queries: SQL that runs itself

```
# runs every day at 06:00 Europe/London
DECLARE d DATE DEFAULT DATE_SUB(@run_date, INTERVAL 1 DAY);

DELETE FROM mart.daily_revenue WHERE day = d;
INSERT INTO mart.daily_revenue (day, orders, revenue)
SELECT d, COUNT(DISTINCT order_id), SUM(value)
FROM shop.v_order_value
WHERE order_date = d;
```

@run_date and **@run_time** are supplied automatically, so each run knows which day it is processing.

Delete then insert makes reruns and backfills idempotent: running a day twice leaves one copy.

Backfill any date range from the console; each day runs with its own @run_date.

Notifications by email or Pub/Sub on failure.

Scheduled queries run as a user or service account; use a service account so pipelines survive staff changes.

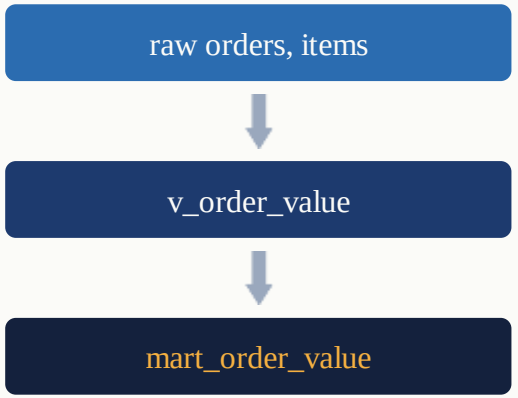
PART 23 · AUTOMATION

Dataform: SQL pipelines as version controlled code

definitions/mart_order_value.sqlx

```
config {
  type: "incremental",
  uniqueKey: ["order_id"],
  bigquery: { partitionBy: "order_date" },
  assertions: { nonNull: ["order_id"] }
}
SELECT order_id, customer_id, order_date, value
FROM ${ref("v_order_value")}
${ when(incremental(),
  "WHERE order_date > (SELECT MAX(order_date) FROM " + self() + ")") }
```

COMPILED DEPENDENCY GRAPH



ref() builds the dependency graph and runs steps in order. Incremental mode adds only new days; assertions fail the run if data quality breaks. Repositories live in Git inside BigQuery Studio.

PART 23 · REAL TIME

Continuous queries: SQL that never stops




```
EXPORT DATA OPTIONS (
  format = 'CLOUD_PUBSUB',
  uri = 'https://pubsub.googleapis.com/projects/acmeretail/topics/big_orders')
AS SELECT TO_JSON_STRING(STRUCT(order_id, customer_id, channel)) AS message
FROM shop.orders
WHERE channel = 'web';
```

Started with the continuous query option in the console or API; runs until cancelled.

Targets: Pub/Sub, Bigtable, Spanner or another BigQuery table, and can call Gemini per row.

Needs an Enterprise tier reservation with a continuous assignment.

PART 23 · BUSINESS INTELLIGENCE

From tables to decisions: the BI layer

Looker Studio

Free self service dashboards with a native BigQuery connector. Ideal for team reports. Use parameters and date ranges that map to partition filters.

Looker

An enterprise semantic layer in LookML: one governed definition of revenue reused by every dashboard, API and AI agent.

Connected Sheets

Pivot tables, charts and formulas over billions of rows from Google Sheets; computation runs in BigQuery.

BI Engine

In memory acceleration that makes interactive dashboards answer in under a second, with no change to SQL.

Design rule. Point dashboards at modelled mart tables or materialized views, never at raw event tables: faster, cheaper and consistent.

Sharing data beyond your project

Analytics Hub

Publish datasets as listings in exchanges. Subscribers get a linked, read only dataset: zero copy, always current, revocable.

Data clean rooms

Partners combine data for joint analysis, such as campaign overlap, under analysis rules that block row level extraction.

BigQuery Omni

Query data held in AWS or Azure storage with the same SQL, without moving it to Google Cloud first.

Public datasets: hundreds of free datasets hosted by Google, from weather to genomics, ready to join with your own data.

Who pays: the subscriber’s project pays for its queries; the publisher pays only for storage.

24

Capstone: an End to End Analysis

Everything together on the Acme sample data: a clean semantic view, trends, customer value, segmentation, category mix, a production reporting table, a schedule and a dashboard, ending with findings a manager can act on.

The brief · Clean base view · Monthly trend · Customer ranking · RFM segments · Category by channel · Reporting table · Automation and dashboard · Findings

PART 24 · CAPSTONE

The brief and the plan

> “How did we do from January to April, who are our most valuable customers, who might we lose, and which channel and categories deserve investment?”
>
> Head of Sales, Acme Retail

1 Clean base view	**2** Monthly trend	**3** Customer ranking	**4** RFM segments	**5** Category by channel	**6** Reporting table	**7** Schedule and dashboard

Parts used: views, joins, aggregation, windows, dates, CASE, PIVOT, DDL, partitioning, scheduling.

Definition agreed up front: revenue excludes cancelled orders; analysis date is 30 April 2026.

Control totals: 7 valid orders, 10 lines, £4,296 revenue, checked at every step.

PART 24 · STEP 1

A clean, wide base view at line grain

```
CREATE OR REPLACE VIEW mart.v_sales_lines AS
SELECT
  o.order_id, o.order_date, o.channel,
  c.customer_id, c.name, c.country, c.tier,
  p.category, p.product_name,
  i.quantity, i.quantity * i.unit_price AS revenue
FROM shop.order_items AS i
JOIN shop.orders AS o USING (order_id)
JOIN shop.customers AS c USING (customer_id)
JOIN shop.products AS p USING (product_id)
WHERE o.status != 'cancelled';

# control totals
SELECT COUNT(*) AS lines,
  COUNT(DISTINCT order_id) AS orders,
  SUM(revenue) AS revenue
FROM mart.v_sales_lines;
```

OUTPUT

lines	orders	revenue
10	7	4296

Reading the output. All three control totals match expectations: the joins neither lost nor duplicated rows. Every later step reads from this one view, so the business rule about cancellations lives in exactly one place.

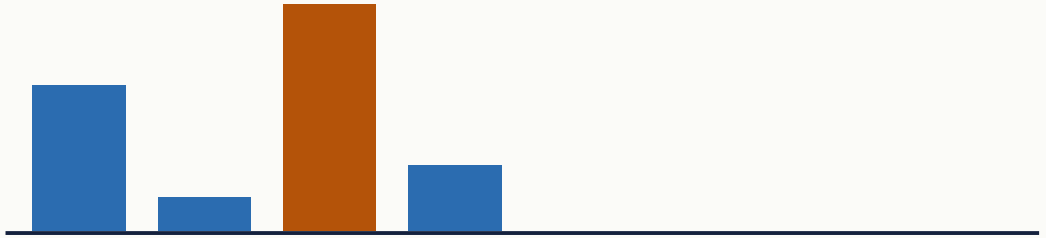
Grain: one row per valid order line. Write it in the view’s description.

PART 24 · STEP 2

Monthly trend with order value and running total

```
SELECT
  FORMAT_DATE('%b', DATE_TRUNC(order_date, MONTH)) AS mon,
  SUM(revenue) AS revenue,
  COUNT(DISTINCT order_id) AS orders,
  ROUND(SUM(revenue) /
    COUNT(DISTINCT order_id), 2) AS avg_order,
  SUM(SUM(revenue)) OVER (
    ORDER BY MIN(order_date)) AS running
FROM mart.v_sales_lines
GROUP BY mon
ORDER BY MIN(order_date);
```

mon	revenue	orders	avg_order	running
Jan	1324	2	662	1324
Feb	300	1	300	1624
Mar	2072	3	690.67	3696
Apr	600	1	600	4296



Reading the output. March is the strongest month in both revenue and orders. February is weak: one order, and the only February web order was cancelled. The window over an aggregate gives the running total in one pass.

PART 24 · STEP 3

Customer value ranking

```
SELECT
  name, tier,
  COUNT(DISTINCT order_id) AS orders,
  SUM(revenue) AS revenue,
  RANK() OVER (ORDER BY SUM(revenue) DESC) AS rnk,
  ROUND(100 * SUM(revenue) /
    SUM(SUM(revenue)) OVER (), 1) AS pct
FROM mart.v_sales_lines
GROUP BY name, tier
ORDER BY rnk;
```

name	tier	orders	revenue	rnk	pct
Amara Okafor	Gold	2	1550	1	36.1
Elena Rossi	Silver	1	1200	2	27.9
Ben Clarke	Silver	2	674	3	15.7
Chloe Martin	Gold	1	475	4	11.1
David Osei	Bronze	1	397	5	9.2



Reading the output. The top two customers generate 64% of revenue: classic concentration risk. Elena, only Silver tier, outspends Gold Chloe threefold, suggesting the tier rules no longer reflect value.

PART 24 · STEP 4

RFM segmentation: recency, frequency, monetary

```
WITH rfm AS (
  SELECT name,
    DATE_DIFF(DATE(2026, 4, 30), MAX(order_date), DAY) AS r,
    COUNT(DISTINCT order_id) AS f,
    SUM(revenue) AS m
  FROM mart.v_sales_lines GROUP BY name)
SELECT *,
  CASE
    WHEN r <= 30 AND f >= 2 THEN 'Champion'
    WHEN m >= 1000 THEN 'High value'
    WHEN r <= 45 THEN 'Active'
    ELSE 'At risk'
  END AS segment
FROM rfm ORDER BY m DESC;
```

name	r	f	m	segment
Amara Okafor	75	2	1550	High value
Elena Rossi	46	1	1200	High value
Ben Clarke	27	2	674	Champion
Chloe Martin	33	1	475	Active
David Osei	59	1	397	At risk

Reading the output. r is days since the last valid order as of 30 April. Ben is the only recent repeat buyer. Amara is the biggest spender but has been silent for 75 days: a priority for a personal call. David is at risk, matching the high churn score in Part 22.

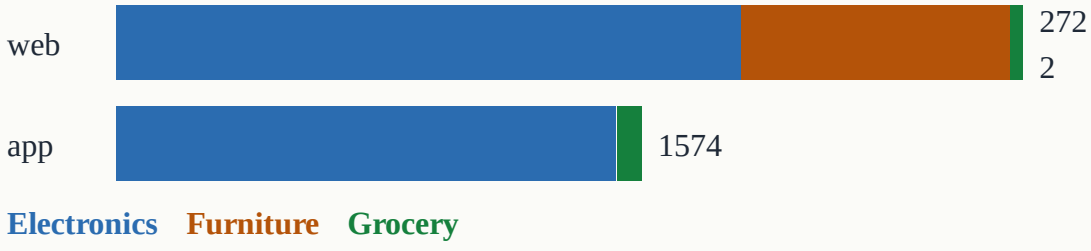
PART 24 · STEP 5

Category mix by channel

```
SELECT *
FROM (
  SELECT channel, category, revenue
  FROM mart.v_sales_lines)
PIVOT (SUM(revenue) FOR category
  IN ('Electronics', 'Furniture', 'Grocery'))
ORDER BY channel DESC;
```

channel	Electronics	Furniture	Grocery
web	1875	810	37
app	1500	NULL	74

Revenue by channel, stacked by category



Reading the output. Web brings 63% of revenue and all furniture sales; nobody has bought furniture in the app. Electronics dominates both channels. Opportunity: promote furniture in the app.

PART 24 · STEP 6

A production reporting table

```
CREATE OR REPLACE TABLE mart.sales_lines
PARTITION BY DATE_TRUNC(order_date, MONTH)
CLUSTER BY customer_id, category
OPTIONS (
  description = 'One row per valid order line',
  labels = [('owner', 'sales_analytics')]
)
AS SELECT * FROM mart.v_sales_lines;
```

OUTPUT

This statement created a new table named acmeretail.mart.sales_lines.

Why materialise? Dashboards query a stored table instead of rerunning four joins on every refresh.

Monthly partitions suit modest volumes, keeping partitions large; switch to daily as data grows.

Clustering on customer and category matches the filters the sales team uses most.

Description and labels make ownership and meaning discoverable.

Rerun the control totals against the table: 10 lines, 7 orders, 4296.

PART 24 · STEP 7

Automate it and put it on a dashboard

06:00 daily
scheduled query rebuilds mart.sales_lines

↓

Data quality checks
control totals; email on failure

↓

Looker Studio
reads the table, BI Engine accelerated

Acme sales overview · January to April 2026

Revenue	Orders	Avg order	Buyers
£4,296	7	£613.71	5



Segments

High value 2

Champion 1

Active 1

At risk 1

Monthly revenue Jan to Apr, peak in March

PART 24 · FINDINGS

What we tell the Head of Sales

1. **£4,296 from 7 valid orders**, average £613.71. March was strongest at £2,072; February weakest at £300.

2. **Concentration risk:** Amara and Elena provide 64% of revenue. Amara has been silent 75 days: call her.

3. **Tiers are misaligned:** Silver Elena outspends Gold Chloe; review tier rules using revenue.

4. **App gap:** zero furniture sales in the app; web carries 63% of revenue.

5. **Retention:** David is at risk; Farid signed up in January and has never ordered, a welcome offer candidate.

25

Reference and Close

Cheat sheets to keep beside you, interview style questions to test your understanding, and a path for what to learn next.

Clause cheat sheet · Function cheat sheet · Interview practice, parts one and two · Next steps

PART 25 · REFERENCE

GoogleSQL statement cheat sheet

Query clause	Purpose
WITH	Named subqueries
SELECT * EXCEPT / REPLACE	Columns, with shortcuts
FROM … JOIN … ON / USING	Sources and combinations
UNNEST	Arrays to rows
WHERE	Row filter
GROUP BY [ALL | ROLLUP]	Grouping
HAVING	Group filter
QUALIFY	Window filter
WINDOW	Named windows
ORDER BY … LIMIT	Sort and cap

Statement	Purpose
CREATE TABLE … AS	New table from a query
CREATE VIEW / MATERIALIZED VIEW	Saved or stored logic
ALTER TABLE	Evolve schema and options
INSERT / UPDATE / DELETE	Change rows
MERGE	Upsert and sync
LOAD DATA / EXPORT DATA	Files in and out
CREATE FUNCTION / PROCEDURE	Reusable logic
CREATE MODEL	Train ML
GRANT / REVOKE	Access
BEGIN TRANSACTION … COMMIT	Atomic changes

Logical order: FROM, WHERE, GROUP BY, HAVING, WINDOW, QUALIFY, SELECT, DISTINCT, ORDER BY, LIMIT.

PART 25 · REFERENCE

Function cheat sheet

Aggregate
COUNT, COUNTIF, SUM, AVG, MIN, MAX, STRING_AGG, ARRAY_AGG, ANY_VALUE, LOGICAL_OR

Window
ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, FIRST_VALUE, LAST_VALUE

Conditional
CASE, IF, IFNULL, COALESCE, NULLIF, GREATEST, LEAST

Conversion
CAST, SAFE_CAST, PARSE_DATE, FORMAT, FORMAT_DATE, TO_JSON_STRING

String
CONCAT, SUBSTR, STRPOS, SPLIT, TRIM, LOWER, UPPER, INITCAP, LPAD, REPLACE

Regex
REGEXP_CONTAINS, REGEXP_EXTRACT, REGEXP_EXTRACT_ALL, REGEXP_REPLACE

Date and time
CURRENT_DATE, EXTRACT, DATE_TRUNC, DATE_ADD, DATE_DIFF, LAST_DAY, GENERATE_DATE_ARRAY

Array
UNNEST, ARRAY_LENGTH, OFFSET, ORDINAL, ARRAY_CONCAT, ARRAY_TO_STRING, GENERATE_ARRAY

JSON
JSON_VALUE, JSON_QUERY, JSON_QUERY_ARRAY, PARSE_JSON, TO_JSON, LAX_INT64

Geography
ST_GEOGPOINT, ST_DISTANCE, ST_DWITHIN, ST_CONTAINS, ST_AREA

Approximate
APPROX_COUNT_DISTINCT, APPROX_QUANTILES, APPROX_TOP_COUNT, HLL_COUNT

ML and AI
ML.EVALUATE, ML.PREDICT, ML.FORECAST, ML.GENERATE_TEXT, VECTOR_SEARCH

PART 25 · INTERVIEW PRACTICE

Questions you should now answer confidently (1)

Why is BigQuery fast without indexes?
Columnar storage, compression, massive parallelism over slots, pruning by partition and cluster metadata, and an in memory shuffle.

Does LIMIT reduce cost?
Generally no. Bytes depend on columns and partitions read; LIMIT only trims returned rows.

Partitioning versus clustering?
One low cardinality column with exact pruning and lifecycle control, versus up to four columns of sorted blocks pruned at run time.

WHERE versus HAVING versus QUALIFY?
Rows before grouping, groups after aggregation, and rows after window functions.

RANK versus DENSE_RANK?
Both tie equal values; RANK leaves gaps afterwards, DENSE_RANK does not.

How do you remove duplicates?
QUALIFY ROW_NUMBER() OVER (PARTITION BY key ORDER BY ts DESC) = 1.

PART 25 · INTERVIEW PRACTICE

Questions you should now answer confidently (2)

When are Editions cheaper than on demand?
When monthly scanning is high and steady; 100 Enterprise slots cost about the same as 700 TiB on demand.

How do you share data without exposing PII?
Authorised views, row access policies, policy tags and dynamic masking.

What causes a join fan out?
Joining to a table at a finer grain, multiplying parent rows. Fix by aggregating first or counting distinct keys.

How do you recover deleted rows?
Time travel: query FOR SYSTEM_TIME AS OF a moment before the delete, within the window, and reinsert.

Why nest with ARRAY of STRUCT?
Child rows live with their parent, avoiding joins and shuffles for data always read together.

What does a slot do?
It is a unit of CPU, memory and network that executes one piece of a query stage at a time.

WHERE TO GO NEXT

Keep querying

Practise

Rebuild the sample dataset in a sandbox project and rerun every query, then try the public datasets.

Build

Turn the capstone into a Dataform project with tests, a schedule and a dashboard of your own.

Certify

Google Cloud’s Professional Data Engineer exam covers most of this material in depth.

Official reference: the BigQuery documentation and release notes on cloud.google.com, which change often. Check pricing and feature availability there before making commitments.